

Master Degree in
Applied and Computational Mathematics
2025-2026

Master Thesis

Physics-Informed and Data-Driven
Neural Networks for Option Pricing,
Calibration, and Greeks Estimation
under the Heston Model

Jose Enrique Zafra Mena

Jorge Morón Vidal

Francisco Manuel Bernal Martínez

Madrid, 2026

AVOID PLAGIARISM

The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

SUMMARY

Neural methods offer a way to accelerate Heston option analytics, but their usefulness depends on whether they remain reliable when pricing, calibration, and Greek estimation are treated as connected tasks. Previous work has shown that neural surrogates can approximate option prices or implied-volatility surfaces efficiently. However, small pricing errors do not necessarily imply stable calibration or accurate sensitivities, especially near short maturities and at-the-money contracts, where the pricing surface changes rapidly.

This thesis studies European put options under the Heston model through a controlled synthetic benchmark. We first build a deterministic numerical reference for prices, implied volatilities, and Greeks, and then use it to evaluate two neural branches: a supervised ANN branch for implied-volatility learning and calibration, and a physics-informed PINN branch for direct price-surface approximation.

The results show that the supervised ANN branch gives the strongest performance for calibration and Greek estimation: Sobolev training substantially improves derivative accuracy, and the Sobolev-based CaNN provides the best calibration fit on the replicated benchmark. The PINN branch has the advantage of learning without price labels and provides a fast differentiable pricing surface, but its Greek estimates are more fragile; Sobolev and analytic control variate corrections reduce this weakness, especially in difficult short-maturity at-the-money regions.

Keywords: Heston model; derivative pricing; calibration; neural networks ;physics-informed neural networks; Greeks; Sobolev training

DEDICATION

CONTENTS

1. INTRODUCTION.	1
1.1. Motivation	1
1.2. Problem Setting	3
1.3. Methodological Approach and Contributions	4
1.4. State of the Art	5
1.5. Scope and Thesis Structure	7
2. MATHEMATICAL AND FINANCIAL BACKGROUND	9
2.1. Mathematical Framework For Option Pricing	9
2.2. Probability Theory	11
2.3. Stochastic Calculus Background	12
2.4. The Black–Scholes Model and Implied Volatility.	12
2.5. The Heston Stochastic Volatility Model	13
2.6. Pricing Via Characteristic Functions	15
2.7. The Fourier-COS method	16
2.7.1. The COS Method Applied To The Heston Model.	17
2.7.2. Semi-Analytical Greeks from the Heston Characteristic Function.	19
2.8. Implied Volatility And Numerical Inversion.	20
3. SUPERVISED NEURAL METHODS FOR HESTON PRICING AND CALI- BRATION	21
3.1. Problem Formulation	22
3.2. Synthetic Dataset And Supervised Learning Vocabulary	23
3.2.1. Parameter Ranges And Constraints.	24
3.2.2. Dataset Splitting And Preprocessing	24
3.2.3. Numerical Label Generation (COS + Brent)	25
3.3. Neural Network Architecture	25
3.3.1. Model Class	26
3.3.2. Architecture Details	26
3.3.3. Output Interpretation.	27

3.4. Training Procedure	28
3.4.1. Baseline ANN Loss Function	29
3.4.2. Sobolev ANN Loss Function	29
3.4.3. Optimizer And Hyperparameters	30
3.4.4. Regularization And Callbacks	31
3.5. Neural Calibration As An Inverse Problem	32
3.5.1. Calibration Setup	32
3.5.2. Search And Stability Checks	33
4. PHYSICS-INFORMED NEURAL PRICING UNDER THE HESTON MODEL.	35
4.1. Problem formulation	35
4.1.1. PINN Theoretical Framework.	36
4.2. Heston Pricing PDE	37
4.3. Neural Network Architecture	38
4.3.1. Model Class	38
4.3.2. Architecture Details	39
4.3.3. Model Inputs and Outputs	40
4.4. Training Procedure	41
4.4.1. Loss Function.	41
4.4.2. Optimizer and Hyperparameters	43
4.4.3. Sampling strategy	43
4.5. Sobolev-Enhanced Training for Greek Accuracy	44
4.6. Analytical Control Variate Correction	46
4.7. Greeks Computation	48
5. NEURAL NETWORK PERFORMANCE AND BENCHMARKING.	50
5.1. Experimental Setup And Evaluation Protocol	50
5.1.1. Primary Error Metrics	50
5.2. Supervised ANN Benchmarks	51
5.2.1. ANN Baseline	52
5.2.2. Sobolev Methods: IV Accuracy And Derivative-Aware Training	55
5.2.3. CaNN: Calibration With The ANN Surrogate	58

5.3. Unsupervised PINN Benchmarks.	61
5.3.1. PINN Baseline	61
5.3.2. Sobolev Methods: Greek Improvement And Pricing Trade-Off	62
5.3.3. PINN Sobolev+ACV: Local Hard-Region Diagnostic	67
5.4. Computational Cost And Model Roles.	69
6. FINAL DISCUSSION	71
6.1. Main Conclusions	71
6.2. Future Work	73
BIBLIOGRAPHY.	74
A. ANALYSIS OF THE COS METHOD ERROR	
A.1. COS Error Control.	
A.2. Reference Configuration	
B. PINN PRICING BENCHMARK SETUP.	
C. SOFTWARE IMPLEMENTATION.	
C.1. Computational Environment	
C.2. Code Excerpts	
C.3. Repository	
D. GLOBAL NOTATION GLOSSARY	

LIST OF FIGURES

1.1	High-level structure of the benchmarked Heston workflow. The ANN branch is supervised and learns the implied-volatility surface, with baseline and Sobolev variants feeding the calibration and Greek diagnostics. The PINN branch learns the price surface from the Heston PDE, and is evaluated through baseline, Sobolev, and Sobolev+ACV variants. Solid arrows show the main computational flow; dashed arrows show how the reference Greeks are injected into the Sobolev-trained models as derivative targets.	4
2.1	Normalized terminal payoff of a European put. The holder has the right, but not the obligation, to exercise at maturity; therefore the payoff is positive only when $S_T/K < 1$ and is zero for $S_T/K \geq 1$	10
3.1	Fully connected ANN pricer used in the forward Heston implied-volatility surrogate. The eight fixed inputs are mapped through four $\tanh$ hidden layers of width 200, and the final linear layer returns the predicted implied volatility.	27
4.1	PINN architecture used for Heston pricing. The central MLP block represents the four hidden layers of width 256 with $\tanh$ activation, followed by a linear output layer. It maps the eight-dimensional input $(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$ to the scalar price V_ϕ . The right-hand branches show how this same output is used during training and evaluation: automatic differentiation gives the derivatives entering the PDE residual, direct evaluation at terminal and boundary points gives the contractual losses, and automatic differentiation of the trained price surface gives the Greeks.	39
5.1	Sensitivity grid for the ANN pricer. Top row: $(\gamma, \bar{v})$, middle row: (ρ, γ) , bottom row: $(\kappa, \bar{v})$. Left panels report IV-value maps with white isoclines; right panels report $\log \text{NN} - \text{Heston} $, where lighter tones indicate lower error and darker tones higher error. Dashed black lines indicate interpolation limits.	54

5.2	Baseline–Sobolev ANN Greek error histograms. Overlaid density histograms of the pointwise Greek errors summarized in Table 5.3, comparing the ANN baseline with the ANN Sobolev surrogate on the fixed (m, τ) benchmark grid. The five panels correspond to Delta, Gamma, Vega, Theta, and Rho; the horizontal axis reports the error with sign convention ANN minus reference, and the vertical axis reports density on a logarithmic scale.	56
5.3	Relative absolute-error maps for ANN-derived Greeks. The two columns compare the ANN baseline and the ANN Sobolev surrogate on the fixed (m, τ) benchmark grid used for the Greek errors summarized in Table 5.3. Rows correspond to Delta, Gamma, Vega, Theta, and Rho; the horizontal axis is maturity τ , the vertical axis is moneyness m , and the shared colorbar reports relative absolute error on a logarithmic scale.	57
5.4	Baseline PINN pricing benchmark versus reference	62
5.5	Sobolev PINN pricing benchmark versus reference	63
5.6	Baseline–Sobolev PINN Greek error histograms	65
5.7	Baseline–Sobolev PINN Greek relative-error maps	66
5.8	Kink-region PINN Greek relative-error maps	68

LIST OF TABLES

1.1	Methodological blocks by role in this thesis	7
2.1	Interpretation of Heston parameters	14
3.1	Input ranges used for synthetic data generation	24
3.2	Neural network architecture used for the ANN pricer	27
3.3	Hyperparameters and training controls of the selected ANN configurations	32
4.1	PINN architecture used in this chapter	40
5.1	ANN baseline optimizer selection.	52
5.2	ANN baseline global IV errors.	52
5.3	ANN Greek RMSE.	55
5.4	Quote universe used in the CaNN benchmark.	58
5.5	CaNN calibration fit.	59
5.6	CaNN calibration results using ANN baseline and ANN Sobolev surro- gates versus Liu et al. On the replicated 35-quote benchmark, all quote weights are equal to one, so the reported weighted MSE coincides with the mean squared IV residual.	59
5.7	CaNN local curvature.	60
5.8	CaNN Sobolev Hessian.	60
5.9	PINN Greek errors.	64
5.10	Hard-region RMSE for the PINN variants.	69
5.11	RMSE summary for the final models.	69
D.1	Global notation glossary: core pricing and model notation	
D.2	Global notation glossary: neural benchmark notation	
D.3	Global notation glossary: transform, COS, and IV notation	

1. INTRODUCTION

1.1. Motivation

Financial derivatives are financial contracts whose value is linked to the behaviour of an underlying market variable, such as an asset price, an interest rate, an index level, or an exchange rate [1]. The payoff of the contract is then determined by how that underlying variable evolves up to the relevant exercise or maturity date. Options are one of the most common examples: they give the holder the right, but not the obligation, to buy or sell the underlying asset at a fixed strike price K , with the contract ending at a specified maturity date T . A European option can only be exercised at that maturity date, which makes it simple to study in a controlled way but still rich enough to show the main numerical difficulties of derivative pricing.

At a high level, derivative pricing is inherently a stochastic problem because the underlying assets are modelled as stochastic processes. The uncertainty in future asset prices is therefore built directly into the mathematical framework used for valuation. While the formal machinery of stochastic calculus is introduced in Chapter 2, the key point for now is that randomness is an intrinsic feature of the pricing model rather than an external source of noise.

In practice, option pricing is not only a question of computing one price. The same pricing model is often used many times: to generate prices for different strikes and maturities, to fit model parameters to quoted option prices, and to measure how sensitive the price is to change in the inputs. These sensitivities, commonly referred to as Greeks, play a crucial role in the financial industry by enabling analysts to measure and manage the risks to which their portfolios are exposed. Greeks also help identify and monitor pricing errors that may become increasingly significant in calibration or risk-management applications.

A classical reference point is the Black-Scholes model [2]. It gives an explicit formula for European option prices in a simplified setting where volatility is assumed to be fixed. Here, volatility can be understood as the parameter that controls the intensity of the asset price fluctuations in the model. The model is still central because it provides a common reference for comparing option prices across strikes and maturities. In later chapters, this idea will also be connected to the market convention of expressing option prices through an equivalent volatility level. The limitation of Black-Scholes is that a single fixed volatility is usually not flexible enough to represent option prices across different strikes and maturities.

This work uses the Heston model [3] because it relaxes this constant-volatility assumption while keeping the problem numerically tractable. In Heston, the variance of

the asset is itself modelled as a random process. This allows the model to represent richer price patterns than Black-Scholes, especially when prices vary across strikes and maturities. At the same time, Heston keeps enough mathematical structure to be priced numerically by the Fourier-COS method introduced in Chapter 2. This extra flexibility also makes calibration more difficult. In this context, calibration means choosing the model parameters so that the model reproduces a given set of option quotes. Under Black-Scholes, the main volatility input is much simpler to handle; under Heston, several interacting parameters must be fitted at the same time.

A standard way to compute the price of derivatives when no analytical solution or other numerical methods are available are Monte Carlo methods [4]. The idea is to simulate many possible future paths of the underlying asset and average the resulting discounted payoffs. This approach is flexible because it can be adapted to many payoff structures and model dynamics. However, the probabilistic nature of Monte Carlo methods leads to statistical errors that can only be reduced through a large number of simulations [5], making these methods computationally expensive in repeated-use settings.

Explicit pricing formulas suitable for deterministic methods such as COS are available only in restricted cases. This thesis deals with one of those favored cases: European put options, where deterministic Heston-COS reference is more appropriate. The experiments we are going to perform require repeated pricing calls, implied-volatility generation, calibration tests, and Greek benchmarks. This repetition can make the numerical experiment considerably more expensive, because the reference pricer is called many times across contract grids, calibration routines, and sensitivity diagnostics. This repeated-use setting is where neural approximations become attractive.

In this project, neural approximations enter the workflow through two different routes. One route learns the implied-volatility representation produced by the numerical reference from generated training examples, which gives the supervised ANN setting. The other route learns the option price surface more directly, using the structure of the Heston pricing problem during training, which gives the PINN setting. This distinction matters because the two models are trained in different output spaces and their errors must be interpreted differently.

A neural network can be trained to approximate a numerical pricing routine and then evaluated very quickly. The idea is to pay the cost of generating accurate reference data once, and then train a model that can be evaluated much faster inside calibration or sensitivity workflows. This is the motivation behind neural pricing and calibration approaches such as [6].

However, speed alone is not sufficient in a financial workflow. A learned component is useful only if its errors can be measured, if its weak regions can be detected and if its outputs remain reliable when used in calibration or sensitivity analysis. The question this thesis aims to answer is not whether neural networks can be fast, but whether they can be assessed in a way that is meaningful for a pricing workflow. The next section makes this

context precise by defining the benchmark problem, the role of the Heston model, and the downstream task used to evaluate reliability

1.2. Problem Setting

A put option gives value to its holder when the underlying asset finishes below the strike price. In the European case considered here, the option is evaluated only at the maturity date. This setting is deliberately restricted: the payoff rule is simple, the exercise date is fixed, and the numerical reference can be built under controlled assumptions. At the same time, the model is expressive enough to generate non-trivial behaviour across strikes, maturities, and volatility regimes.

The problem is therefore not the complexity of the contract itself, but the reliability of the computational workflow built around it. The project studies three connected tasks: approximating the outputs of the Heston pricing routine, using those approximations inside calibration of the model parameters, and evaluating the quality of the resulting sensitivities. Each of these tasks can fail in a different way. A neural surrogate may have a small average pricing error but still introduce structured errors across moneyness¹ or maturity. A calibration routine may fit observed quotes while recovering unstable parameter values. A differentiable pricing model may produce reasonable prices while giving inaccurate Greeks.

This is especially relevant in regions where the pricing surface, understood here as the option price as a function of moneyness and maturity, changes rapidly. The most delicate cases in this thesis are short maturities and near-the-strike contracts, where the current underlying price is close to the strike. This ATM region is especially relevant because small movements in the underlying can produce large changes in local curvature. For a European put, the payoff changes slope abruptly at the strike: below K it increases as the asset price falls, while above K it is zero. This sharp corner is often called a payoff kink. Near maturity, it makes the pricing surface harder to approximate, especially for second-order sensitivities such as Gamma.

For this reason, the thesis treats benchmarking as part of the problem itself. Neural components are not evaluated only by speed or by global error metrics, but also by their behaviour in the downstream tasks for which they are intended. The central question is whether these components can be useful inside a Heston pricing workflow without hiding errors that later affect calibration or Greek computation.

¹Moneyness, denoted by m , measures the relative position of the underlying price with respect to the strike. For the European put options considered in this thesis, at-the-money (ATM) contracts have an underlying price close to the strike, in-the-money (ITM) contracts have an underlying price below the strike, and out-of-the-money (OTM) contracts have an underlying price above the strike.

1.3. Methodological Approach and Contributions

The proposed methodology is built around a benchmarked Heston workflow that first constructs a deterministic numerical reference and then evaluates neural approximation methods against it. This numerical reference has two roles: it provides accurate option values and Greek benchmarks, and it generates the controlled data used to train and test the machine-learning components. This makes it possible to evaluate neural methods under explicit numerical conditions and to distinguish computational acceleration from genuine loss of reliability.

Using this reference as the common benchmark, we study two families of neural network approaches. The first is a supervised artificial neural network (ANN) surrogate trained to approximate the implied-volatility surface generated from the Heston reference. This branch contains the ANN baseline and its Sobolev-augmented version. This surrogate is then used inside a calibration routine, where the objective is to recover model parameters efficiently from option quotes. The second approach is a physics-informed neural network (PINN) trained without direct price labels and used to approximate the option price surface directly from the Heston pricing equation. Since this learned price surface is differentiable, it can also be used to compute Greeks through automatic differentiation [7]. In this branch, we compare the PINN baseline with a Sobolev-augmented PINN and with a further analytic control variate (ACV) correction, which uses an analytic reference component to improve the stability of the learned sensitivities. The complete structure of this benchmarked workflow is summarized in Figure 1.1.

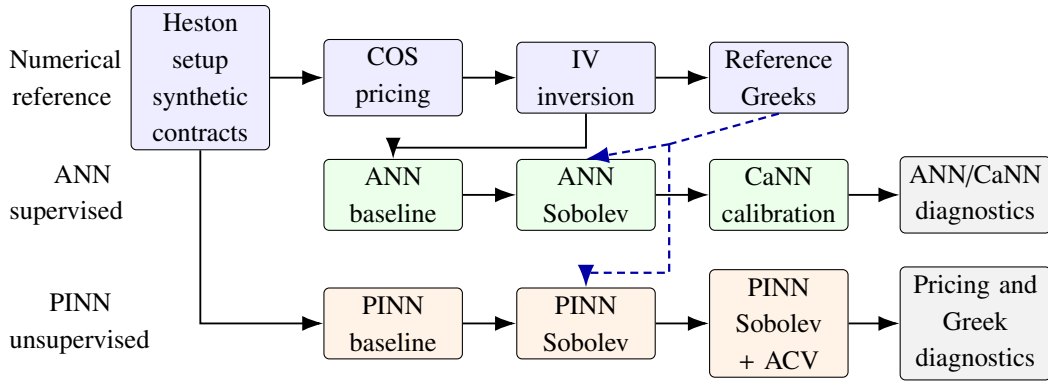


Fig. 1.1. High-level structure of the benchmarked Heston workflow. The ANN branch is supervised and learns the implied-volatility surface, with baseline and Sobolev variants feeding the calibration and Greek diagnostics. The PINN branch learns the price surface from the Heston PDE, and is evaluated through baseline, Sobolev, and Sobolev+ACV variants. Solid arrows show the main computational flow; dashed arrows show how the reference Greeks are injected into the Sobolev-trained models as derivative targets.

A particular focus is placed on the quality of these Greeks. Sensitivity computation is therefore not treated as a secondary output of the pricing model, but as a separate diagnostic problem. For this reason, the final part of the workflow studies two complementary

ways of improving or analysing Greek accuracy. One introduces derivative information during training, while the other uses an analytic reference component to correct or diagnose the learned sensitivities. These two ideas are later developed as the Sobolev-enhanced and analytic-control-variate experiments.

The main contribution is the construction and evaluation of this full workflow under a common benchmark. The ANN block studies a supervised implied-volatility surrogate, including both the baseline model and its Sobolev-augmented version, and then uses this surrogate family inside the CaNN calibration problem. The PINN block follows a different route: it studies an unsupervised price-surface approximation through the baseline, Sobolev, and Sobolev+ACV variants. This separation is deliberate, because the two families are trained in different spaces and are not meant to be compared as interchangeable models. The final experiments analyse the regions where Greek estimation is most difficult and assess how the derivative-aware and analytic-correction strategies behave there.

This organization is important for the interpretation of the results. The ANN is judged not only as a fast function approximator, but also as the forward engine used by the calibration routine. The PINN is judged not only by price error, but also by spatial error structure and derivative quality. The Greek analysis is treated as a separate problem because automatic differentiation through a learned price surface does not by itself guarantee reliable sensitivities.

1.4. State of the Art

The literature relevant to this thesis sits at the intersection of numerical option pricing, model calibration, and neural approximation. Classical model-based pricing provides the financial foundation. The Black–Scholes model gives a closed-form benchmark under constant volatility [2], while the Heston model extends this framework by introducing stochastic variance and semi-analytical pricing formulas [3]. Simulation-based approaches, especially Monte Carlo methods, are also a central reference in derivative pricing because of their flexibility across payoffs and dynamics [4]. Their main drawback in the present setting is that repeated simulation can be expensive and introduces sampling noise, which is inconvenient when the objective is to build deterministic benchmarks for calibration and Greek diagnostics.

Fourier and spectral methods provide another important line of work. When the characteristic function of the model is available, transform methods can price options accurately without simulating paths. FFT-based pricing [8] and the Fourier-COS method [9] are standard examples of this approach. The COS method is especially useful here because it gives a deterministic and accurate reference layer for repeated pricing, implied-volatility generation, and benchmark construction. Its performance, however, still depends on numerical choices such as the truncation interval, the number of expansion terms, and the stability of the implied volatility inversion.

Neural methods enter this landscape as acceleration and approximation tools. Feed-forward ANN surrogates can learn pricing or implied-volatility maps from synthetic labels and then evaluate them much faster than the numerical solver, which makes them attractive for the ANN/CaNN calibration workflow followed in this thesis. Liu et al. [6] are therefore used as the closest reference for the supervised ANN calibration branch. Physics-informed neural networks follow a different route: instead of learning only from labels, they use the pricing equation itself as part of the training objective. Recent work applies this idea to Heston-type option pricing and related PDE settings [10], [11]. More recent operator-learning work for stochastic-volatility models also reports a useful warning for the present thesis: highly accurate Heston prices can still lead to noisy autodiff Greeks, especially near the at-the-money region [12]. At the same time, the PINN literature reports well-known difficulties with optimization, loss balancing, gradient flow, and robustness across the training domain [13], [14], [15].

For Greek computation, automatic differentiation provides a natural way to obtain sensitivities from differentiable neural surfaces [7], although accurate prices do not automatically imply accurate derivatives. This motivates derivative-aware training strategies such as Sobolev-type supervision [16], deep differential learning of Heston parameter sensitivities [17], and broader derivative-informed operator-learning methods for financial surfaces [18]. In this thesis, these references frame the Sobolev experiments in both neural branches. They are complemented by analytic control variates, which are used as an additional correction and diagnostic mechanism connected to the variance-reduction tradition in Monte Carlo methods [5].

Table 1.1 summarizes these methodological blocks and clarifies their role in this thesis. The table also motivates the structure of the proposed workflow: COS pricing is kept as the deterministic reference, ANN and CaNN components are used for fast calibration, and PINN-based methods are evaluated as differentiable pricing surfaces whose Greek quality must be diagnosed separately.

TABLE 1.1. METHODOLOGICAL BLOCKS BY ROLE IN THIS THESIS

Block	Representative methods	Role in this thesis	Main limitations
Numerical reference	Black–Scholes, Heston, COS pricing [2], [3], [9]	Deterministic prices, implied-volatility labels, and Greek references	Requires careful numerical setup and stable inversion
ANN surrogate family	ANN baseline, ANN Sobolev, CaNN calibration [6], [16]	Supervised implied-volatility surrogate and calibration block	Depends on labelled synthetic data and interpolation-domain validity
PINN family	PINN baseline, PINN Sobolev, PINN Sobolev+ACV [10], [11], [12]	Unsupervised price-surface approximation and Greek diagnostics	Sensitive optimization, loss balancing, and Greek stability [12], [13], [14], [15]
Greek diagnostics and correction	Automatic differentiation, Sobolev training, differential learning, control variates [5], [7], [16], [17], [18]	Framework for computing, evaluating, and improving sensitivities	Derivative accuracy depends on surface smoothness and target quality; control variates are problem-specific

Source: Author’s synthesis from the cited literature

1.5. Scope and Thesis Structure

The scope is intentionally controlled. All experiments are carried out for European vanilla options under the Heston model, using synthetic data and numerical references whose settings can be inspected. This restriction is appropriate for the objective of the work: to test whether neural components can accelerate pricing, calibration, and sensitivity computation while remaining comparable to established numerical methods.

The remainder of the thesis is organized as follows. Chapter 2 introduces the mathematical and financial background needed for the rest of the work, including option pricing, the Heston model, Greeks, and the numerical methods used as reference. Chapter 3 develops the supervised ANN branch: the baseline implied-volatility surrogate, its Sobolev-

augmented version, and the CaNN calibration workflow built on top of these surrogates. Chapter 4 introduces the PINN branch for Heston pricing and sensitivity computation, covering the baseline PINN, the Sobolev-augmented PINN, and the Sobolev+ACV variant. Chapter 5 reports the numerical benchmarks by model family, separating ANN/CaNN calibration and Greek diagnostics from the PINN pricing and sensitivity experiments. Chapter 6 summarizes the main conclusions and discusses possible extensions.

2. MATHEMATICAL AND FINANCIAL BACKGROUND

2.1. Mathematical Framework For Option Pricing

This section fixes the common mathematical setup used in the rest of the thesis for European put pricing.

Following standard stochastic-process notation [19, Ch. 1], we work on a filtered probability space

$$(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t \in [0, T]}, \mathbb{P}).$$

Here, Ω is the set of possible outcomes, $\mathcal{F}$ is the collection of measurable events, $\mathbb{P}$ is the original probability measure, and $\mathcal{F}_t$ represents the information available up to time t . The filtration is increasing: if $s \leq t$, then $\mathcal{F}_s \subseteq \mathcal{F}_t$, so information is accumulated over time.

The maturity is finite, $T > 0$. Time is continuous, $t \in [0, T]$, and the time to maturity is $\tau = T - t \in [0, T]$. The underlying asset price $S = (S_t)_{t \in [0, T]}$ is assumed to be positive and adapted to the filtration, meaning that S_t is observable with the information available at time t . The strike $K > 0$ and the risk-free rate $r \geq 0$ are constant. When a fixed valuation date is needed, it is denoted by $t_0 \in [0, T)$ and $S_0 := S_{t_0}$. When it is useful to distinguish the two uses of moneyness, we write $m_0 := S_0/K$ for the fixed-date moneyness used in quote grids and $m := S/K$ for the generic moneyness state used in PDE formulations. Thus, m_0 denotes the coordinate observed at the valuation date, while m denotes the state variable over which the pricing equation is written.

The contract studied throughout the numerical pipeline is a European vanilla put. Its terminal payoff is

$$\Psi_{\text{put}} = (K - S_T)^+, \quad (x)^+ = \max(x, 0). \quad (2.1)$$

The payoff changes slope at the strike, $S_T = K$. This non-differentiable point is often referred to as the payoff kink. Figure 2.1 shows the normalized put payoff Ψ_{put}/K as a function of the normalized terminal price S_T/K .

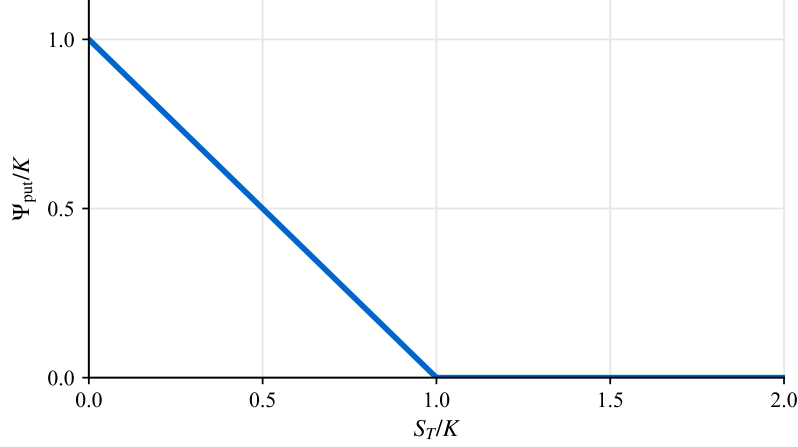


Fig. 2.1. Normalized terminal payoff of a European put. The holder has the right, but not the obligation, to exercise at maturity; therefore the payoff is positive only when $S_T/K < 1$ and is zero for $S_T/K \geq 1$.

Although the numerical pipeline in this thesis is built around European puts, the corresponding relation with European calls will also be useful. In the no-dividend setting, European call and put prices with the same underlying, strike, maturity, and risk-free rate satisfy the put-call parity relation [1]

$$V_t^{\text{call}} - V_t^{\text{put}} = S_t - Ke^{-r(T-t)}. \quad (2.2)$$

Equivalently, in terms of time to maturity $\tau = T - t$,

$$V_t^{\text{put}} = V_t^{\text{call}} - S_t + Ke^{-r\tau}. \quad (2.3)$$

This no-arbitrage identity will be used later to translate call-based Heston formulas into put quantities and to define parity residuals in the numerical benchmark.

A global notation glossary is provided in Appendix D and will be extended as new symbols are introduced.

For valuation, assume that there exists an equivalent risk-neutral measure $\mathbb{Q}$ under which discounted tradable prices are martingales. The standard risk-neutral valuation formula for the European payoffs considered in this thesis is [19, Ch. 8]

$$V_t = e^{-r(T-t)} \mathbb{E}^{\mathbb{Q}}[\Psi \mid \mathcal{F}_t] = e^{-r\tau} \mathbb{E}_t^{\mathbb{Q}}[\Psi], \quad 0 \leq t \leq T. \quad (2.4)$$

Here, V_t is the option value at time t , $\mathbb{E}_t^{\mathbb{Q}}[\cdot]$ denotes the conditional expectation given $\mathcal{F}_t$, and in the put case $\Psi = \Psi_{\text{put}}$. The risk-neutral measure is the pricing probability measure under which discounted tradable asset prices have no expected excess return. This is why future payoffs can be evaluated by taking conditional expectations under $\mathbb{Q}$ and discounting at the risk-free rate r .

In the following sections, this framework is connected to the main ingredients used throughout the thesis: the probability notation behind transform pricing, the stochastic-calculus tools needed for the model equations, the Black–Scholes benchmark, the Heston stochastic-volatility dynamics, and the characteristic-function/Fourier-COS pricing route.

2.2. Probability Theory

This section collects the probability notation used later in the transform-pricing formulas. We introduce distribution functions, densities, characteristic functions, and cumulants, since these objects are needed to construct the Fourier-COS pricing representation. The presentation follows standard probability definitions and the transform-pricing notation adopted in [19, Ch. 1] and [9, Sec. 2].

For a real random variable X , the cumulative distribution function is

$$F_X(x) = \mathbb{P}(X \leq x). \quad (2.5)$$

Here, x is a real evaluation point and $\mathbb{P}$ denotes probability. If F_X is differentiable, the associated probability density function is

$$f_X(x) = \frac{\partial F_X(x)}{\partial x}. \quad (2.6)$$

Following this notation, the characteristic function of X is

$$\phi_X(u) := \mathbb{E}[e^{iuX}] = \int_{-\infty}^{\infty} e^{iux} dF_X(x) = \int_{-\infty}^{\infty} e^{iux} f_X(x) dx, \quad (2.7)$$

for $u \in \mathbb{R}$, where i is the imaginary unit.

Applying the logarithm to ϕ_X , we obtain the cumulant characteristic function

$$\zeta_X(u) = \log \mathbb{E}[e^{iuX}] = \log \phi_X(u). \quad (2.8)$$

Its Maclaurin expansion is

$$\zeta_X(u) = \sum_{k=0}^{\infty} \frac{\partial^k \zeta_X(u)}{\partial u^k} \Big|_{u=0} \frac{u^k}{k!} = \sum_{k=0}^{\infty} \zeta_k \frac{u^k}{k!}, \quad (2.9)$$

where $\zeta_k \equiv \frac{\partial^k \zeta_X(u)}{\partial u^k} \Big|_{u=0}$ is the k -th cumulant.

The moment generating function is

$$\mathcal{M}_X(u) := \phi_X(-iu) = \mathbb{E}[e^{uX}]. \quad (2.10)$$

Using $\mathcal{M}_X(u) = \phi_X(-iu)$ and $\zeta_X(u) = \log \mathcal{M}_X(u)$, cumulants can be computed from the characteristic function as

$$\zeta_k = \frac{\partial^k \zeta_X(u)}{\partial u^k} \Big|_{u=0} = \frac{\partial^k}{\partial u^k} \log \phi_X(-iu) \Big|_{u=0}. \quad (2.11)$$

These objects are central in the rest of this chapter: the characteristic function provides direct access to Fourier-domain pricing formulas, and low-order cumulants are later used to construct truncation intervals for the COS method.

2.3. Stochastic Calculus Background

The Black–Scholes and Heston dynamics introduced below are written as stochastic differential equations (SDEs), so they require a continuous-time source of randomness. Following standard SDE notation [19, Chs. 2–4], a Brownian motion, also called a Wiener process, is an adapted process W_t with $W_0 = 0$, continuous paths, independent increments, and $W_t - W_s \sim \mathcal{N}(0, t - s)$ for $0 \leq s < t \leq T$. In stochastic differential equations, dW_t denotes the infinitesimal random component generated by this process. If two Brownian motions are correlated, their instantaneous quadratic covariation is written as $d\langle W^{(1)}, W^{(2)} \rangle_t = \rho dt$, with $\rho \in [-1, 1]$.

More generally, the asset dynamics used in this thesis are examples of Itô diffusions. In one dimension, such a process can be written as

$$dX_t = \mu(t, X_t) dt + \eta(t, X_t) dW_t, \quad (2.12)$$

where μ is the drift coefficient and η is the diffusion coefficient. The drift describes the deterministic infinitesimal trend, while the diffusion coefficient scales the random Brownian shock. Black–Scholes corresponds to a one-dimensional diffusion with proportional volatility, whereas Heston extends the state by adding a second stochastic equation for the variance.

The main calculus rule for such processes is Itô’s formula. If $f(t, x)$ is sufficiently smooth and X_t satisfies Equation (2.12), then

$$df(t, X_t) = \left(\frac{\partial f}{\partial t} + \mu \frac{\partial f}{\partial x} + \frac{1}{2} \eta^2 \frac{\partial^2 f}{\partial x^2} \right) dt + \eta \frac{\partial f}{\partial x} dW_t. \quad (2.13)$$

The second-order term is the distinctive feature of Itô calculus and comes from the quadratic variation of Brownian motion. In multidimensional models such as Heston, the same principle applies with cross-derivative terms determined by the instantaneous covariations between the Brownian drivers.

This connection is also what links the stochastic dynamics to pricing PDEs. Under the risk-neutral measure, if $V(t, x)$ denotes the price of a European claim with terminal payoff $\Psi(X_T)$, the Feynman–Kac representation relates the discounted conditional expectation to a partial differential equation of the form

$$\frac{\partial V}{\partial t} + \mathcal{L}V - rV = 0, \quad V(T, x) = \Psi(x), \quad (2.14)$$

where $\mathcal{L}$ is the infinitesimal generator of the diffusion. In Chapter 4, this link is used in the reverse direction: starting from the Heston SDEs, the PINN branch enforces the associated pricing PDE instead of learning from simulated paths or external price labels.

2.4. The Black–Scholes Model and Implied Volatility

To understand the Heston model, it is necessary to first recall the Black–Scholes model [1], [2], because Heston can be read as a direct extension of its constant-volatility dynam-

ics. In Black–Scholes, volatility is a fixed parameter: all randomness in the underlying price comes from a single Brownian driver with constant diffusion scale. This gives a closed-form pricing formula and, more importantly for this thesis, provides the convention used to define implied volatility. In the no-dividend Black–Scholes model, under the risk-neutral measure $\mathbb{Q}$, the underlying follows the geometric Brownian motion

$$dS_t = rS_t dt + \sigma S_t dW_t^{\mathbb{Q}}, \quad (2.15)$$

where r is the constant risk-free rate, $W_t^{\mathbb{Q}}$ is a Brownian motion under $\mathbb{Q}$, and $\sigma > 0$ is the constant volatility parameter. This is the modelling restriction that Heston relaxes by replacing the fixed variance level with a stochastic variance process. In the notation of Equation (2.12), this is a one-dimensional Itô diffusion with drift $\mu(t, S_t) = rS_t$ and diffusion coefficient $\eta(t, S_t) = \sigma S_t$.

For a European put, the corresponding Black–Scholes price is

$$V_{BS}^{\text{put}}(S_0, K, r, \tau, \sigma) = Ke^{-r\tau}\Phi(-d_2) - S_0\Phi(-d_1), \quad (2.16)$$

where Φ is the cumulative distribution function of a standard normal random variable and

$$d_1 = \frac{\log(S_0/K) + (r + \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}, \quad d_2 = d_1 - \sigma\sqrt{\tau}. \quad (2.17)$$

Given a target European put value V_{target} , its implied volatility is the value σ_{imp} that reproduces this price when inserted into the same Black–Scholes formula:

$$V_{BS}^{\text{put}}(S_0, K, r, \tau, \sigma_{\text{imp}}) = V_{\text{target}}. \quad (2.18)$$

Thus, implied volatility is a transformation of prices into volatility units; it is not a structural parameter of the Heston model. It is nevertheless the main comparison space in this thesis, because option quotes and calibration errors are usually interpreted across moneyness and maturity in implied-volatility units. The numerical inversion used to compute σ_{imp} is detailed in Section 2.8.

2.5. The Heston Stochastic Volatility Model

The Heston model [3] extends the constant-volatility benchmark of Section 2.4 by making variance stochastic. In the risk-neutral pricing form used in this thesis, the dynamics are given by the following system of coupled stochastic differential equations:

$$\begin{cases} dS_t = rS_t dt + \sqrt{v_t} S_t dW_t^{(S, \mathbb{Q})}, \\ dv_t = \kappa(\bar{v} - v_t) dt + \gamma \sqrt{v_t} dW_t^{(v, \mathbb{Q})}, \\ d\langle W^{(S, \mathbb{Q})}, W^{(v, \mathbb{Q})} \rangle_t = \rho dt. \end{cases} \quad (2.19)$$

Here, S_t is the underlying price, v_t is the instantaneous variance, and $W_t^{(S, \mathbb{Q})}$ and $W_t^{(v, \mathbb{Q})}$ are Brownian motions under the risk-neutral measure $\mathbb{Q}$. In this background chapter we

write the variance process as ν_t . Later, in the implementation-oriented chapters, the same variance state is denoted by ν_t ; in particular, the initial variance at the valuation date is $\nu_0 = \nu_{t_0}$ here and ν_0 in the dataset notation. The variance equation is a square-root, or CIR-type, diffusion: it is mean-reverting toward the long-run level $\bar{\nu}$ at speed κ , and its random variation has instantaneous scale $\gamma \sqrt{\nu_t}$. Thus, γ is the volatility of variance, often called vol-of-vol. Compared with the one-dimensional diffusion in Equation (2.12), Heston is a two-dimensional Itô diffusion with correlated Brownian drivers; this correlation is what generates the mixed derivative term in the Heston pricing PDE used later in Chapter 4.

The model parameters are summarized in Table 2.1. The table keeps the interpretation operational: each row states how the parameter enters the dynamics and which part of the generated price or IV grid it most directly affects.

TABLE 2.1. INTERPRETATION OF HESTON PARAMETERS

Parameter	Role in the dynamics	Main practical effect
ρ	Instantaneous correlation between the Brownian drivers of price and variance.	Changes the asymmetry of prices and IV values across moneyness.
γ	Volatility of the variance process, or vol-of-vol.	Controls the amplitude of random variation in the variance path.
$\bar{\nu}$	Long-run mean level of the variance process.	Sets the level toward which variance reverts.
ν_0	Initial variance level at valuation time (ν_{t_0}).	Sets the starting variance state, which is especially relevant for short maturities.
κ	Speed of mean reversion of ν_t toward $\bar{\nu}$.	Controls how quickly variance moves back toward its long-run level.

Source: Based on [3] and [20, Chs. 2–3]

A common positivity condition for the CIR variance process is the Feller condition

$$2\kappa\bar{\nu} \geq \gamma^2. \quad (2.20)$$

In practice, calibrated parameter sets may violate this inequality while still producing usable market fits; therefore, it is treated as a diagnostic and stability indicator rather than as a hard calibration constraint in many numerical workflows [20, Chs. 2–3].

The other output quantities used later are Greeks. To keep them distinct from Heston parameters such as γ and ρ , Greek names are written in roman text. For a value function

$V(S, \nu, t; r)$, we use

$$\begin{aligned}\Delta &= \frac{\partial V}{\partial S}, & \Gamma &= \frac{\partial^2 V}{\partial S^2}, \\ \text{Vega} &= \frac{\partial V}{\partial \nu}, & \text{Rho} &= \frac{\partial V}{\partial r}, \\ \Theta &= \frac{\partial V}{\partial t} = -\frac{\partial V}{\partial \tau}.\end{aligned}\tag{2.21}$$

Delta is the first-order sensitivity to the underlying price. Gamma is the second-order sensitivity to the underlying price. In this thesis, Vega denotes sensitivity with respect to the Heston variance state; in constant-volatility models Vega is often defined with respect to volatility instead. Rho is the sensitivity to the risk-free rate, and Theta is the sensitivity to calendar time, equivalently the negative derivative with respect to time to maturity.

2.6. Pricing Via Characteristic Functions

We now turn to the Fourier representation used for pricing. The relevant object is the characteristic function of a log-price state. For example, if $X_T = \log S_T$, then

$$\phi_X(u; t, S_t, \nu_t) = \mathbb{E}_t^{\mathbb{Q}}[e^{iuX_T} \mid S_t, \nu_t].\tag{2.22}$$

When pricing by log-moneyness, the same definition is applied to $Y_T = \log(S_T/K)$. Under Heston dynamics, this characteristic function has a semi-closed representation [3], which is why the model is well suited to the Fourier-COS pricing route developed in the next sections.

This transform representation is useful because the density of the terminal log-state is usually not handled directly in pricing. Fourier pricing instead works with the characteristic function and recovers prices through transform integrals.

Using the density and characteristic-function notation introduced above, let $x = X(t_0)$ be the current state and $y = X(T)$ the terminal state. The discounted valuation formula can be written as

$$V(t_0, x) = e^{-r\tau} \int_{\mathbb{R}} V(T, y) f_X(T, y; t_0, x) dy.\tag{2.23}$$

Here, $V(T, y)$ is the terminal payoff as a function of the terminal state, and $f_X(T, y; t_0, x)$ is the density of $X(T) = y$ conditioned on the current state $X(t_0) = x$. The corresponding characteristic function is

$$\phi_X(u; t_0, x) := \mathbb{E}_{t_0}^{\mathbb{Q}}[e^{iuX(T)} \mid X(t_0) = x] = \int_{\mathbb{R}} e^{iuy} f_X(T, y; t_0, x) dy.\tag{2.24}$$

If $f_X(T, \cdot; t_0, x)$ exists and $\int_{\mathbb{R}} |\phi_X(u; t_0, x)| du < \infty$, Fourier inversion gives:

$$f_X(T, y; t_0, x) = \frac{1}{2\pi} \int_{\mathbb{R}} e^{-iuy} \phi_X(u; t_0, x) du.\tag{2.25}$$

Let $\hat{V}_T(u) := \int_{\mathbb{R}} e^{-iuy} V(T, y) dy$ denote the Fourier transform of the payoff at maturity. Substituting the inverse representation of f_X into the pricing integral and exchanging the order of integration gives

$$V(t_0, x) = e^{-rt} \frac{1}{2\pi} \int_{\mathbb{R}} \phi_X(u; t_0, x) \hat{V}_T(u) du. \quad (2.26)$$

Compared with PDE solvers, this approach avoids full space-time gridding. Compared with Monte Carlo, it is deterministic and does not introduce sampling noise. In repeated-pricing tasks (surface generation, calibration loops, and large synthetic datasets), this is a major practical advantage.

The Fourier-COS method used in the next section is a specific spectral discretization of this transform-based pricing framework: it replaces the continuous integral by a finite cosine expansion over a truncated interval, preserving high accuracy with efficient vectorized evaluation.

2.7. The Fourier-COS method

The previous section showed that option values can be written as transform integrals once the characteristic function of the terminal state is known. The Fourier-COS method makes this representation numerically usable by replacing the terminal-state density on a finite interval with a cosine expansion [9, Secs. 3–4]. The key point is that the cosine coefficients of this density can be computed directly from the characteristic function, so pricing reduces to evaluating the characteristic function on a fixed frequency grid and summing finitely many terms.

Its practical appeal in this thesis is that it combines spectral convergence with efficient vectorized evaluation once the model characteristic function is available.

Let $x = X(t_0)$, $y = X(T)$, and let $[a, b] \subset \mathbb{R}$, with $a < b$, be a truncation interval chosen to concentrate most of the probability mass of $X(T) \mid X(t_0) = x$. On this interval, the density of the terminal state is approximated by

$$\hat{f}_X(y) = \sum_{k=0}^{N-1}{}' \bar{F}_k \cos\left(\pi k \frac{y-a}{b-a}\right), \quad (2.27)$$

where $\hat{f}_X$ is the truncated cosine approximation of the density, $N \in \mathbb{N}$ is the number of cosine terms, $k \in \{0, \dots, N-1\}$ indexes the expansion terms, $\sum'$ indicates that the $k=0$ term is weighted by $1/2$, and the cosine coefficients are obtained from the characteristic function:

$$\bar{F}_k := \frac{2}{b-a} \Re \left[\phi_X \left(\frac{\pi k}{b-a} \right) \exp \left(-\frac{i\pi a k}{b-a} \right) \right]. \quad (2.28)$$

The operator $\Re[\cdot]$ denotes the real part; for example, $\Re[\alpha + i\beta] = \alpha$ for $\alpha, \beta \in \mathbb{R}$.

Starting from

$$V(t_0, x) = e^{-rt} \int_{\mathbb{R}} V(T, y) f_X(T, y; t_0, x) dy, \quad (2.29)$$

the COS discretization yields

$$V(t_0, x) \approx e^{-r\tau} \sum_{k=0}^{N-1} \Re \left[\phi_X \left(\frac{\pi k}{b-a} \right) \exp \left(-\frac{i\pi a k}{b-a} \right) \right] H_k, \quad (2.30)$$

where the payoff coefficients H_k , for $k = 0, \dots, N-1$, are

$$H_k := \frac{2}{b-a} \int_a^b V(T, y) \cos \left(\frac{y-a}{b-a} \pi k \right) dy. \quad (2.31)$$

For plain-vanilla options, H_k has closed-form expressions, so the numerical effort is concentrated in evaluating the characteristic function on a fixed frequency grid [9, Sec. 3].

The choice of $[a, b]$ controls truncation error. Following the cumulant-based rule in [9, Sec. 5], we use

$$[a, b] = \left[c_1 - L \sqrt{c_2 + \sqrt{c_4}}, c_1 + L \sqrt{c_2 + \sqrt{c_4}} \right], \quad (2.32)$$

with c_1, c_2, c_4 the first, second, and fourth cumulants of $X(T)$ and $L > 0$ a scaling parameter. This connects directly with the cumulant machinery introduced in Section 2.2. In the numerical experiments of this thesis, the integration interval is selected using the refined cumulant-based strategy detailed in Appendix A.

The cumulant interval in Equation (2.32) addresses the state-space truncation part of the approximation, but it does not by itself control the error introduced by using only N cosine terms. It is therefore useful to separate two error sources:

$$|V - V_N^{\text{COS}}| \leq \underbrace{\varepsilon_{\text{space}}([a, b])}_{\text{state-space truncation}} + \underbrace{\varepsilon_{\text{series}}(N; [a, b])}_{\text{finite COS series}}. \quad (2.33)$$

Here, V_N^{COS} is the COS approximation using N terms, $\varepsilon_{\text{space}}$ is the error from replacing the full state space $\mathbb{R}$ by the interval $[a, b]$, and $\varepsilon_{\text{series}}$ is the error from keeping only finitely many cosine terms. Increasing L reduces $\varepsilon_{\text{space}}$ but may require larger N to keep $\varepsilon_{\text{series}}$ small; reducing L has the opposite effect. In practice, we choose (L, N) jointly and verify stability, especially for extreme strikes and short maturities, where numerical artifacts appear first.

Implementation is fully vectorized: frequencies $u_k = \pi k / (b-a)$, exponential phase factors, and payoff coefficients are precomputed once per maturity, then reused across strike grids. This is one of the reasons COS is well suited for repeated-pricing workflows such as calibration loops and synthetic dataset generation.

2.7.1. The COS Method Applied To The Heston Model

Following the notation of Sections 2.6–2.7 and [9, Sec. 3], define the COS frequencies and log-moneyness variables as

$$u_k := \frac{k\pi}{b-a}, \quad k = 0, \dots, N-1, \quad x := \log\left(\frac{S_0}{K}\right), \quad y := \log\left(\frac{S_T}{K}\right).$$

Here, x and y are log-moneyness variables: they measure the underlying price relative to the strike before and at maturity. Positive log-moneyness means the underlying is above the strike, while negative log-moneyness means it is below the strike. Under Heston dynamics, the characteristic function of y can be written in semi-closed form [3], [20]

$$\varphi_H(u; \tau, x, v_0) = \exp(C(u, \tau) + D(u, \tau)v_0 + iux), \quad (2.34)$$

with

$$\begin{aligned} d(u) &= \sqrt{(\kappa - i\rho\gamma u)^2 + \gamma^2(u^2 + iu)}, \\ g(u) &= \frac{\kappa - i\rho\gamma u - d(u)}{\kappa - i\rho\gamma u + d(u)}, \\ C(u, \tau) &= iur\tau + \frac{\kappa\bar{v}}{\gamma^2} \left[(\kappa - i\rho\gamma u - d(u))\tau - 2 \log \left(\frac{1 - g(u)e^{-d(u)\tau}}{1 - g(u)} \right) \right], \\ D(u, \tau) &= \frac{\kappa - i\rho\gamma u - d(u)}{\gamma^2} \left(\frac{1 - e^{-d(u)\tau}}{1 - g(u)e^{-d(u)\tau}} \right). \end{aligned} \quad (2.35)$$

Here, φ_H denotes the Heston characteristic function of the log-moneyness state, while C , D , d , and g are complex-valued auxiliary functions of the Fourier variable u and maturity τ . This is the standard semi-closed representation used in Fourier pricing; numerically, consistency in branch selection for $d(u)$ and the complex logarithm is required for stability [20, Chs. 2–3].

The COS price for one strike can then be written as

$$V(t_0, x) \approx K e^{-r\tau} \sum_{k=0}^{N-1} \Re \left[\varphi_H(u_k; \tau, x, v_0) e^{-iu_k a} \right] U_k, \quad (2.36)$$

where U_k are payoff-dependent COS coefficients for $k = 0, \dots, N-1$. For plain-vanilla options:

$$U_k^{\text{call}} = \frac{2}{b-a} (\chi_k(0, b) - \psi_k(0, b)), \quad U_k^{\text{put}} = \frac{2}{b-a} (-\chi_k(a, 0) + \psi_k(a, 0)), \quad (2.37)$$

with

$$\begin{aligned} \chi_k(\ell, q) &= \frac{1}{1 + u_k^2} \left[e^q (\cos(u_k(q-a)) + u_k \sin(u_k(q-a))) \right. \\ &\quad \left. - e^\ell (\cos(u_k(\ell-a)) + u_k \sin(u_k(\ell-a))) \right], \\ \psi_k(\ell, q) &= \begin{cases} q - \ell, & k = 0, \\ \frac{b-a}{k\pi} [\sin(u_k(q-a)) - \sin(u_k(\ell-a))], & k \geq 1. \end{cases} \end{aligned} \quad (2.38)$$

Here, χ_k and ψ_k are analytical payoff integrals over generic bounds ℓ and q satisfying $a \leq \ell < q \leq b$. In the implementation of this thesis, data generation is run on European puts as a numerical convention, mainly associated with the COS-pricing setup used in the project configuration.

This subsection establishes the analytical Heston-COS pricing block used as the reference framework in the following chapters.

2.7.2. Semi-Analytical Greeks from the Heston Characteristic Function

For validation purposes, and later as derivative targets for Sobolev training and as references in the model comparisons, the option Greeks defined in Section 2.5 can also be computed directly from the Heston characteristic-function pricing representation. A standard presentation is often written for a European call [3] and is developed in [20, Chapter 1]

$$V^{\text{call}}(S_0, K, \tau) = S_0 \Pi_1 - K e^{-r\tau} \Pi_2, \quad (2.39)$$

where $V^{\text{call}}(S_0, K, \tau)$ is the European call price and Π_1, Π_2 are risk-neutral probability-type terms in the Heston pricing formula:

$$\Pi_j = \frac{1}{2} + \frac{1}{\pi} \int_0^\infty \Re \left(\frac{e^{-iu \log K} \Psi_j(u)}{iu} \right) du, \quad j \in \{1, 2\}, \quad (2.40)$$

with

$$\Psi_2(u) = \phi_X(u), \quad \Psi_1(u) = \frac{\phi_X(u - i)}{\phi_X(-i)}, \quad (2.41)$$

where j selects the two probability terms, Ψ_j are the corresponding Fourier integrands, and ϕ_X is the characteristic function of $X_T = \log S_T$ under Heston [3]; see also [20, Chapter 1]. For puts, the same characteristic-function machinery is used through the put payoff coefficients or, equivalently, through the put-call parity relation in Equation (2.2).

Let p denote a generic model/input variable ($r, \tau, v_0, \rho, \kappa, \gamma, \bar{v}$, etc.). If $\partial_p \Psi_j(u)$ exists and the differentiated integrand is integrable on $(0, \infty)$, differentiation under the integral sign yields [20, Chapter 11]

$$\partial_p \Pi_j = \frac{1}{\pi} \int_0^\infty \Re \left(\frac{e^{-iu \log K}}{iu} \partial_p \Psi_j(u) \right) du. \quad (2.42)$$

Therefore, Greeks are obtained by combining the derivatives of Π_1, Π_2 through the pricing identity [20, Chapter 11]. For example:

$$\text{Delta}_{\text{call}} = \partial_{S_0} V^{\text{call}} = \Pi_1 + S_0 \partial_{S_0} \Pi_1 - K e^{-r\tau} \partial_{S_0} \Pi_2, \quad (2.43)$$

$$\text{Gamma}_{\text{call}} = \partial_{S_0}^2 V^{\text{call}} = 2 \partial_{S_0} \Pi_1 + S_0 \partial_{S_0}^2 \Pi_1 - K e^{-r\tau} \partial_{S_0}^2 \Pi_2, \quad (2.44)$$

$$\text{Vega}_{\text{call}} = \partial_{v_0} V^{\text{call}} = S_0 \partial_{v_0} \Pi_1 - K e^{-r\tau} \partial_{v_0} \Pi_2, \quad (2.45)$$

$$\text{Rho}_{\text{call}} = \partial_r V^{\text{call}} = S_0 \partial_r \Pi_1 + K \tau e^{-r\tau} \Pi_2 - K e^{-r\tau} \partial_r \Pi_2, \quad (2.46)$$

$$\text{Theta}_{\text{call}} = \partial_t V^{\text{call}} = -\partial_\tau V^{\text{call}}. \quad (2.47)$$

The thesis focuses on European puts, so the corresponding quantities are obtained by differentiating the put-call parity relation in Equation (2.3). This gives

$$\begin{aligned} \text{Delta}_{\text{put}} &= \text{Delta}_{\text{call}} - 1, & \text{Gamma}_{\text{put}} &= \text{Gamma}_{\text{call}}, & \text{Vega}_{\text{put}} &= \text{Vega}_{\text{call}}, \\ \text{Rho}_{\text{put}} &= \text{Rho}_{\text{call}} - \tau K e^{-r\tau}, & \text{Theta}_{\text{put}} &= \text{Theta}_{\text{call}} + r K e^{-r\tau}. \end{aligned} \quad (2.48)$$

The practical advantage is that all sensitivities are expressed in terms of derivatives of the same characteristic-function integrands. Equation (2.34) shows that the Heston

characteristic function is built from $C(u, \tau)$, $D(u, \tau)v_0$, and the log-state term. Therefore, parameter derivatives can be obtained by differentiating these components and inserting the resulting integrands into the formulas above.

2.8. Implied Volatility And Numerical Inversion

Implied volatility was defined in Section 2.4 as the Black–Scholes volatility that reproduces a given target option value. This section focuses on the numerical inversion used to compute it for market quotes or model-generated prices. In the workflow of this thesis, this inversion is used to build the implied-volatility labels for the supervised ANN baseline and ANN Sobolev models. By contrast, the PINN branch is trained in price space through the Heston PDE, so IV inversion is not part of its primary loss.

Numerically, we solve the scalar nonlinear equation

$$F(\sigma) := V_{BS}^{\text{put}}(S_0, K, r, \tau, \sigma) - V_{\text{target}} = 0, \quad \sigma > 0. \quad (2.49)$$

For European vanilla options, V_{BS} is strictly increasing in σ when $\tau > 0$, so the root is unique whenever V_{target} satisfies static no-arbitrage bounds.

The inversion routine used in this thesis is the bracketed Brent method [21, Ch. 9]. Brent’s method keeps a root bracket and combines bisection with secant and inverse-quadratic interpolation steps, falling back to bisection when the interpolated step is not reliable. This choice is robust for the one-dimensional IV root and avoids derivative-sensitivity issues in low-vega regions. As an alternative, the least-squares method of Levenberg and Marquardt [22], [23] was also tested in preliminary experiments, but it showed lower numerical stability in this workflow; therefore, it is not used in the final pipeline.

Stopping is based on standard price and parameter tolerances:

$$|F(\sigma_n)| \leq \varepsilon_{\text{price}} \quad \text{or} \quad |\sigma_{n+1} - \sigma_n| \leq \varepsilon_{\sigma}, \quad n \leq n_{\text{max}}. \quad (2.50)$$

Here, σ_n is the volatility iterate at step n , $\varepsilon_{\text{price}}$ and ε_{σ} are stopping tolerances, and n_{max} is the maximum number of iterations. If convergence is not reached, or if the target price violates arbitrage bounds, the point is flagged as invalid for IV reporting. In edge cases near the IV floor, the pricing stage may be recomputed with a wider COS truncation interval before retrying inversion.

The benchmark metrics used to evaluate the resulting IV, price, and Greek quantities are defined in Section 5.1.1.

3. SUPERVISED NEURAL METHODS FOR HESTON PRICING AND CALIBRATION

The main objective of this chapter is to construct the supervised neural block used in the thesis for Heston pricing, calibration, and sensitivity analysis. After the Black–Scholes implied-volatility convention (Section 2.4), the Heston dynamics (Section 2.5), and the COS reference solver (Section 2.7) have been fixed in Chapter 2, we now move from the numerical pricing layer to its supervised neural approximation. Within this block, we distinguish two final ANN pricers. The first one is the baseline ANN, which learns the Heston implied-volatility surface as a supervised regression problem and approximates the forward map

$$(\Theta, m, \tau, r) \mapsto \sigma_{imp}.$$

This model provides the reference supervised surrogate in IV space, using the notation and implied-volatility definition introduced in Section 2.4 and computed through the inversion procedure of Section 2.8.

The second model is the Sobolev ANN. This variant keeps the same input-output structure, but adds differential information during training. The goal is not to redefine the pricing problem, but to improve the local geometry of the learned IV surface. This matters because a surface with low pointwise IV error does not necessarily produce accurate sensitivities. The Sobolev extension addresses this limitation by adding derivative information computed from the Heston reference model.

Reference pricing based on the numerical methods introduced in Chapter 2 is highly accurate but computationally expensive when repeated many times, as required in calibration and sensitivity workflows. In this project, the reference layer combines Heston-COS valuation (Section 2.7) with implied-volatility inversion (Section 2.8). The baseline ANN aims to reproduce this numerical pipeline efficiently, while the Sobolev ANN additionally uses derivative targets to make the surrogate more reliable for Greek estimation.

Finally, the chapter also introduces the neural calibration workflow. In this stage, the trained pricer is frozen and used as a fast evaluator inside an inverse problem that recovers Heston parameters from an observed implied-volatility surface. Methodologically, we follow the ANN-pricing and neural-calibration framework of Liu et al. [6] as the main reference, while documenting the specific implementation choices adopted in this project. The numerical experiments were implemented in Python using PyTorch as the main neural-network framework; in particular, its automatic differentiation tools are used both for derivative-aware training terms and for the sensitivity diagnostics computed from the learned surfaces. Appendix C gives additional implementation and repository details. The benchmark results are reported later in Chapter 5; here the focus is the construction of the supervised pricing dataset, the pricer architecture, the baseline and Sobolev training

protocols, and the calibration interface.

3.1. Problem Formulation

After the Heston dynamics and their COS pricing representation have been fixed in Sections 2.5 and 2.7, we now pose vanilla option pricing under Heston as a supervised regression problem. Given a dataset

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad y_i = \sigma_{imp,i},$$

the neural network approximates the map

$$f_{NN} : \mathbb{R}^8 \rightarrow \mathbb{R}, \quad \mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \mapsto \sigma_{imp},$$

where m and τ follow the moneyness and time-to-maturity notation fixed in Chapter 2. Each label σ_{imp} is obtained from Heston-consistent synthetic prices through numerical inversion.

The training objective is to estimate parameters $\mathbf{w}$ such that the empirical prediction error over $\mathcal{D}$ is minimized. In the pricing part of this chapter, the ANN family is used as a surrogate, i.e., as a fast approximation of the reference numerical pipeline in the forward map $(\Theta, m, \tau, r) \mapsto \sigma_{imp}$. In the calibration part, the same forward map is reused with the quote descriptors fixed and the Heston parameter vector varied.

This formulation defines the baseline ANN, which is trained only to minimize point-wise implied-volatility error. We also consider a Sobolev ANN with the same input-output map, but with additional targets for derivatives of the implied-volatility surface. The purpose of this second variant is not to change the pricing representation, but to improve the local geometry of the learned IV surface, which later makes the Greeks computed from the ANN implied volatility more stable.

We learn implied volatility instead of option price because the IV representation is directly comparable across contracts with different maturities and moneyness levels, and it is the market-standard quotation format for calibration and diagnostics. In contrast, raw prices mix scale effects (spot/strike/time) that make learning less homogeneous over the domain.

The resulting surrogates are primarily in-domain approximators. Preliminary out-of-domain checks indicate that the networks remain stable for moderate departures from the training bounds, while errors increase in extreme regimes. For this reason, the methodological core of this chapter is developed on the predefined training domain, and the extrapolation behaviour is discussed separately in the generalization and robustness analysis.

3.2. Synthetic Dataset And Supervised Learning Vocabulary

Training these supervised surrogates requires a large set of labeled input-output pairs. To obtain a controlled and reproducible source, we generate synthetic data under the Heston model introduced in Section 2.5. The pricing and implied-volatility conventions come from Chapter 2; this section fixes the dataset vocabulary used by the implementation.

In the current pipeline, generation is organized as a *master dataset*: a table whose rows are available to several downstream workflows. Each row stores the variables needed for supervised learning and numerical diagnostics:

- The *features* are the neural-network inputs $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$.
- The *training label* is the implied volatility σ_{imp} , stored as `sigma_imp_brent`, obtained from the reference numerical pipeline.
- The *auxiliary target* is the COS price `price_cos`, retained as a reference quantity for diagnostics.
- The *quality flags* are metadata used to assess numerical reliability. They record the Feller diagnostic and the Brent-inversion status and residual used for filtering and robustness checks.

The terms *split* and *preprocessing* refer, respectively, to the train/validation/test partition and to the transformations applied before training, such as row filtering, precision changes, or feature/target normalization. This design allows ANN and PINN workflows to reuse the same synthetic source while applying experiment-specific filters only at training or evaluation time.

The raw variable set contains the Heston parameters from Equation (2.19) and the contract/market variables (K, S_0, τ, r) . The background chapter uses $\bar{v}$ and v_0 for variance quantities; the dataset columns $\bar{v}$ and v_0 denote the same long-run and initial variance levels in implementation notation. We fix $K = 1$ and keep the moneyness coordinate m as the relative underlying level. This is consistent with the log-moneyness state used in the Heston-COS formula in Equation (2.34), since $x = \log(S_0/K) = \log m$. The resulting ANN input vector has eight features:

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \in \mathbb{R}^8.$$

A full Cartesian grid is computationally prohibitive in this dimension, since using N points per variable scales as N^8 (already intractable for moderate N). Therefore, the synthetic inputs are sampled with Latin Hypercube Sampling (LHS), which stratifies each coordinate and provides broad marginal coverage for a prescribed number of samples [24]. In our implementation, LHS is applied directly to the full input vector with a fixed seed and a target size of 10^7 samples in the master dataset. The specific ranges, constraints, and splitting/preprocessing rules are detailed below.

3.2.1. Parameter Ranges And Constraints

The sampling domain combines Heston parameters, contract variables, and market rate. Following the ranges used in our configuration (largely aligned with [6]), the training domain is summarized in Table 3.1.

TABLE 3.1. INPUT RANGES USED FOR SYNTHETIC DATA GENERATION

Parameter	Range
ρ	$[-0.9, 0.0]$
κ	$[0.0, 3.0]$
γ	$[0.01, 0.8]$
$\bar{v}$	$[0.01, 0.5]$
v_0	$[0.05, 0.5]$
m	$[0.6, 1.4]$
τ	$[0.05, 3.0]$
r	$[0.0, 0.05]$

Source: Based on [6] and project configuration choices

The maturity range enforces $\tau > 0$, which is required by the pricing and implied-volatility conventions introduced in Chapter 2. We also record the Feller diagnostic from Equation (2.20). In the dataset, `feller_slack` stores the signed margin of that condition and `feller_ok` stores the corresponding Boolean pass/fail flag. Feller-violating points are *not* removed during generation; they are kept in the master dataset for controlled ablation studies and can be filtered later if a particular experiment requires it.

Recalling that the strike is fixed at $K = 1$, the quote grid is expressed through m . This reduces the dimensionality from nine variables $(\Theta, S_0, K, \tau, r)$ to eight inputs (Θ, m, τ, r) without loss of information for this setup.

3.2.2. Dataset Splitting And Preprocessing

After generation, we keep all selected rows in the master dataset and split into train/validation/test subsets with proportions 0.8/0.1/0.1. In the reported configuration, splits are random (without mandatory stratification), and filtering policies are deferred to each training experiment.

In the baseline configuration, no explicit feature or target normalization is applied (Liu-style setup). Therefore, the model is trained directly on the physical scale of the variables $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$ and predicts IV in absolute units.

Data quality control is integrated in the synthesis pipeline through explicit flags, not hard row deletion by default. In particular, we track Brent inversion status and the absolute

price reconstruction residual associated with the IV inversion of Section 2.8. Keeping these quantities in the table makes the filtering rule auditable: a later experiment can state exactly whether it used all rows, only successful IV inversions, only Feller-admissible rows, or a stricter residual threshold. For storage efficiency, output tables are persisted in `float32`, whereas COS pricing and Brent inversion are computed internally in `float64`.

3.2.3. Numerical Label Generation (COS + Brent)

Each label is produced in two deterministic numerical steps already defined in the background chapter. First, the target option value is computed with the Heston-COS solver for the sampled feature vector. Second, that value is converted to implied volatility using the Brent inversion routine from Section 2.8. Thus, the ANN target is a Heston-consistent implied volatility, not an independently assumed Black–Scholes volatility.

In our implementation, Brent is configured with strict numerical controls (absolute tolerance 10^{-8} , bounded search interval, and iteration cap). Additionally, when the inferred volatility hits the lower IV floor, an adaptive COS fallback can be triggered by widening the truncation interval before retrying inversion. This is the same numerical-error concern discussed in Appendix A: COS labels must be stable because they are later treated as supervised targets.

Rather than dropping those edge cases at generation time, the pipeline records quality metadata (`brent_success`, `brent_abs_residual`, and `retry` diagnostics) so that robustness filters can be applied transparently at training time. This preserves full traceability of numerical reliability across the domain.

3.3. Neural Network Architecture

Both ANN pricers are implemented as fully connected neural networks that approximate a smooth nonlinear map from model/contract inputs to implied volatility. This choice is consistent with the structure of the problem: inputs are low-dimensional continuous features without spatial topology or sequential dependence, and the target is a scalar regression value.

From a function-approximation perspective, a feed-forward multilayer perceptron (MLP) provides enough expressive capacity to capture the interactions between Heston parameters, moneyness, maturity, and rates on the selected domain. Compared with architectures designed for images or sequences (e.g., CNNs or transformers), an MLP is more parsimonious and easier to train for this tabular setting.

This choice is also consistent with the classical universal-approximation results: under standard assumptions, feed-forward networks with non-polynomial activations can approximate continuous functions on compact sets to arbitrary accuracy [25], [26]. In this thesis, this theorem is used as a representation-capacity motivation, while practical

accuracy is established empirically through out-of-sample benchmarks.

3.3.1. Model Class

Following [6], we use a fully connected feed-forward network (FCNN). Let $\mathbf{x} \in \mathbb{R}^8$ be

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r),$$

and let $y = \sigma_{imp} \in \mathbb{R}$ denote the target implied volatility. The model defines a parametric function

$$\hat{\sigma}_{imp} = f_{NN}(\mathbf{x}; \mathbf{w}),$$

with trainable parameters $\mathbf{w}$.

Once $\mathbf{w}$ is fixed after training, inference is deterministic: for the same input $\mathbf{x}$, the network always returns the same prediction $\hat{\sigma}_{imp}$. This property is important for downstream calibration and sensitivity analysis, where numerical consistency across repeated calls is required.

The FCNN acts as a surrogate only at inference time. Target labels are still generated with the numerical pipeline (Heston COS pricing followed by implied-volatility inversion), so the network learns to emulate that reference solver on the sampled domain.

3.3.2. Architecture Details

The selected architecture used in the final pipeline is

$$8 \rightarrow 200 \rightarrow 200 \rightarrow 200 \rightarrow 200 \rightarrow 1,$$

with $\tanh$ activation in hidden layers and linear activation in the output layer. In compact form,

$$h^{(1)} = \phi(W^{(1)}x + b^{(1)}), \quad \dots, \quad h^{(4)} = \phi(W^{(4)}h^{(3)} + b^{(4)}), \quad \hat{y} = W^{(5)}h^{(4)} + b^{(5)},$$

where $\phi(\cdot) = \tanh(\cdot)$.

Weights are initialized with Xavier uniform initialization and biases are initialized to zero. In the baseline setup, dropout is disabled ($p = 0$) and no batch normalization layers are used. This configuration follows a controlled, reproducible design intended to isolate the impact of optimization and data quality choices.

TABLE 3.2. NEURAL NETWORK ARCHITECTURE USED FOR THE ANN PRICER

Hyperparameter	Value
Input dimension	8
Hidden layers	4
Hidden width	200 neurons per layer
Hidden activation	$\tanh$
Output dimension	1
Output activation	Linear
Weight initialization	Xavier uniform
Bias initialization	Zeros
Dropout	$p = 0.0$
Batch normalization	Disabled

Source: Based on [6], with selected $\tanh$ activation

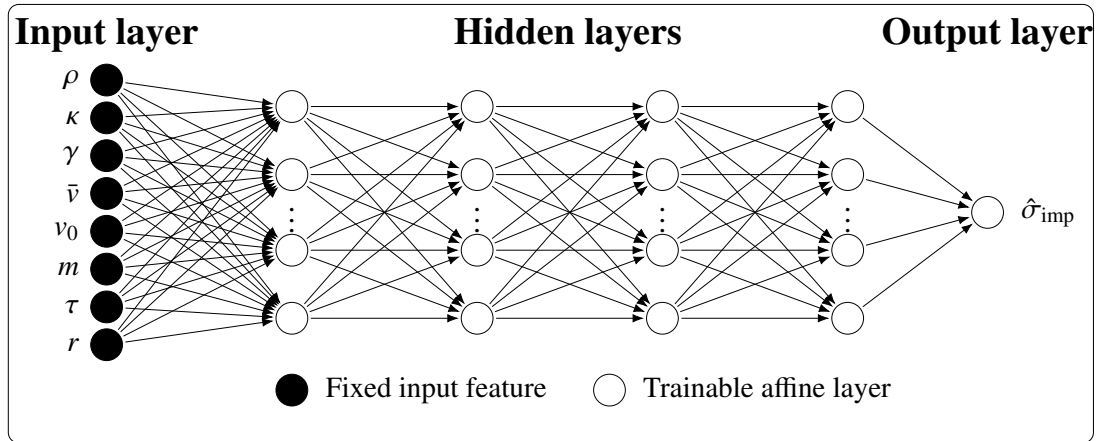


Fig. 3.1. Fully connected ANN pricer used in the forward Heston implied-volatility surrogate. The eight fixed inputs are mapped through four $\tanh$ hidden layers of width 200, and the final linear layer returns the predicted implied volatility.

3.3.3. Output Interpretation

The network output is a scalar implied volatility in annualized units (decimal form), not an option price. Therefore, prediction errors are interpreted directly in IV space, which is the representation used in the calibration workflow and in most volatility-surface diagnostics.

Given a network prediction

$$\hat{\sigma}_{\text{imp}} = f_{NN}(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r),$$

the price implied by the ANN output is obtained as

$$\hat{V}_{\text{ANN}} = V_{BS}(m, \tau, r, \hat{\sigma}_{\text{imp}}).$$

Greeks are then computed by differentiating this composed map with respect to the relevant input variable q :

$$\text{Greek}_{\text{ANN}}(q) = \frac{\partial}{\partial q} V_{BS}(m, \tau, r, f_{NN}(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)). \quad (3.1)$$

Thus, Greek accuracy depends not only on the pointwise IV error, but also on the smoothness and derivative quality of the learned IV surface.

Because the output layer is linear, the model is not hard-constrained to produce strictly positive values for every possible input. In practice, training on a physically admissible domain with positive IV labels strongly reduces this risk inside the sampled region. For downstream use, any out-of-domain prediction can be explicitly checked and, if required, filtered before subsequent numerical steps.

This output definition also preserves differentiability of the surrogate with respect to inputs, which is useful in later chapters when the ANN pricer is embedded in calibration and sensitivity pipelines.

3.4. Training Procedure

Training the network means adjusting the weights $\mathbf{w}$ so that the empirical loss on the supervised dataset decreases. At this stage we describe the common training loop; the numerical rule used to update the weights, and the meaning of optimizer in this setting, are introduced in Section 3.4.3.

The training loop is fully configuration-driven and follows an epoch-based supervised learning scheme. At each epoch, the configured update rule modifies the parameters using the training split, then the model is evaluated on the validation split without gradient computation. This separates the learning step from the monitoring step: the former changes $\mathbf{w}$, while the latter only measures out-of-sample behaviour under the current weights.

Each epoch stores train/validation metrics, current update-rule identity, and learning rate. When enabled, the validation monitor can be either the raw validation loss or a trimmed variant that discards the top-tail hardest samples before aggregation. This provides a robust alternative when a small fraction of outlier points may dominate the monitor value.

Execution device is selected automatically (mps when available, otherwise cpu), and data-shuffling reproducibility is controlled through a fixed generator seed from configuration. During training, two checkpoints are maintained: the *last* checkpoint (always overwritten each epoch) and the *best* checkpoint (updated only when the monitored validation metric improves). The best checkpoint is used as the reference model for post-training evaluation and downstream calibration.

3.4.1. Baseline ANN Loss Function

Let $\{(\mathbf{x}_i, \sigma_i)\}_{i=1}^B$ denote a mini-batch, where σ_i is the target implied volatility and $\hat{\sigma}_i = f_{NN}(\mathbf{x}_i; \mathbf{w})$. The baseline objective is mean squared error (MSE):

$$\mathcal{L}_{\text{MSE}}(\mathbf{w}) = \frac{1}{B} \sum_{i=1}^B (\hat{\sigma}_i - \sigma_i)^2. \quad (3.2)$$

Here B is the mini-batch size, σ_i is the reference implied volatility obtained from the Heston-COS/Brent pipeline, and $\hat{\sigma}_i$ is the ANN prediction for the same input $\mathbf{x}_i$.

MSE is preferred because it is smooth, differentiable everywhere, and strongly penalizes large deviations, which is desirable when the ANN is later used in calibration loops sensitive to local pricing errors. RMSE and MAPE are then used as reporting metrics, following the convention fixed in Chapter 5.

3.4.2. Sobolev ANN Loss Function

For the Sobolev ANN, the MSE term is kept as the value-matching component, but the loss is augmented with derivative information from a separate anchor set. Let

$$\mathcal{J} = \{m, \tau, r, v_0\}$$

be the input directions used for Sobolev supervision, and let $d_{i,q}$ denote the target derivative $\partial\sigma_{\text{imp}}/\partial q$ at anchor point $\mathbf{x}_i$. The derivative loss is

$$\mathcal{L}_{\text{Sob}}(\mathbf{w}) = \frac{1}{B_{\text{Sob}}|\mathcal{J}|} \sum_{i=1}^{B_{\text{Sob}}} \sum_{q \in \mathcal{J}} \left(\frac{\partial_q f_{NN}(\mathbf{x}_i; \mathbf{w}) - d_{i,q}}{s_q} \right)^2. \quad (3.3)$$

Here B_{Sob} is the Sobolev mini-batch size, q indexes the supervised derivative directions, and $d_{i,q}$ is the Heston-consistent IV derivative target. The factor s_q is a robust scale computed from the reference IV derivatives in direction q . It normalizes the derivative errors so that directions with larger raw magnitudes do not dominate the loss. This follows the Sobolev-training idea of matching both function values and derivatives [16], adapted here to the implied-volatility surface. The total Sobolev objective is then

$$\mathcal{L}_{\text{ANN-Sob}}(\mathbf{w}) = \mathcal{L}_{\text{MSE}}(\mathbf{w}) + \lambda_{\text{Sob}} \mathcal{L}_{\text{Sob}}(\mathbf{w}). \quad (3.4)$$

Here $\mathcal{L}_{\text{MSE}}$ is the baseline value loss from Equation (3.2). In the retained Sobolev ANN run, $\lambda_{\text{Sob}} = 0.10$, the Sobolev batch size is 512, and the derivative term is activated after a warm-up of 500 epochs. A compact implementation excerpt showing how this derivative loss is computed with PyTorch automatic differentiation is included in Appendix C.

3.4.3. Optimizer And Hyperparameters

In this context, an optimizer is the numerical rule that updates the network weights $\mathbf{w}$ in order to reduce the loss. In a full-gradient method, the update direction would be computed from all training samples at each step. For a dataset of this size, this is unnecessarily expensive, so the practical alternative is to estimate the gradient from mini-batches. This gives a stochastic gradient: it is cheaper to compute, but it introduces sampling noise because each batch only approximates the full empirical gradient.

Adam belongs to the family of stochastic gradient methods, but it modifies plain stochastic gradient descent by using adaptive moments. More precisely, it keeps exponential moving averages of the mini-batch gradient and of its squared values, and uses them to scale the update of each coordinate. The learning rate fixes the base step size before this adaptive rescaling, while the optimizer weight decay controls whether the update also includes an L^2 -type shrinkage of the weights. The values used in the retained ANN configurations are reported in Table 3.3: the Adam learning rate starts at 10^{-3} , and weight decay is set to 0.0, so no additional regularization is imposed through Adam itself. This is useful in our supervised ANN setting because the dataset is large, the loss is non-convex, and the different input directions may produce gradients with different magnitudes.

L-BFGS follows a different logic. It is a limited-memory method that uses recent parameter and gradient differences to build a compact approximation of the local curvature of the loss [27, Ch. 7]. In practical terms, it tries to use more information than the current gradient alone, which can help when the parameters are already close to a good region. The trade-off is computational: each L-BFGS step is heavier and requires reevaluating the loss and its gradient through a closure. For this reason, in this project L-BFGS is mainly tested as a refinement stage after an Adam phase, not as the default training rule for the full ANN run.

The optimizer experiments followed this logic. First, Adam-only runs were used to establish stable baselines under the Liu-style architecture, varying activation, learning-rate schedule, weight decay, and early-stopping policy. Then, mixed schedules were tested by switching from Adam to L-BFGS after an initial stochastic phase. The motivation for these mixed runs was reasonable: Adam can move efficiently through the large dataset, while L-BFGS might improve the final local fit. The retained mixed candidate did not improve the held-out error profile of the selected Adam–tanh run, so L-BFGS remains part of the experimental path but not part of the final reference surrogate. The quantitative comparison is reported with the ANN benchmark results in Table 5.1.

Under this criterion, the selected baseline ANN is the Liu-style architecture trained with Adam and tanh hidden activations. It preserves the reference width and depth, improves on the earlier ReLU baseline in the retained benchmark comparison, and provides the baseline forward evaluator used in the CaNN comparison. The Sobolev ANN keeps this same architecture and optimizer backbone, but replaces the objective by Equation (3.4).

3.4.4. Regularization And Callbacks

Regularization in the selected ANN configurations is intentionally light: dropout is disabled ($p = 0$), batch normalization is disabled, and Adam weight decay is set to zero. In addition, feature/target normalization is turned off in the Liu-style reference setup. This design keeps the training protocol close to the reference architecture and isolates the effect of the selected hidden activation, optimizer scheduling, and data quality controls.

A learning-rate scheduler is a callback that changes the optimizer step size during training according to a prescribed rule. In the selected ANN configurations we use StepLR, which reduces Adam’s learning rate by a factor γ after a fixed number of epochs. Early stopping follows a different purpose: it monitors validation performance and interrupts training when the selected validation criterion stops improving over a given patience window. In the retained runs this mechanism is available in the training pipeline but disabled, so model selection relies on the stored *best* and *last* checkpoints rather than on an automatic stopping rule.

Callback management therefore includes: (i) checkpointing of both *best* and *last* models, (ii) StepLR scheduling for Adam, and (iii) an early-stopping module available in the pipeline but disabled in the baseline experiments. When enabled, early stopping supports both standard validation loss and trimmed validation loss monitors.

From an experimental standpoint, this callback configuration reduces the risk of selecting a late degraded checkpoint without over-constraining the optimization path. The scheduler controls the Adam step size in the selected ANN configurations, while the Adam-to-L-BFGS transition remains available for mixed experiments. Generalization is then assessed empirically on held-out validation and test splits rather than enforced through strong explicit regularizers.

TABLE 3.3. HYPERPARAMETERS AND TRAINING CONTROLS
OF THE SELECTED ANN CONFIGURATIONS

Hyperparameter	Value
Training mode	Adam
Hidden activation	tanh
Seed	42
Epochs	8000
Train batch size	1024
Validation batch size	All validation samples
Baseline loss	MSE, Equation (3.2)
Sobolev extension	$\lambda_{\text{Sob}} = 0.10$, batch 512, warm-up 500 epochs
Adam learning rate	10^{-3}
Adam weight decay	0.0
LR scheduler	StepLR, step size = 500, $\gamma = 0.5$
Early stopping	Disabled

Source: Own elaboration from run configuration copies

3.5. Neural Calibration As An Inverse Problem

The supervised ANN pricers constructed in the previous sections approximate the forward Heston map from parameters and contract descriptors to implied volatility. Neural calibration uses the same idea in the opposite direction: given a finite implied-volatility surface, infer the Heston parameter vector

$$\theta = (\rho, \kappa, \gamma, \bar{v}, v_0).$$

Here the parameter meanings are those fixed in Table 2.1, with the same dataset notation convention for $\bar{v}$ and v_0 described in Section 3.2. The calibration workflow follows the neural-calibration idea of Liu et al. [6]: a trained ANN pricer is frozen and used as a fast evaluator inside a calibration loop.

The purpose is not to introduce a new calibration algorithm. The contribution in this thesis is the integration of this calibration step into the ANN–PINN workflow and the diagnostic reading of its output. For that reason, this section keeps only the setup needed to interpret the benchmark in Chapter 5.

3.5.1. Calibration Setup

Each trained ANN surrogate approximates the forward map

$$f_{NN} : (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \mapsto \sigma_{IV},$$

where m , τ , r , and σ_{IV} keep the notation fixed in Chapter 2. In calibration, the quote descriptors are fixed and only θ is varied. For each candidate θ , the network is evaluated on the quote set

$$\mathcal{Q} = \{(m_j, \tau_j, r_j, \sigma_j^{\text{mkt}}, w_j)\}_{j=1}^{N_q},$$

where $w_j \geq 0$ is an optional quote weight. The selected benchmark uses $N_q = 35$ quotes on a complete 5×7 moneyness–maturity grid with constant $r = 0.03$; the exact nodes are reported in Section 5.2.3.

For quote j , the predicted implied volatility and residual are

$$\hat{\sigma}_j(\theta) = f_{NN}(\rho, \kappa, \gamma, \bar{v}, v_0, m_j, \tau_j, r_j), \quad e_j(\theta) = \hat{\sigma}_j(\theta) - \sigma_j^{\text{mkt}}.$$

Calibration is therefore performed in implied-volatility space, matching the ANN training target and the market convention used for volatility smiles.

The objective minimized in the calibration loop is

$$J(\theta) = \frac{1}{\sum_{j=1}^{N_q} w_j} \sum_{j=1}^{N_q} w_j e_j(\theta)^2 + \lambda R(\theta), \quad (3.5)$$

where $R(\theta)$ is an optional regularization penalty. The calibrated vector is

$$\theta^* = \arg \min_{\theta \in \mathcal{B}} J(\theta), \quad (3.6)$$

with $\mathcal{B}$ the admissible Heston parameter box. In this thesis, the bounds are aligned with the ANN training domain: $\rho \in [-0.9, 0]$, $\kappa \in [0, 3]$, $\gamma \in [0.01, 0.8]$, $\bar{v} \in [0.01, 0.5]$, and $v_0 \in [0.05, 0.5]$. This restriction keeps calibration inside the intended interpolation regime of the forward surrogate.

3.5.2. Search And Stability Checks

The inverse residual surface is not assumed to be convex, so the reference calibration uses Differential Evolution followed by local polishing. This is a pragmatic choice: the method is robust to initialization, does not require gradients of the neural objective, and can evaluate a population of candidates through batched ANN inference.

The calibration protocol is the same for both CaNN variants considered in the benchmark. The baseline CaNN uses the frozen ANN baseline surrogate, while the Sobolev CaNN uses the frozen ANN Sobolev surrogate. In both cases, the quote grid, parameter bounds, objective $J(\theta)$, and Differential Evolution plus polishing strategy are kept fixed; only the forward implied-volatility evaluator f_{NN} is replaced. Thus, Sobolev training changes the quality of the surrogate used inside calibration, but it does not define a different calibration objective.

The calibration configuration is kept inside the interpolation domain of the ANN surrogates. It uses the 35-quote grid described above, no regularization in the reference

objective ($\lambda = 0$), and the same parameter bounds as the ANN training domain. Differential Evolution is run with maximum iteration count 1000, population multiplier 20, tolerance 10^{-4} , mutation range (0.5, 1.0), recombination 0.7, and final polishing enabled.

The selected solution is assessed at two levels. First, quote-level residuals check whether the implied-volatility surface is reproduced. Second, parameter diagnostics check whether the selected optimum is locally stable. Around a low-residual optimum, the residual data term can be read through the Gauss–Newton approximation

$$H(\theta^*) \approx 2J_e(\theta^*)^\top W J_e(\theta^*),$$

where $J_e(\theta^*)$ is the Jacobian of quote residuals with respect to θ , and W is the diagonal matrix of quote weights. Large curvature indicates directions where the quote surface reacts strongly to parameter changes; small curvature indicates flatter, less identified directions.

These diagnostics are not interpreted here as results; they define the checks used later to decide whether a calibration is both low-residual and locally stable. The numerical recovery, residual metrics, and parameter-error tables are reported in Chapter 5.

4. PHYSICS-INFORMED NEURAL PRICING UNDER THE HESTON MODEL

This chapter develops the physics-informed branch of the thesis for Heston option pricing. Unlike the supervised ANN branch of Chapter 3, which learns the implied-volatility surface from numerical labels and is later reused inside the CaNN calibration workflow, the PINN branch works directly in price space. Its baseline version approximates the option-price function by enforcing the Heston pricing PDE, terminal payoff, and boundary conditions, without using precomputed price or implied-volatility labels during training. The methodology follows the general PINN philosophy and is inspired by the Heston-specific formulation proposed in [10], while adapting the state representation and notation to the conventions used in the present work.

This gives a different methodological object: the PINN is not an extension of the ANN–CaNN pipeline, but a separate PDE-based pricer whose main strength is direct price-surface approximation. Since recent physics-informed operator-learning evidence for Heston suggests that accurate pricing can still coexist with noisy autodiff Delta and Gamma near the at-the-money region [12], the chapter then introduces two derivative-aware variants: a Sobolev PINN and a Sobolev PINN with an analytical control variate (ACV) correction. The purpose of these variants is to improve the Greek diagnostics of the baseline PINN, while keeping explicit the trade-off between pricing accuracy and sensitivity regularity.

4.1. Problem formulation

In this section, we make the distinction introduced above operational by specifying the price function, the state variables, and the parameter dependence used by the PINN. The baseline formulation remains limited to the unsupervised price-surface problem: the network represents V_ϕ , is trained on collocation points, and does not use implied-volatility labels. The Sobolev and ACV components are introduced later as derivative-oriented variants in Sections 4.5 and 4.6, so they should not be read as part of the baseline PINN.

To keep the price function visually separate from the variance state, the option value will be denoted by V , while the instantaneous variance state will be denoted by v . This distinction is useful because the Heston model contains both a variance process and a pricing function, and the PDE notation should make clear which object is being differentiated. The unknown function approximated by the PINN is therefore written as

$$V_\phi(\tau, m, v, \Theta, r),$$

where ϕ denotes the trainable weights of the network, $m = S/K$ is moneyness, $\tau = T - t$ is time-to-maturity, v is the current instantaneous variance, and Θ collects the structural

Heston parameters.

More precisely, the calibrated tuple obtained in the previous chapter is

$$\Theta^* = (\rho, \kappa, \gamma, \bar{v}, v_0).$$

However, the role of v_0 must be distinguished from that of the other parameters. The quantities ρ , κ , γ and $\bar{v}$ define the operator of the Heston PDE, whereas v_0 is the initial condition of the variance process. In the PDE formulation, the state variable is the generic variance level v , not the fixed initial value v_0 . Therefore, the parametric PINN is constructed to learn a family of Heston pricing solutions indexed by the structural parameters $(\rho, \kappa, \gamma, \bar{v}, r)$, while the current variance v_0 is used only when evaluating the trained network at the present market state. The calibrated market valuation is thus recovered through

$$V_\phi(\tau, m, v_0, \rho^*, \kappa^*, \gamma^*, \bar{v}^*, r).$$

This point is conceptually important: during training, the PINN learns the solution over a domain in $(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$, whereas at inference time the current market valuation is recovered by fixing the variance coordinate to the calibrated value v_0^* and the structural parameters to the calibrated tuple Θ^* .

The network is trained on collocation points sampled in the interior and on the relevant boundaries of the pricing domain. These points are not generated by simulating full paths of the Heston variance process. Instead, they are drawn directly from a bounded domain of admissible states, for instance using Latin Hypercube Sampling (LHS), and used to evaluate the PDE residual. In that sense, the method is unsupervised: the learning signal comes from the Heston structure itself, not from externally computed price labels. This is the main methodological difference with respect to the ANN surrogate developed in Chapter 3.

At a high level, the PINN workflow is organized in three variants. The baseline PINN learns a price surface from the Heston PDE, terminal payoff, and boundary constraints, and is later evaluated at $(\tau, m, v_0^*, \Theta^*, r)$ for a calibrated market configuration. The calibrated tuple may come from the supervised calibration block of Chapter 3, but it is only an external input for PINN inference. The Sobolev PINN then adds derivative information, while the Sobolev+ACV variant adds a local analytical correction near the short-maturity at-the-money region; these extensions are introduced in Sections 4.5 and 4.6.

4.1.1. PINN Theoretical Framework

A PINN can be viewed as a parametric approximation of the solution of a differential equation. In the present setting, the network V_ϕ is required to satisfy the Heston pricing PDE in the interior of the domain and to match the contractual conditions at expiry and on selected boundaries. If $\mathcal{N}$ denotes the Heston differential operator, the target problem can be summarized abstractly as

$$\mathcal{N}[V] = 0 \quad \text{in the interior domain,} \quad \mathcal{B}[V] = 0 \quad \text{on the boundaries,}$$

where $\mathcal{B}$ denotes the set of terminal and boundary constraints.

The PINN replaces the unknown analytical solution by a neural approximation V_ϕ and defines a residual function

$$\mathcal{R}_\phi = \mathcal{N}[V_\phi].$$

Training then consists of minimizing a composite loss built from three sources of error:

$$\mathcal{L}_{\text{PINN}}(\phi) = \mathcal{L}_{\text{PDE}}(\phi) + \mathcal{L}_{\text{term}}(\phi) + \mathcal{L}_{\text{bc}}(\phi). \quad (4.1)$$

Here, $\mathcal{L}_{\text{PDE}}$ penalizes violations of the Heston equation in the interior of the domain, $\mathcal{L}_{\text{term}}$ enforces the payoff at expiry, and $\mathcal{L}_{\text{bc}}$ imposes the selected boundary conditions. This decomposition is standard in the PINN literature and matches the structure adopted in [10].

An essential ingredient of this approach is automatic differentiation. Since the Heston PDE contains first- and second-order derivatives with respect to the state variables, the network output must remain differentiable with respect to its inputs. The required quantities are obtained by differentiating the computational graph of the network rather than by using finite-difference approximations. This is particularly convenient in the present case because the same differentiable surrogate can later be reused for the computation of Greeks.

4.2. Heston Pricing PDE

Starting from the risk-neutral Heston dynamics fixed in Section 2.5, Equation (2.19), we now write the pricing equation that will be enforced by the PINN. We keep the implementation notation v for the variance state, corresponding to ν in Chapter 2, and denote the option value by V . By the Feynman–Kac representation introduced in Equation (2.14), $V(t, S, v)$ satisfies the Heston pricing PDE [3]

$$V_t + \frac{1}{2}vS^2V_{SS} + \rho\gamma vS V_{Sv} + \frac{1}{2}\gamma^2vV_{vv} + rS V_S + \kappa(\bar{v} - v)V_v - rV = 0. \quad (4.2)$$

This is the continuous-time constraint that will be enforced by the PINN at interior collocation points. In the present chapter, the price function is denoted by V precisely to keep it separate from the variance state v .

For implementation purposes, it is more convenient to work with time-to-maturity $\tau = T - t$ and moneyness $m = S/K$ instead of calendar time t and spot S . Since the synthetic and market pipelines of this project use a fixed strike normalization $K = 1$, the pricing problem can be rewritten in terms of the state variables (τ, m, v) . Defining the same pricing function in transformed coordinates, the PDE becomes

$$V_\tau = \frac{1}{2}vm^2V_{mm} + \rho\gamma vmV_{mv} + \frac{1}{2}\gamma^2vV_{vv} + rmV_m + \kappa(\bar{v} - v)V_v - rV. \quad (4.3)$$

This is the formulation that will be used to define the PDE residual in the training loss. It remains fully equivalent to the original Heston pricing equation, but it is better aligned with the representation used in the rest of the thesis.

The PINN is trained on a bounded computational domain

$$\mathcal{D} = [0, \tau_{\max}] \times [m_{\min}, m_{\max}] \times [v_{\min}, v_{\max}],$$

for fixed or sampled values of the Heston parameters. On this domain, the baseline loss enforces the PDE in the interior, the put payoff at $\tau = 0$, and the lower-moneyness boundary at $m = 0$. The upper-moneyness and variance boundaries are not imposed as separate hard constraints in the baseline implementation; their effect is controlled through the bounded sampling domain and the interior residual. This choice keeps the baseline PINN close to the minimal Heston-PINN formulation used in [10], while leaving additional boundary constraints as possible extensions.

For a European put option with normalized strike $K = 1$, the terminal condition at expiry follows directly from the payoff in Equation (2.1) and is

$$V(0, m, v) = (1 - m)^+. \quad (4.4)$$

The payoff does not depend explicitly on the variance state, which means that the terminal condition is imposed uniformly over the v -dimension of the domain. In addition, a natural lower boundary condition is obtained at zero underlying value:

$$V(\tau, 0, v) = e^{-r\tau}. \quad (4.5)$$

For a general strike K , this condition would read $V(\tau, 0, v) = Ke^{-r\tau}$. The far-field behavior as $m \rightarrow \infty$ for a put, namely $V(\tau, m, v) \rightarrow 0$, can also be incorporated if a stricter boundary specification is required.

4.3. Neural Network Architecture

4.3.1. Model Class

The pricing PINN is implemented as a fully connected feed-forward neural network (MLP) that approximates the mapping

$$V_\phi : \mathbb{R}^8 \rightarrow \mathbb{R}, \quad x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r) \mapsto V_\phi(x),$$

where the scalar output is the option price. This architecture is appropriate for the present setting because all inputs are continuous tabular variables and no spatial or sequential inductive bias is required. This is also the standard architectural choice in many PINN implementations, where a smooth MLP acts as the differentiable trial function for the PDE solution [10], [15].

Compared with Chapter 3, where the ANN surrogate learned implied volatility from supervised labels, the role of the network here is to represent a differentiable pricing function that is constrained by the Heston PDE and boundary conditions. The model class is therefore selected not only for approximation capacity, but also for stable first- and second-order automatic differentiation. This differentiability requirement is central when PDE residuals or derivative losses are part of the objective, as in gradient-enhanced and Sobolev-style PINN variants [11], [16].

4.3.2. Architecture Details

The reference architecture used in the implementation is

$$8 \rightarrow 256 \rightarrow 256 \rightarrow 256 \rightarrow 256 \rightarrow 1,$$

with `tanh` activation in hidden layers and linear activation in the output layer. In compact notation,

$$h^{(1)} = \varphi(W^{(1)}x + b^{(1)}), \dots, h^{(4)} = \varphi(W^{(4)}h^{(3)} + b^{(4)}), \quad V_\phi = W^{(5)}h^{(4)} + b^{(5)},$$

with $\varphi = \tanh$.

Weights are initialized with Xavier uniform initialization and biases are initialized to zero. In the baseline setup, dropout is disabled and no batch-normalization layers are included.

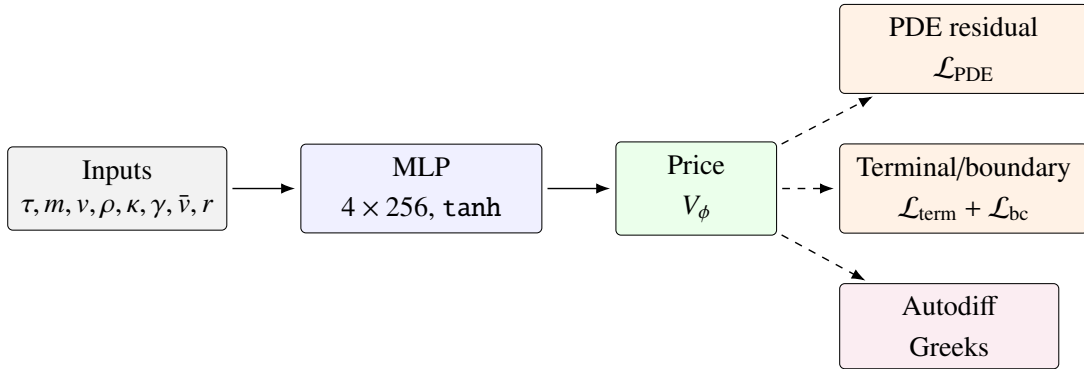


Fig. 4.1. PINN architecture used for Heston pricing. The central MLP block represents the four hidden layers of width 256 with `tanh` activation, followed by a linear output layer. It maps the eight-dimensional input $(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$ to the scalar price V_ϕ . The right-hand branches show how this same output is used during training and evaluation: automatic differentiation gives the derivatives entering the PDE residual, direct evaluation at terminal and boundary points gives the contractual losses, and automatic differentiation of the trained price surface gives the Greeks.

TABLE 4.1. PINN ARCHITECTURE USED IN THIS CHAPTER

Hyperparameter	Value
Input dimension	8
Hidden layers	4
Hidden width	256 neurons per layer
Hidden activation	$\tanh$
Output dimension	1
Output activation	Linear
Weight initialization	Xavier uniform
Bias initialization	Zeros
Dropout	$p = 0.0$
Batch normalization	Disabled

Source: Own implementation (`configs/pinn_model_architecture.yaml`)

4.3.3. Model Inputs and Outputs

The model receives the following ordered input vector:

$$x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r),$$

where (τ, m, v) are state variables, r is the risk-free rate, and $(\rho, \kappa, \gamma, \bar{v})$ are the structural Heston parameters that define the PDE operator.

The network output is a scalar price $V_\phi(x)$, corresponding to the normalized-strike vanilla contract considered in this chapter ($K = 1$). This output choice is consistent with the PDE-constrained formulation: the PINN directly approximates the pricing function rather than an implied-volatility transformation. The trade-off is that prices are less homogeneous than implied volatility, because they retain contract-scale effects more directly. This makes the learned surface more delicate near short maturities, and especially around the payoff kink at $m \approx 1$, which later motivates the local ACV correction. At inference time, the CaNN calibration block supplies the market-specific values of $(\rho, \kappa, \gamma, \bar{v}, v_0)$; the PINN then evaluates the contract by fixing $v = v_0$ and varying the contract coordinates (τ, m) and the market rate r .

In the current experimental setup, collocation generation is run with fixed values of $(\rho, \kappa, \gamma, \bar{v})$, while (τ, m, v, r) vary across points. This means the model remains formally eight dimensional, but some coordinates may be constant in a given run due to calibration conditioned sampling. This is the baseline fixed Θ benchmark used in the reported experiments; the same input convention leaves the fully parametric variant available as a direct extension.

Input Scaling

Although the pricing problem is formulated in physical variables, the optimization is performed with an affine rescaling of the network inputs. The motivation is numerical: (τ, m, v) and the Heston-related coordinates do not share comparable magnitudes, which can deteriorate conditioning and gradient balance. Following practical PINN recommendations, centering and scaling are used as a stabilization device; this is consistent with the broader observation that conditioning and gradient-scale imbalance strongly affect PINN training dynamics [13], [14], [15].

In the implemented pipeline, the transformation is applied componentwise to the full PINN input vector through

$$\tilde{x}_d = a_d + b_d x_d, \quad d = 1, \dots, 8.$$

The coefficients are estimated from the collocation points of the training split:

$$\mu_d = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} x_{i,d}, \quad \sigma_d = \sqrt{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (x_{i,d} - \mu_d)^2},$$

and then

$$a_d = -\frac{\mu_d}{\sigma_d}, \quad b_d = \frac{1}{\sigma_d}.$$

To avoid numerical issues on nearly constant coordinates, the implementation uses a small threshold $\varepsilon = 10^{-8}$: if $\sigma_d \leq \varepsilon$, that coordinate is left unscaled ($a_d = 0$, $b_d = 1$).

The PDE residual is still defined in physical variables and differentiated by automatic differentiation. In code, the network is evaluated at $\tilde{x}$, while derivatives are taken with respect to the original coordinates x ; the chain rule is therefore handled by the computational graph. Consequently, input scaling changes the numerical conditioning of training, but not the financial meaning of the PDE constraint.

4.4. Training Procedure

4.4.1. Loss Function

The PINN is trained by minimizing a composite loss that enforces the pricing PDE in the interior of the domain together with the terminal payoff and the lower boundary condition. Let $V_\phi(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$ denote the output of the neural network. From the transformed Heston PDE introduced above, the residual at an interior point is defined as

$$\mathcal{R}_\phi = V_\tau - \frac{1}{2}vm^2V_{mm} - \rho\gamma vmV_{mv} - \frac{1}{2}\gamma^2vV_{vv} - rmV_m - \kappa(\bar{v} - v)V_v + rV, \quad (4.6)$$

where all derivatives are evaluated by automatic differentiation on the computational graph of the network. In the ideal case, the exact pricing function would satisfy $\mathcal{R}_\phi = 0$ for every admissible interior point.

Let $\mathcal{S}_D = \{z_j\}_{j=1}^{N_D}$ denote the set of interior collocation points, with

$$z_j = (\tau_j, m_j, v_j, \rho_j, \kappa_j, \gamma_j, \bar{v}_j, r_j).$$

The interior loss is the mean squared residual

$$\mathcal{L}_{\text{PDE}}(\phi) = \frac{1}{N_D} \sum_{j=1}^{N_D} \mathcal{R}_\phi(z_j)^2. \quad (4.7)$$

On the terminal set $\mathcal{S}_T = \{z_k^T\}_{k=1}^{N_T}$, where $\tau = 0$, the network must match the payoff of the European put. The corresponding error is

$$e_k^T(\phi) = V_\phi(0, m_k, v_k, \rho_k, \kappa_k, \gamma_k, \bar{v}_k, r_k) - (1 - m_k)^+, \quad (4.8)$$

and the terminal loss is defined by

$$\mathcal{L}_{\text{term}}(\phi) = \frac{1}{N_T} \sum_{k=1}^{N_T} \left(e_k^T(\phi)\right)^2. \quad (4.9)$$

On the lower boundary set $\mathcal{S}_{\text{low}} = \{z_l^{\text{low}}\}_{l=1}^{N_{\text{low}}}$, where $m = 0$, the exact put price is the discounted strike. Since the implementation uses the normalized strike $K = 1$, the lower-boundary error is

$$e_l^{\text{low}}(\phi) = V_\phi(\tau_l, 0, v_l, \rho_l, \kappa_l, \gamma_l, \bar{v}_l, r_l) - e^{-r_l \tau_l}, \quad (4.10)$$

and the associated loss is

$$\mathcal{L}_{\text{bc}}(\phi) = \frac{1}{N_{\text{low}}} \sum_{l=1}^{N_{\text{low}}} \left(e_l^{\text{low}}(\phi)\right)^2. \quad (4.11)$$

The total training objective is then written as

$$\mathcal{L}_{\text{PINN}}(\phi) = \mathcal{L}_{\text{PDE}}(\phi) + \mathcal{L}_{\text{term}}(\phi) + \mathcal{L}_{\text{bc}}(\phi). \quad (4.12)$$

For implementation, it is convenient to keep an explicit weighted form,

$$\mathcal{L}_{\text{PINN}}(\phi) = \lambda_{\text{PDE}} \mathcal{L}_{\text{PDE}}(\phi) + \lambda_{\text{term}} \mathcal{L}_{\text{term}}(\phi) + \lambda_{\text{bc}} \mathcal{L}_{\text{bc}}(\phi), \quad (4.13)$$

with non-negative coefficients $(\lambda_{\text{PDE}}, \lambda_{\text{term}}, \lambda_{\text{bc}})$ that can be tuned if one component dominates the optimization. In the baseline specification of this thesis, all three weights are initialized to one, i.e. $(1, 1, 1)$, following the simplest formulation in [10].

A practical point is that the PDE term requires second-order derivatives with respect to inputs. Therefore, the automatic-differentiation graph must be kept when computing first derivatives, so that mixed and second derivatives such as V_{mv} and V_{vv} can be obtained consistently in the same computational graph. This is the key implementation detail that enables direct optimization of the PDE residual without finite-difference approximations. A cleaned implementation excerpt of the baseline PDE residual construction is given in Appendix C.

4.4.2. Optimizer and Hyperparameters

The optimization follows a two-stage strategy that is common in PINN practice: an initial stochastic phase with Adam, followed by a quasi-Newton refinement with L-BFGS. In this work, this strategy is implemented as a mixed schedule in which Adam is used during the first part of training and L-BFGS in the second part, while a pure Adam or pure L-BFGS run remains available as an ablation. In the retained baseline run, this corresponds to the `mix_half` mode: Adam is used for epochs 1–2000, and full-batch L-BFGS is used from epoch 2001 to epoch 4000.

During the Adam phase, the three loss terms are evaluated on separate mini-batches: one from $\mathcal{S}_D$ (interior), one from $\mathcal{S}_T$ (terminal), and one from $\mathcal{S}_{\text{low}}$ (lower boundary). This allows different batch sizes for interior and boundary constraints and keeps the training loop aligned with the composite objective. During the L-BFGS refinement, the same loss components are kept, but the retained configuration uses full-batch evaluation.

The reference hyperparameter block for the first implementation includes: learning rates and optimizer-specific settings, the Adam-to-L-BFGS switching epoch in mixed mode, batch sizes for interior and boundary sets, and the triplet of loss weights $(\lambda_{\text{PDE}}, \lambda_{\text{term}}, \lambda_{\text{bc}})$.

For diagnostics, the training run stores epoch-wise train/validation curves for the total weighted loss and for each component separately. In addition, a two-dimensional error map over (m, τ) is produced from validation interior points by averaging $|\mathcal{R}_\phi|$ in bins. This map helps identify whether specific regions of the contract domain remain systematically underfitted even when the global loss decreases.

4.4.3. Sampling strategy

Training uses three collocation sets, one for each component of the baseline loss. These sets are not built from labeled prices; they are points drawn directly in the state and parameter domain where the PDE, terminal condition, or boundary condition is evaluated. Interior points are used for the PDE residual, terminal points lie on $\tau = 0$ and impose the payoff, and lower boundary points lie on $m = 0$ and impose the discounted strike condition.

The retained baseline uses a fixed Θ sampling regime. The structural Heston parameters $(\rho, \kappa, \gamma, \bar{v})$ are fixed to the calibrated tuple used in the benchmark, while the sampled coordinates are (τ, m, v, r) . Therefore, although the network input remains eight dimensional, the effective computational domain explored by the baseline run is

$$\tau \in [0, 3], \quad m \in [0, 2], \quad v \in [0.01, 0.5], \quad r \in [0, 0.05],$$

with $(\rho, \kappa, \gamma, \bar{v})$ kept constant. The v -dimension is still sampled explicitly as a state coordinate of the PDE. This is a crucial point: the PINN is trained on generic variance states v , whereas the calibrated initial variance v_0 is only used later, when evaluating the trained network at the current market state.

To obtain a representative coverage of this domain with a controlled number of points, the retained baseline uses the same LHS principle introduced in Section 3.2, but applied here to the PINN collocation sets rather than to supervised pairs. In the present setting, this is useful because the PINN must cover maturity, moneyness, variance state, and interest rate levels under the fixed calibrated Heston configuration.

For the interior collocation set $\mathcal{S}_D$, each point has the form

$$z_j = (\tau_j, m_j, v_j, \rho_j, \kappa_j, \gamma_j, \bar{v}_j, r_j), \quad j = 1, \dots, N_D.$$

The structural parameter entries are kept constant across these points in the retained fixed Θ run. For the terminal collocation set $\mathcal{S}_T$, points are sampled in the subdomain

$$\tau = 0, \quad m \in [0, 2], \quad v \in [0.01, 0.5],$$

again with variable r and fixed $(\rho, \kappa, \gamma, \bar{v})$. For the lower boundary collocation set $\mathcal{S}_{\text{low}}$, points satisfy

$$m = 0, \quad \tau \in [0, 3], \quad v \in [0.01, 0.5],$$

again with variable r and the same fixed Heston parameter tuple.

In the retained baseline, the three collocation sets are precomputed once and reused across training. This static design simplifies reproducibility and keeps the comparison with the Sobolev refinement stage controlled. Dynamic resampling may improve coverage of the domain and reduce overfitting to a fixed cloud of collocation points, but it is not part of the reported baseline.

This sampling strategy preserves the fully unsupervised nature of the method. No prices from COS, FFT, or Monte Carlo are required to build the training objective. Numerical pricing methods remain useful for ex post validation and benchmarking, but not as a source of labels for fitting the PINN itself. This statement applies to the baseline PINN. The Sobolev variant later adds semi analytical Greek targets, and the Sobolev+ACV variant adds a local analytical correction; these additions are not part of the baseline training objective.

4.5. Sobolev-Enhanced Training for Greek Accuracy

The second PINN variant retained in the final benchmark is the Sobolev PINN. It starts from the baseline price space PINN of the previous section and adds derivative supervision on Greek anchor points. The purpose is not to redefine the pricing problem or to present a uniformly better pricer, but to test whether explicit sensitivity information can make the PINN Greek estimates more reliable.

This extension should therefore be read as a derivative oriented correction. Accurate prices do not automatically imply accurate first and second order sensitivities: Gamma and variance/rate sensitivities may show larger local errors because they depend on curvature properties of the learned surface. This issue is also observed in recent physics

informed Heston operator learning, where autodiff Greeks become noisy even when price errors are small [12]. Sobolev training can reduce Greek errors relative to the baseline PINN, but it also introduces an additional loss balancing problem and may affect the price/PDE fit. For this reason, the benchmark later evaluates pricing and Greeks separately.

The formulation below is an application specific hybrid inspired by Sobolev training [16] and gradient enhanced PINN (gPINN) formulations [11]. It follows Sobolev Training in matching target derivatives, but unlike gPINN it does not penalize gradients of the PDE residual directly.

The Sobolev variant keeps the original physics objective and adds supervised derivative terms on a dedicated set of Greek anchor points. Let

$$\mathcal{L}_{\text{base}} = \lambda_{\text{PDE}} \mathcal{L}_{\text{PDE}} + \lambda_{\text{term}} \mathcal{L}_{\text{term}} + \lambda_{\text{bc}} \mathcal{L}_{\text{bc}},$$

The two Greek losses below are adapted from Sobolev training [16] and derivative enriched PINN objectives such as gPINNs [11]. What we add here is the use of semi analytical Heston Greeks as derivative targets, together with a normalized split between first order Greeks and Gamma:

$$\mathcal{L}_{\text{g},1} = \frac{1}{|\mathcal{S}_{\text{g}}| |\mathcal{Q}_1|} \sum_{x \in \mathcal{S}_{\text{g}}} \sum_{q \in \mathcal{Q}_1} \left(\frac{q_{\phi}(x) - q^{\star}(x)}{s_q + \varepsilon} \right)^2, \quad (4.14)$$

$$\mathcal{L}_{\text{g},2} = \frac{1}{|\mathcal{S}_{\text{g}}|} \sum_{x \in \mathcal{S}_{\text{g}}} \left(\frac{\text{Gamma}_{\phi}(x) - \text{Gamma}^{\star}(x)}{s_{\text{Gamma}} + \varepsilon} \right)^2, \quad (4.15)$$

where $\mathcal{L}_{\text{g},1}$ and $\mathcal{L}_{\text{g},2}$ are normalized Sobolev derivative matching losses, $\mathcal{S}_{\text{g}}$ is the Greek anchor set, $x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$ is a PINN input point, and $\mathcal{Q}_1 = \{\text{Delta}, \text{Vega}_v, \text{Theta}, \text{Rho}\}$ collects the first order Greeks supervised in $\mathcal{L}_{\text{g},1}$. For each $q \in \mathcal{Q}_1$, $q_{\phi}(x)$ is obtained by automatic differentiation of the PINN price surface V_{ϕ} , while $q^{\star}(x)$ is the corresponding semi analytical Heston Greek computed from the characteristic function, as described in Section 2.7.2. The constants s_q and s_{Gamma} are fixed scale factors, for example robust quantiles of the reference Greek magnitudes on $\mathcal{S}_{\text{g}}$, and $\varepsilon > 0$ prevents division by values close to zero. The Gamma term is kept separate because it is a second order sensitivity and is typically more unstable than the first order Greeks grouped in $\mathcal{Q}_1$.

The Sobolev PINN objective is then

$$\mathcal{L}_{\text{Sob}} = \mathcal{L}_{\text{base}} + \lambda_{\text{g},1} \mathcal{L}_{\text{g},1} + \lambda_{\text{g},2} \mathcal{L}_{\text{g},2}. \quad (4.16)$$

From an optimization viewpoint, this formulation introduces additional imbalance risks between loss terms, already documented in the PINN literature [13], [14], [15]. The retained implementation therefore uses the trained baseline PINN as initialization and then applies a Sobolev refinement phase with Greek anchor losses, while monitoring possible regressions in price/PDE metrics.

A stricter physics-informed alternative would be to differentiate the Heston PDE itself with respect to the state variables and penalize the residuals of the PDEs satisfied by the sensitivities. This is the type of Sobolev or sensitivity-equation route suggested in [12]. It has the advantage of reducing the dependence on external Greek labels, but it also introduces higher-order derivatives and additional loss-balancing difficulties. The experiment in this thesis therefore uses semi-analytical Heston Greeks as derivative targets: this makes the diagnostic controlled and directly comparable with the reference engine, while leaving sensitivity-PDE training as a natural extension.

4.6. Analytical Control Variate Correction

The Sobolev variant of Section 4.5 reduces the derivative error of the PINN by adding explicit Greek information to the training objective. However, the most difficult region for the learned price surface remains structurally tied to the payoff itself. For a put option, the terminal condition introduced in Equation (2.1) has a kink at $m = 1$ and $\tau = 0$, as illustrated in Figure 2.1.

The short-maturity at-the-money layer therefore contains a sharp curvature profile. Since second-order sensitivities such as Gamma are obtained by differentiating the learned surface, small price irregularities in this layer can generate disproportionately large Greek errors. This motivates a third variant, denoted PINN Sobolev+ACV, in which the Sobolev PINN is kept as the global model and an analytical control variate is activated only near the kink. Control variates are most commonly introduced as variance-reduction devices in Monte Carlo methods [5], and related ideas have also been used in Heston pricing to stabilize characteristic-function based numerical procedures [28].

Here the control variate is used in a different way: not to stabilize a numerical integral, but to patch locally the part of the Sobolev PINN surface where autodiff Greeks are most fragile. Thus, Sobolev+ACV is a hybrid correction for this benchmark, combining global Sobolev supervision with a local analytical correction near the short-maturity at-the-money kink.

The correction is written in log-moneyness coordinates

$$x = \log m,$$

because the singular layer around the strike is naturally centered at $x = 0$. This is the same log-moneyness convention used in the Heston-COS formulation of Section 2.7, but here it is introduced only to center the local correction around the strike. Let $V_{\text{Sob}}(z)$ be the price produced by the Sobolev PINN at the input vector

$$z = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r).$$

The vector z contains the time to maturity, moneyness, current variance state, structural Heston parameters, and risk-free rate at which the global Sobolev surface is evaluated.

Following the standard control-variate decomposition [5], the ACV model combines this global price with a local Black–Scholes put approximation $V_{\text{BS}}^{\text{put}}$, using the pricing convention fixed in Section 2.4 and the instantaneous variance v as the local variance level [2]:

$$V_{\text{ACV}}(z) = (1 - \chi(\tau, x)) V_{\text{Sob}}(z) + \chi(\tau, x) V_{\text{BS}}^{\text{put}}(\tau, x, v, r). \quad (4.17)$$

In this expression, V_{ACV} is the corrected price surface, $\chi(\tau, x) \in [0, 1]$ is the localization gate, V_{Sob} is the global Sobolev PINN price, and $V_{\text{BS}}^{\text{put}}$ is the analytical local component. Therefore, the reported Sobolev+ACV experiment isolates the effect of the analytical control variate and of the local blending mechanism, rather than adding another supervised Greek-fitting stage.

Using the normalized-strike convention of Chapter 2, the Black–Scholes control variate is obtained from the put formula in Equation (2.16) by setting $K = 1$, $S = e^x$, and $\sigma^2 = v$:

$$V_{\text{BS}}^{\text{put}}(\tau, x, v, r) = e^{-r\tau} \Phi(-d_2) - e^x \Phi(-d_1), \quad (4.18)$$

with

$$d_1 = \frac{x + (r + \frac{1}{2}v)\tau}{\sqrt{v\tau + \varepsilon_{\text{BS}}}}, \quad d_2 = d_1 - \sqrt{v\tau + \varepsilon_{\text{BS}}}, \quad (4.19)$$

which is the same d_1, d_2 construction as in Equations (2.16)–(2.17), with the additional numerical floor $\varepsilon_{\text{BS}} > 0$ to avoid division by zero when $v\tau$ is very small. This expression is not used as a global replacement for the Heston price. It is only a local analytical component whose role is to encode the leading short-maturity payoff geometry around the strike.

Following the standard use of sigmoidal functions as smooth approximations to step-like transitions [25], the localization is controlled by the gate

$$\chi(\tau, x) = \sigma\left(\frac{x_c - |x|}{\delta_x}\right) \sigma\left(\frac{\tau_c - \tau}{\delta_\tau}\right), \quad (4.20)$$

where σ is the logistic function, x_c and τ_c set the half-width of the active region in log-moneyness and maturity, and $\delta_x, \delta_\tau > 0$ control how smooth the transition is. Thus χ is close to one near short maturities and near $x = 0$, and it decays smoothly toward zero outside the patch. This avoids a discontinuous transition between the analytical approximation and the global Sobolev PINN.

For evaluation, the hard region is defined as

$$\mathcal{D}_{\text{hard}} = \{(m, \tau) : |\log(m)| < 0.03, \tau < 0.05\}. \quad (4.21)$$

The thresholds in Equation (4.21) select the short-maturity at-the-money neighborhood where the payoff kink dominates the derivative error: $|\log(m)| < 0.03$ keeps the grid close to the strike, and $\tau < 0.05$ restricts the diagnostic to very short maturities. This region is not a new training domain for the baseline PINN; it is a diagnostic set designed to measure whether the control variate reduces the outlier contribution generated by the payoff-kink

layer. The broader PINN surface is still provided by V_{Sob} , and reference Greeks for the evaluation are computed with the characteristic-function machinery summarized in Section 2.7.2. The empirical effect of ACV is therefore assessed separately in Chapter 5, where the local hard-region reading is compared with the global error metrics.

4.7. Greeks Computation

The Greek definitions used in this thesis were fixed in Section 2.5, and the semi-analytical Heston references used for validation were summarized in Section 2.7.2. In the PINN branch, the additional point is computational: once the trained network represents the price surface $V_\phi(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$, the required sensitivities are obtained by automatic differentiation of this learned surface.

In this work, the spatial coordinate is moneyness $m = S/K$, not spot directly. Hence a chain-rule conversion is required:

$$\frac{\partial V_\phi}{\partial S} = \frac{\partial V_\phi}{\partial m} \frac{\partial m}{\partial S} = \frac{1}{K} \frac{\partial V_\phi}{\partial m}, \quad (4.22)$$

$$\frac{\partial^2 V_\phi}{\partial S^2} = \frac{1}{K^2} \frac{\partial^2 V_\phi}{\partial m^2}. \quad (4.23)$$

For the normalized-strike setup used in the thesis ($K = 1$), both coordinate systems coincide numerically.

The derivative computation is based on automatic differentiation over the computational graph of the trained network. For a single input point

$$x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r),$$

we compute

$$V_\phi(x), \quad J(x) = \nabla_x V_\phi(x), \quad H(x) = \nabla_x^2 V_\phi(x).$$

Because the model is differentiable end-to-end, these quantities are exact derivatives of the neural surrogate, up to floating-point precision, and not finite-difference approximations. This exactness is internal to the trained network: automatic differentiation differentiates V_ϕ , not the exact Heston price function. Therefore, the resulting Greeks are reliable only to the extent that the learned surface reproduces the local geometry of the Heston reference solution. This distinction is precisely what motivates the Sobolev and ACV variants introduced in Sections 4.5 and 4.6.

Compared with bump-and-reprice finite differences, this avoids choosing artificial perturbation sizes and removes the associated truncation/cancellation trade-off. It also keeps first- and second-order sensitivities within the same computational graph, which is useful when the evaluation is performed on batches of (m, τ) points. From a financial perspective, the interpretation remains standard: each Greek is the local slope or curvature of the price surface produced by the PINN around the evaluation point. The quality of those

sensitivities therefore depends on both the local smoothness of the trained network and the fidelity of the learned price function in that region. This closes the link with the Heston dynamics introduced in Chapter 2: the PINN price surface is trained through the PDE implied by Equation 2.19, and the Greeks are then derivatives of that model-consistent learned surface.

5. NEURAL NETWORK PERFORMANCE AND BENCHMARKING

5.1. Experimental Setup And Evaluation Protocol

This chapter reports the final empirical comparison of the neural models developed in the thesis. The results are organized around two methodological branches. The first branch is supervised and works with implied-volatility labels: it includes the ANN baseline and the ANN Sobolev variant, together with the CaNN calibration workflow that uses the trained ANN surrogate as a fast evaluator. The second branch is physics-informed and works directly in price space: it includes the PINN baseline, the PINN Sobolev variant, and the PINN Sobolev variant including an analytical control variate (ACV) correction.

Unless explicitly stated, all reported metrics are computed on fixed evaluation sets with identical feature conventions, consistent preprocessing, and the same numerical references used in previous chapters.

5.1.1. Primary Error Metrics

To keep the empirical comparison coherent, the chapter uses three primary error metrics throughout: MSE, RMSE, and MAPE. Let z_i^{ref} be the reference value and $\hat{z}_i$ the model value at evaluation point i , for $i = 1, \dots, n$. The metrics are

$$\begin{aligned} \text{MSE} &= \frac{1}{n} \sum_{i=1}^n (\hat{z}_i - z_i^{\text{ref}})^2, \\ \text{RMSE} &= \sqrt{\text{MSE}}, \\ \text{MAPE} &= 100 \frac{1}{n} \sum_{i=1}^n \frac{|\hat{z}_i - z_i^{\text{ref}}|}{\max(|z_i^{\text{ref}}|, \varepsilon_{\text{MAPE}})}. \end{aligned} \tag{5.1}$$

MSE is aligned with the training objective used in the supervised components, RMSE restores the error to the natural output units, and MAPE provides a scale-relative percentage reading. The denominator floor $\varepsilon_{\text{MAPE}}$ avoids unstable ratios near zero. Tail quantiles and maximum errors are used only as auxiliary diagnostics when discussing distributional behavior, not as headline metrics.

The minimal pre-benchmark validation protocol combines three checks: convergence with respect to (N, L) in the COS expansion, put-call parity residual control based on Equation (2.2), and Black–Scholes consistency in near-constant-variance regimes.

These checks are used as a guardrail before comparing models, so that downstream differences can be attributed to model behavior rather than to numerical artifacts. For consistency across Sections 5.2–5.3.2, each block is analyzed with the same reading logic:

global summary metrics, localized error diagnostics over relevant domains, and runtime indicators.

The protocol is fixed as follows. Randomness is controlled with a common seed (42) in data generation and training configurations; in calibration, the Differential Evolution seed follows the same base value unless explicitly overridden. The Heston domain is shared across experiments, with $\rho \in [-0.9, 0]$, $\kappa \in [0, 3]$, $\gamma \in [0.01, 0.8]$, $\bar{v} \in [0.01, 0.5]$, $v_0 \in [0.05, 0.5]$, and contract coordinates $m \in [0.6, 1.4]$, $\tau \in [0.05, 3.0]$ for ANN/CaNN benchmarks. Reference numerics are also fixed: COS pricing with $N = 1500$, $L = 50$, and fallback $L = 80$ when required by stability checks, plus Brent IV inversion with $\text{tol} = 10^{-8}$ and bounded search interval.

Reported metrics are always computed on fixed evaluation sets. For the supervised ANN branch, the split policy is fixed to 0.8/0.1/0.1 (train/validation/test). For the PINN branch, the collocation data use a train/validation split rather than supervised labels; the final configurations use 0.8/0.2, and pricing or Greek metrics are then reported on fixed benchmark grids. Values are reported in decimal units (IV for ANN/CaNN, price for PINN). These units are not used to build a direct ANN-versus-PINN ranking: ANN results are read in implied-volatility space, with price or Greek diagnostics reported as derived quantities, whereas PINN results are read first in price space and then through separate Greek diagnostics.

Methodological details are kept in the corresponding method chapters: Chapter 3 for the supervised ANN/CaNN block and Chapter 4 for PINN. This chapter focuses on consolidated benchmark outcomes under the protocol above.

5.2. Supervised ANN Benchmarks

This section reports the performance of the supervised ANN benchmarks introduced in Chapter 3. The objective is to assess mean implied-volatility accuracy and the distribution of the IV error over (m, τ) , using a fully specified evaluation protocol that can be reproduced without implementation-specific references. The section first fixes the ANN baseline, then compares it with the Sobolev-augmented ANN, and finally uses the selected surrogate inside the CaNN calibration workflow.

The baseline ANN analyzed here corresponds to the selected tanh configuration from Chapter 3. The benchmark uses fixed weights (no retraining in Chapter 5) and fixed train, validation and test splits, with $8.0 \cdot 10^5$ training samples, $1.0 \cdot 10^5$ validation samples, and $1.0 \cdot 10^5$ test samples, respectively. Pointwise error is defined as

$$e = \hat{\sigma}_{\text{IV}} - \sigma_{\text{IV}}^{\text{ref}},$$

where $\hat{\sigma}_{\text{IV}}$ is the ANN-predicted implied volatility and $\sigma_{\text{IV}}^{\text{ref}}$ is the reference implied volatility obtained from the Heston-COS/Brent pipeline. The error e is reported in decimal implied-volatility units.

Global quality is summarized with MSE, RMSE, and MAPE. Regional diagnostics use fixed bins over m and τ , and extrapolation behavior is analyzed through sensitivity maps with explicit interpolation limits (Figure 5.1).

Before reporting the final split-wise metrics, Table 5.1 summarizes the retained candidates used to select the reference ANN baseline, using validation and test RMSE together with test MAPE under the same train/validation/test split and Liu-style network size. The table uses descriptive names rather than internal run identifiers: the activation and update schedule are the relevant differences. The Sobolev model is not included in this optimizer-selection table because it answers a different question: it starts from the supervised ANN setting and adds derivative information, so it is evaluated separately in the following subsections.

TABLE 5.1. ANN BASELINE OPTIMIZER SELECTION.

Candidate	Activation	Update schedule	Val RMSE	Test RMSE	Test MAPE
Selected tanh ANN	tanh	Adam	$2.00 \cdot 10^{-3}$	$1.86 \cdot 10^{-3}$	15.88%
ReLU baseline	ReLU	Adam	$2.16 \cdot 10^{-3}$	$3.03 \cdot 10^{-3}$	18.33%
Adam-L-BFGS refinement	ReLU	Adam $\rightarrow$ L-BFGS	$2.13 \cdot 10^{-3}$	$3.07 \cdot 10^{-3}$	27.17%

Source: Own elaboration from stored ANN evaluation summaries

5.2.1. ANN Baseline

Table 5.2 reports split-wise global errors for the selected ANN baseline. This model is used as the supervised reference for the rest of the ANN benchmark.

TABLE 5.2. ANN BASELINE GLOBAL IV ERRORS.

Split	n	RMSE	MAPE (%)
Train	$8.0 \cdot 10^5$	$2.64 \cdot 10^{-4}$	2.21
Validation	$1.0 \cdot 10^5$	$2.00 \cdot 10^{-3}$	18.12
Test	$1.0 \cdot 10^5$	$1.86 \cdot 10^{-3}$	15.88

Source: Own elaboration

The test RMSE remains in the 10^{-3} range in absolute IV units, improving by 38.5% with respect to the earlier ReLU Liu-style reference run ($3.03 \cdot 10^{-3}$ to $1.86 \cdot 10^{-3}$). MAPE is larger than the raw IV error scale suggests because relative errors are amplified in low-IV regions; therefore it is read jointly with the spatial diagnostics below rather than as an isolated scalar.

Figure 5.1 provides a panel-wise diagnostic. Each row corresponds to one parameter plane: top $(\gamma, \bar{v})$ (vol-of-vol versus long-run variance), middle (ρ, γ) (correlation

versus vol-of-vol), and bottom $(\kappa, \bar{v})$ (mean reversion versus long-run variance). The left column shows ANN IV maps with white isoclines, while the right column shows $\log |NN - \text{Heston}|$ on the same grids, using lighter tones for lower errors and darker tones for larger errors. The dashed black lines delimit interpolation limits; regions beyond those limits are extrapolation zones where the network was not trained.

The most stable behavior is observed in the interior of each plane, where isoclines are smooth and error colors remain comparatively light. Error increases near boundary segments and especially outside the dashed limits, with the strongest deterioration concentrated in outer corners of the tested planes. This is consistent with the regional-error reading: the surrogate remains accurate over the interpolation core, while low-moneyness/short-maturity-like boundary regimes are the most fragile. In practical terms, Figure 5.1 supports using the ANN as a robust interpolation surrogate and treating extrapolated regions as lower-confidence zones.

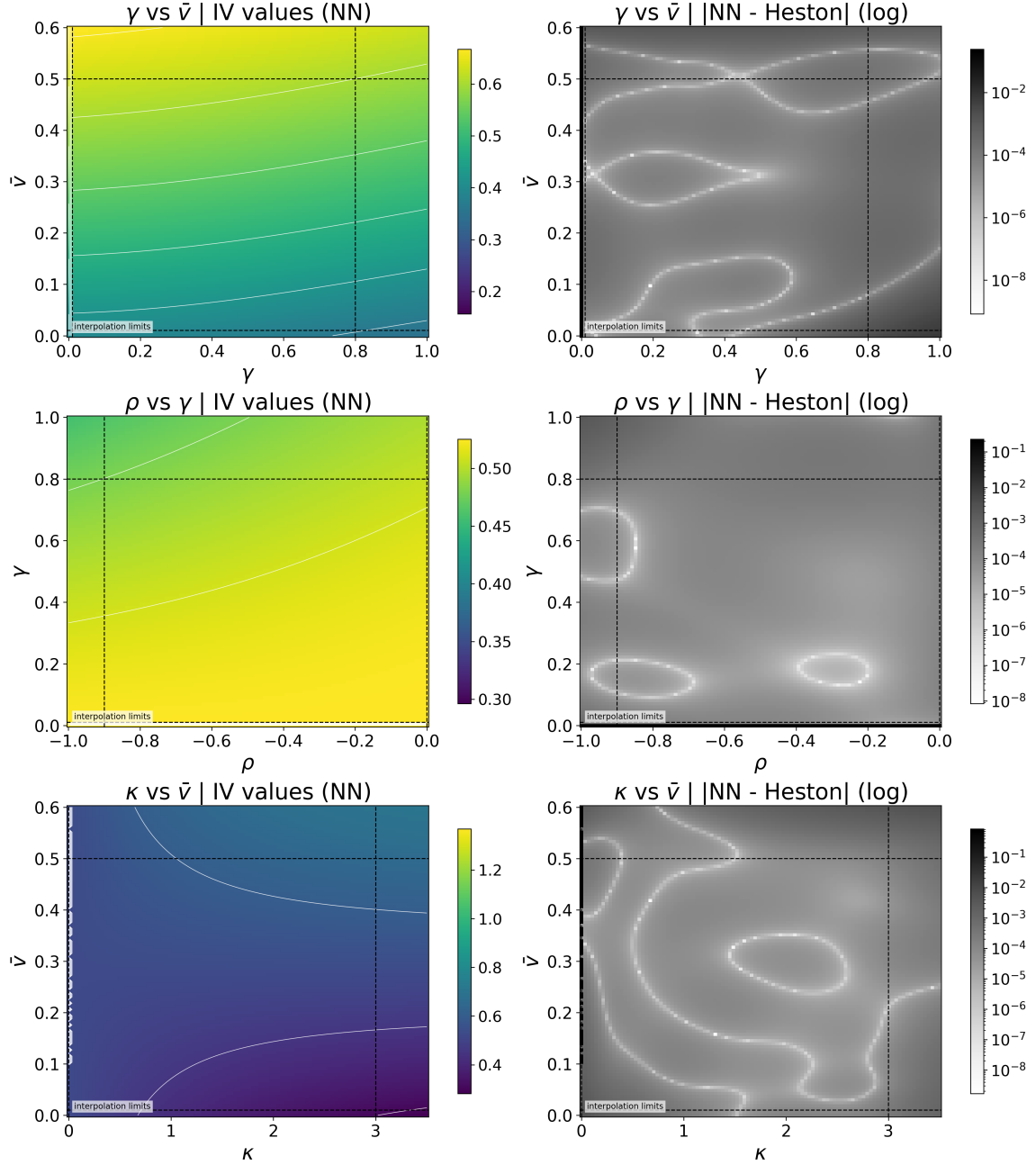


Fig. 5.1. Sensitivity grid for the ANN pricer. Top row: $(\gamma, \bar{v})$, middle row: (ρ, γ) , bottom row: $(\kappa, \bar{v})$. Left panels report IV-value maps with white isoclines; right panels report $\log |NN - \text{Heston}|$, where lighter tones indicate lower error and darker tones higher error. Dashed black lines indicate interpolation limits.

Regional Error Profile

For the baseline ANN, regional diagnostics identify where approximation error concentrates. On the test split, the largest MSE values are concentrated at low moneyness and short maturity: the bin $m \in [0.6, 0.8]$ accounts for roughly one quarter of test points, while $\tau \in [0.05, 0.25]$ accounts for roughly 6.8%. Interior and longer-horizon bins show a clear error drop, so the pattern is structural rather than driven by isolated rare points.

The same pattern appears in the regional MSE curves by moneyness and maturity bins. In both validation and test curves, the first bin is clearly the most difficult, while errors drop sharply and remain much lower in interior bins. This is consistent with boundary/extrapolation effects and with the smoother behavior observed around central bins.

This baseline therefore provides a strong implied-volatility reference, but its derivative behaviour is not optimized explicitly. The Sobolev subsection uses the same evaluation logic to assess whether Sobolev training preserves the implied-volatility fit while improving the derived Greeks.

5.2.2. Sobolev Methods: IV Accuracy And Derivative-Aware Training

IV And Price-Level Comparison

At IV level, Sobolev training does not improve the held-out scalar regression error: test RMSE increases from $1.86 \cdot 10^{-3}$ for the ANN baseline to $2.55 \cdot 10^{-3}$ for the Sobolev variant. However, on the fixed Greek-diagnostic grid, price-level errors remain of the same order and even decrease slightly, from $2.05 \cdot 10^{-5}$ to $1.95 \cdot 10^{-5}$. Therefore, the Sobolev variant should not be read as a uniformly better IV regressor, but as a derivative-aware surrogate whose main value appears in the sensitivity diagnostics.

Derived Greek Diagnostics

The main effect of Sobolev training appears in the Greeks obtained by composing the ANN-implied volatility with the Black–Scholes pricing map. The compact RMSE summary below compares both ANN surrogates against the Heston characteristic-function reference on the fixed (m, τ) grid. The Sobolev variant reduces the error for all reported Greeks, with especially large gains in Rho, Gamma, and Delta. This confirms that the derivative information introduced during training changes the local geometry of the learned IV surface, even though the global IV RMSE does not improve.

TABLE 5.3. ANN GREEK RMSE.

Greek	ANN baseline	ANN Sobolev	RMSE ratio
Delta	$1.60 \cdot 10^{-4}$	$2.29 \cdot 10^{-5}$	7.0×
Gamma	$1.91 \cdot 10^{-3}$	$1.50 \cdot 10^{-4}$	12.7×
Vega	$3.29 \cdot 10^{-4}$	$1.47 \cdot 10^{-4}$	2.2×
Theta	$1.02 \cdot 10^{-4}$	$2.05 \cdot 10^{-5}$	5.0×
Rho	$7.92 \cdot 10^{-4}$	$3.81 \cdot 10^{-5}$	20.8×

Source: Own elaboration

Before moving to the spatial maps, Figure 5.2 provides a distributional check of the same Greek errors. The panels use signed errors on a log-density scale, so they show

both the concentration around zero and the presence of tails. For Delta, Gamma, Theta, and Rho, the Sobolev ANN concentrates most of the error mass in a much narrower neighborhood of zero than the baseline ANN; Vega also shifts toward the center, although with a less pronounced separation. This supports the interpretation that Sobolev training improves the derivative surface across a broad part of the grid, while the following maps indicate where that improvement is located.

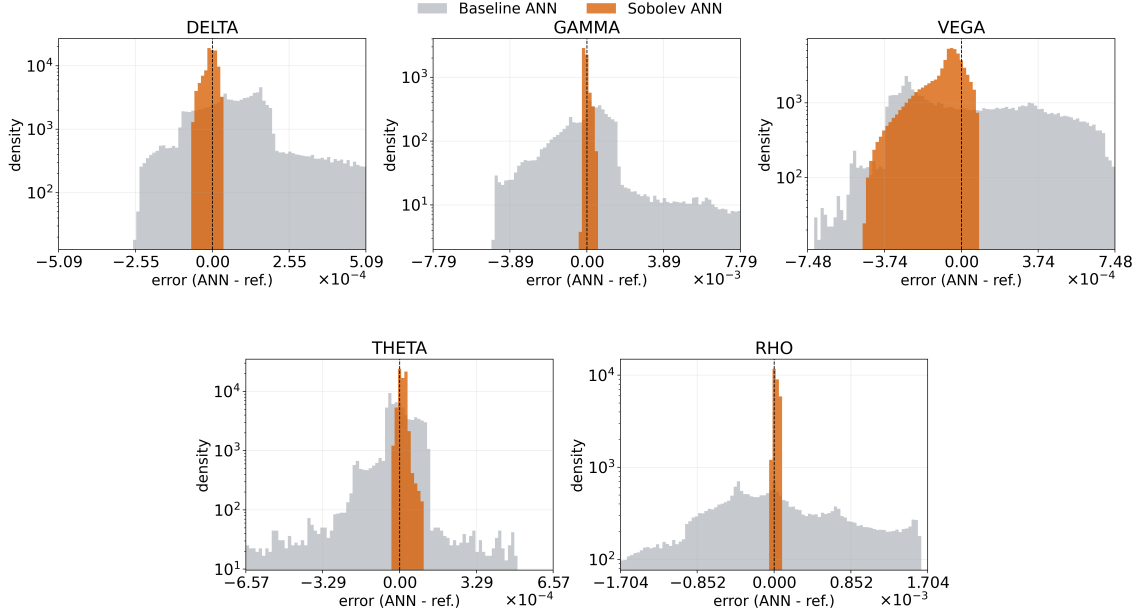


Fig. 5.2. Baseline–Sobolev ANN Greek error histograms. Overlaid density histograms of the pointwise Greek errors summarized in Table 5.3, comparing the ANN baseline with the ANN Sobolev surrogate on the fixed (m, τ) benchmark grid. The five panels correspond to Delta, Gamma, Vega, Theta, and Rho; the horizontal axis reports the error with sign convention ANN minus reference, and the vertical axis reports density on a logarithmic scale.

Figure 5.3 gives the spatial counterpart to the aggregate Greek metrics. Although Sobolev training improves the global Greek errors reported in Table 5.3, the maps show that this improvement is not uniform over the (m, τ) domain. In particular, the short-maturity region remains difficult: near small τ , where the payoff kink makes derivative approximation more sensitive, the relative error is not consistently reduced by the Sobolev surrogate.

The remaining error is also spatially structured rather than purely scattered. Several panels keep coherent high-error regions or bands after Sobolev training, which indicates that the derivative surface still contains systematic approximation errors in localized parts of the grid. Therefore, the Sobolev ANN should be read as a more stable differentiable surrogate in aggregate, but not as a correction that removes the kink-related short-maturity difficulty.

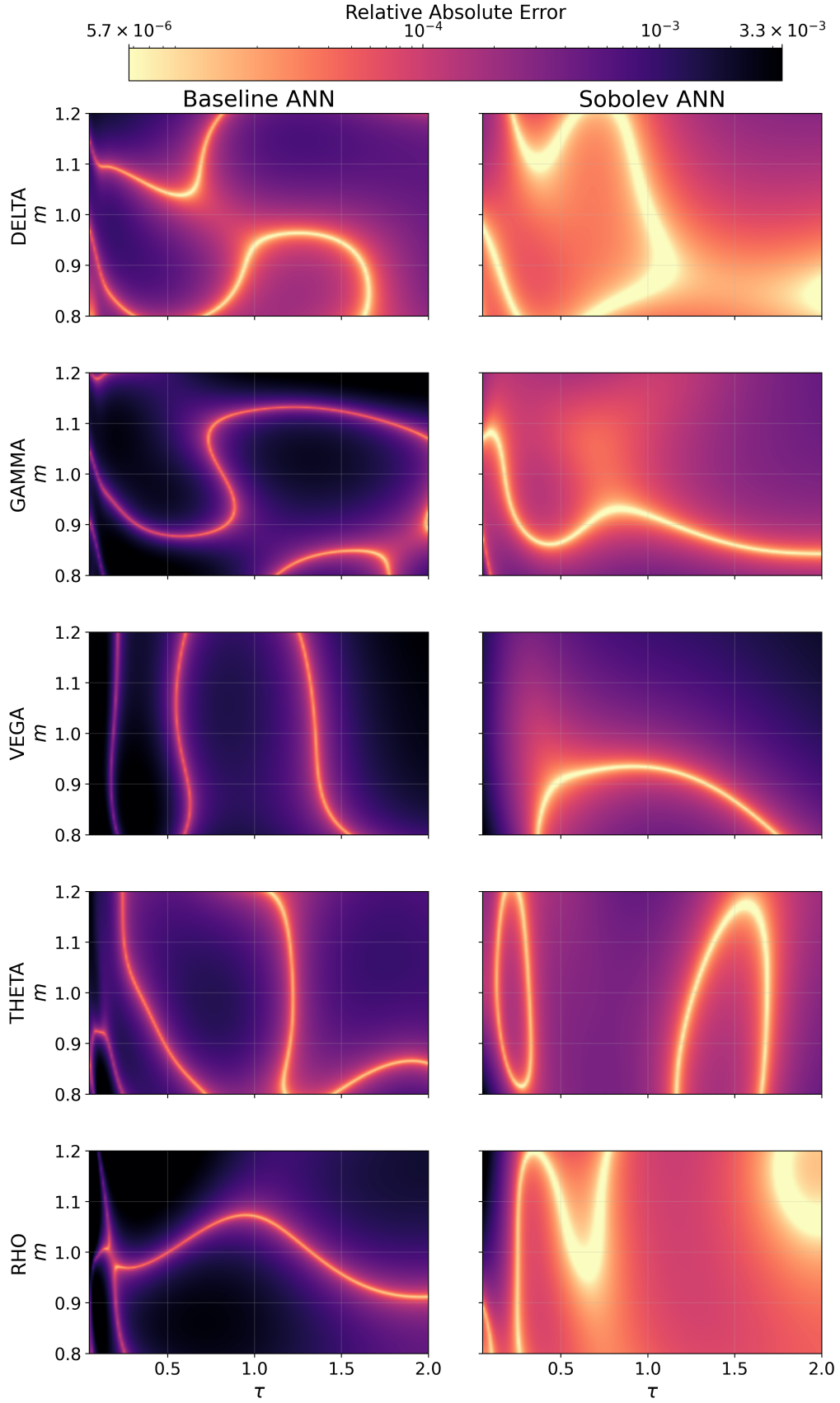


Fig. 5.3. Relative absolute-error maps for ANN-derived Greeks. The two columns compare the ANN baseline and the ANN Sobolev surrogate on the fixed (m, τ) benchmark grid used for the Greek errors summarized in Table 5.3. Rows correspond to Delta, Gamma, Vega, Theta, and Rho; the horizontal axis is maturity τ , the vertical axis is moneyiness m , and the shared colorbar reports relative absolute error on a logarithmic scale.

5.2.3. CaNN: Calibration With The ANN Surrogate

This section reports the benchmark outcomes of the CaNN workflow after the ANN baseline and the ANN Sobolev surrogate have been fixed. The compact calibration setup and diagnostic reading are introduced in Section 3.5; here we focus on the final calibration metrics and on how the inverse problem changes when the frozen forward evaluator is replaced by the Sobolev-trained ANN.

Benchmark Snapshot

The calibration benchmark follows the inverse-problem construction of Section 3.5.1. In both cases, CaNN uses a frozen supervised ANN surrogate as fast implied-volatility evaluator and calibrates the vector $(\rho, \kappa, \gamma, \bar{v}, v_0)$ from one fixed quote universe of 35 contracts (Table 5.4). The comparison is centered on the forward surrogate used inside the inverse loop: the ANN baseline or the ANN Sobolev model. Both calibrations use the same quote grid, the same synthetic truth, the same admissible Heston parameter box, and the same Differential Evolution plus local-polish strategy, with the final optimizer controls taken from each stored run configuration.

TABLE 5.4. QUOTE UNIVERSE USED IN THE CANN BENCHMARK.

Item	Value
Number of quotes	35
Grid structure	Complete 5×7 grid over (m, τ)
Moneyness nodes m	$\{0.85, 0.925, 1.00, 1.075, 1.15\}$
Maturity nodes τ	$\{0.50, 0.75, 1.00, 1.25, 1.50, 1.75, 2.00\}$
Interest rate r	0.03 (constant across quotes)

Source: Own elaboration

Fit Quality And Convergence

Table 5.5 reports quote-level calibration residuals in implied-volatility units for the two CaNN variants. The residual is defined as $e = \hat{\sigma}_{IV} - \sigma_{IV}^{\text{mkt}}$ on the fixed 35-quote Liu grid. The table does not include Liu et al. because the reference paper reports the weighted objective and parameter errors used in Table 5.6, but not the same quote-level RMSE and MAPE decomposition.

TABLE 5.5. CANN CALIBRATION FIT.

Forward surrogate	n	MSE	RMSE	MAPE (%)	max e
ANN baseline	35	$7.43 \cdot 10^{-10}$	$2.73 \cdot 10^{-5}$	0.0048	$6.79 \cdot 10^{-5}$
ANN Sobolev	35	$3.69 \cdot 10^{-11}$	$6.08 \cdot 10^{-6}$	0.0011	$1.36 \cdot 10^{-5}$

Source: Own elaboration from stored quote-level calibration outputs

The Sobolev surrogate gives the tighter quote fit: the residual RMSE falls from $2.73 \cdot 10^{-5}$ to $6.08 \cdot 10^{-6}$, and the maximum absolute residual is reduced by roughly a factor of five. The MAPE values are very small in both cases because the replicated Liu grid is smooth and remains inside the interpolation region, but the same ordering is preserved in relative terms. This suggests that the Sobolev information does not only improve derivative diagnostics later in the ANN branch; in this controlled calibration problem it also makes the inverse fit more precise in IV space.

Parameter Recovery Profile

Since this quote set includes sidcar synthetic truth, parameter-level errors can be reported directly. Table 5.6 compares both CaNN variants with the Liu et al. reference values.

TABLE 5.6. CANN CALIBRATION RESULTS USING ANN BASELINE AND ANN SOBOLEV SURROGATES VERSUS LIU ET AL. ON THE REPLICATED 35-QUOTE BENCHMARK, ALL QUOTE WEIGHTS ARE EQUAL TO ONE, SO THE REPORTED WEIGHTED MSE COINCIDES WITH THE MEAN SQUARED IV RESIDUAL.

Metric	CaNN ANN baseline	CaNN ANN Sobolev	Liu et al. [6]
Weighted MSE	$7.43 \cdot 10^{-10}$	$3.69 \cdot 10^{-11}$	$\sim 10^{-8}$
$ \rho - \rho^* $	$8.05 \cdot 10^{-3}$	$1.79 \cdot 10^{-3}$	$4.84 \cdot 10^{-2}$
$ \kappa - \kappa^* $	$8.48 \cdot 10^{-4}$	$8.36 \cdot 10^{-3}$	$4.88 \cdot 10^{-2}$
$ \gamma - \gamma^* $	$6.57 \cdot 10^{-3}$	$2.40 \cdot 10^{-3}$	$3.28 \cdot 10^{-2}$
$ \bar{v} - \bar{v}^* $	$1.02 \cdot 10^{-4}$	$3.08 \cdot 10^{-5}$	$4.54 \cdot 10^{-3}$
$ v_0 - v_0^* $	$1.83 \cdot 10^{-4}$	$8.03 \cdot 10^{-5}$	$4.39 \cdot 10^{-4}$

Source: Own elaboration based on selected run outputs and [6].

In Table 5.6, the weighted MSE is directly comparable with the Liu et al. reference because both evaluations use uniform quote weights on the same 35-quote grid. Both CaNN variants improve the Liu et al. reference values on the replicated benchmark. The Sobolev surrogate gives the lowest quote residual and improves four of the five parameter errors relative to the ANN baseline, especially for ρ , γ , $\bar{v}$, and v_0 . The exception is κ , where the baseline ANN remains closer to the synthetic truth. Therefore, the Sobolev calibration

is retained as the main CaNN result, while the baseline calibration remains useful as a reference showing that the improvement is not only due to the inverse optimizer but also to the forward surrogate used inside it.

Local Smoothness And Curvature Study

To characterize local identifiability around the calibrated optimum, Jacobian and Hessian diagnostics were computed at θ^* for both CaNN variants. In Table 5.7, $\text{cond}_{\text{abs}}(H)$ denotes the absolute Hessian condition number, computed as the ratio between the largest and smallest absolute eigenvalues. A large value indicates local ill-conditioning: small changes along some parameter directions are weakly penalized compared with others, so the inverse problem is less stable in those directions.

TABLE 5.7. CANN LOCAL CURVATURE.

Variant	$J(\theta^*)$	$\ \nabla J(\theta^*)\ _2$	$\lambda_{\min}(H)$	$\lambda_{\max}(H)$	$\text{cond}_{\text{abs}}(H)$
ANN baseline	$2.60 \cdot 10^{-8}$	$1.24 \cdot 10^{-5}$	$4.67 \cdot 10^{-5}$	41.8	$8.93 \cdot 10^5$
ANN Sobolev	$1.31 \cdot 10^{-9}$	$2.28 \cdot 10^{-6}$	$4.56 \cdot 10^{-5}$	42.0	$9.20 \cdot 10^5$

Source: Own elaboration

Table 5.7 separates two effects. The Sobolev surrogate gives a much smaller objective value and a smaller gradient norm at the calibrated point, so the retained solution is closer to first-order stationarity. However, the extreme Hessian eigenvalues and $\text{cond}_{\text{abs}}(H)$ remain almost unchanged. Therefore, this curvature diagnostic should not be interpreted as the main source of the Sobolev improvement; it mainly reflects the local geometry of the same 35-quote inverse problem.

To make the matrix-level reading explicit, Table 5.8 reports the Hessian $H = \nabla^2 J(\theta^*)$ for the retained Sobolev CaNN run.

TABLE 5.8. CANN SOBOLEV HESSIAN.

	ρ	κ	γ	$\bar{v}$	v_0
ρ	$2.55 \cdot 10^{-2}$	$2.91 \cdot 10^{-3}$	$-6.33 \cdot 10^{-2}$	0.429	0.525
κ	$2.91 \cdot 10^{-3}$	$6.45 \cdot 10^{-4}$	$-5.68 \cdot 10^{-3}$	$3.55 \cdot 10^{-2}$	$5.89 \cdot 10^{-3}$
γ	$-6.33 \cdot 10^{-2}$	$-5.68 \cdot 10^{-3}$	0.198	-1.63	-2.08
$\bar{v}$	0.429	$3.55 \cdot 10^{-2}$	-1.63	15.2	18.6
v_0	0.525	$5.89 \cdot 10^{-3}$	-2.08	18.6	28.8

Source: Own elaboration

Table 5.8 shows a block structure centered on $(\bar{v}, v_0)$. Diagonal terms measure local curvature with respect to each individual parameter, while off-diagonal terms measure local cross-coupling between pairs of parameters. In the Sobolev Hessian, the diagonal

entries associated with $\bar{v}$ and v_0 are the largest, and their off-diagonal term is also large, which indicates that the objective reacts strongly to movements in this variance block and that both coordinates are locally coupled. By contrast, the κ -related diagonal curvature is much smaller, so perturbations in that direction are less strongly penalized around θ^* . This mixed pattern explains why the Sobolev run can deliver a very tight IV fit while κ remains the one parameter where the baseline calibration is closer to the synthetic truth.

These results define the retained CaNN benchmark for the supervised branch: the ANN Sobolev surrogate is used as the final calibration result, and the ANN baseline calibration is kept as the direct Liu-style reference.

5.3. Unsupervised PINN Benchmarks

This section reports the pricing benchmark for the baseline PINN defined in Chapter 4. Using the Heston–COS engine of Section 2.7 as numerical reference and the evaluation protocol summarized in Appendix B, we measure price-level accuracy and runtime on the fixed benchmark grid. These results establish the baseline reference for the physics-informed branch before analyzing the Sobolev and Sobolev+ACV diagnostics later in the section.

5.3.1. PINN Baseline

The detailed reproducible setup (input convention, standardization, reference parameters, filtering, and arithmetic context) is moved to Appendix B. In the final pricing diagnostic used here, the baseline parametric PINN without Sobolev terms is evaluated against the Heston reference on the same fixed grid later used for the Sobolev comparison: 25 921 valid points over $m \in [0.8, 1.2]$ and $\tau \in [0.05, 2.0]$.

Global Pricing Metrics

The compact global summary is embedded in Figure 5.4: $\text{MSE} = 9.43 \cdot 10^{-8}$, $\text{RMSE} = 3.07 \cdot 10^{-4}$, and $\text{MAPE} = 0.28\%$. These values provide the price-level reference for the baseline PINN on the final fixed grid. In absolute price units, the error is small, while the MAPE remains below one percent, which indicates that the model captures the benchmark Heston surface accurately enough to serve as the reference point for the subsequent derivative diagnostics.

Spatial And Distributional Diagnostics

The pointwise agreement between PINN and reference prices, the global metric table, the spatial error map, and the distributional diagnostic are consolidated in Figure 5.4.

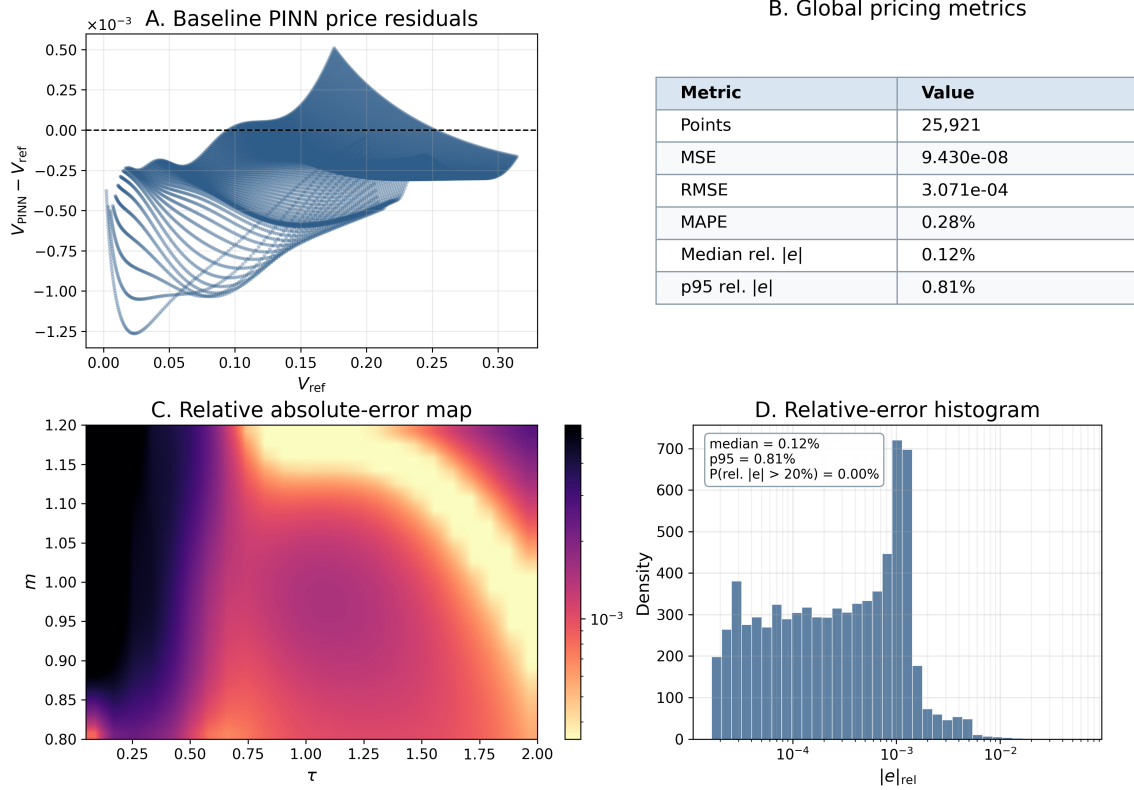


Fig. 5.4. Baseline PINN pricing benchmark versus reference. Top-left: price residual scatter, with V_{ref} on the horizontal axis and $V_{\text{PINN}} - V_{\text{ref}}$ on the vertical axis. Top-right: global MSE/RMSE/MAPE and relative-error summary. Bottom-left: mean relative absolute-error map over (τ, m) , with the color scale representing the cellwise mean relative absolute error. Bottom-right: histogram of relative absolute pricing errors.

The residual scatter shows that the baseline PINN errors remain small in absolute price units, although they are not perfectly centered around zero across the reference-price range. On this common local grid, the largest relative deviations appear near short maturities and in low-reference-price regions, while the central pricing region remains more stable. These values must be interpreted jointly with the absolute-unit metrics (MSE and RMSE): when the reference price is small, relative error can magnify deviations that remain modest in price units. Distributionally, the median relative absolute error is 0.12%, the p_{95} relative absolute error is 0.81%, and no point exceeds 20% relative error.

5.3.2. Sobolev Methods: Greek Improvement And Pricing Trade-Off

The baseline PINN established the reference pricing result for the physics-informed branch. The Sobolev variant is evaluated next as the derivative-aware modification defined in Section 4.5, motivated by the Sobolev experiment already studied in the supervised ANN branch. In the PINN setting, however, this modification is not interpreted as an automatic improvement: the central question is whether the extra derivative information improves the Greek diagnostics while preserving, or possibly sacrificing, the pricing quality that de-

defines the role of the PINN. For this reason, we first report the Sobolev pricing benchmark on the same grid and then analyze the Greek ablation against the baseline.

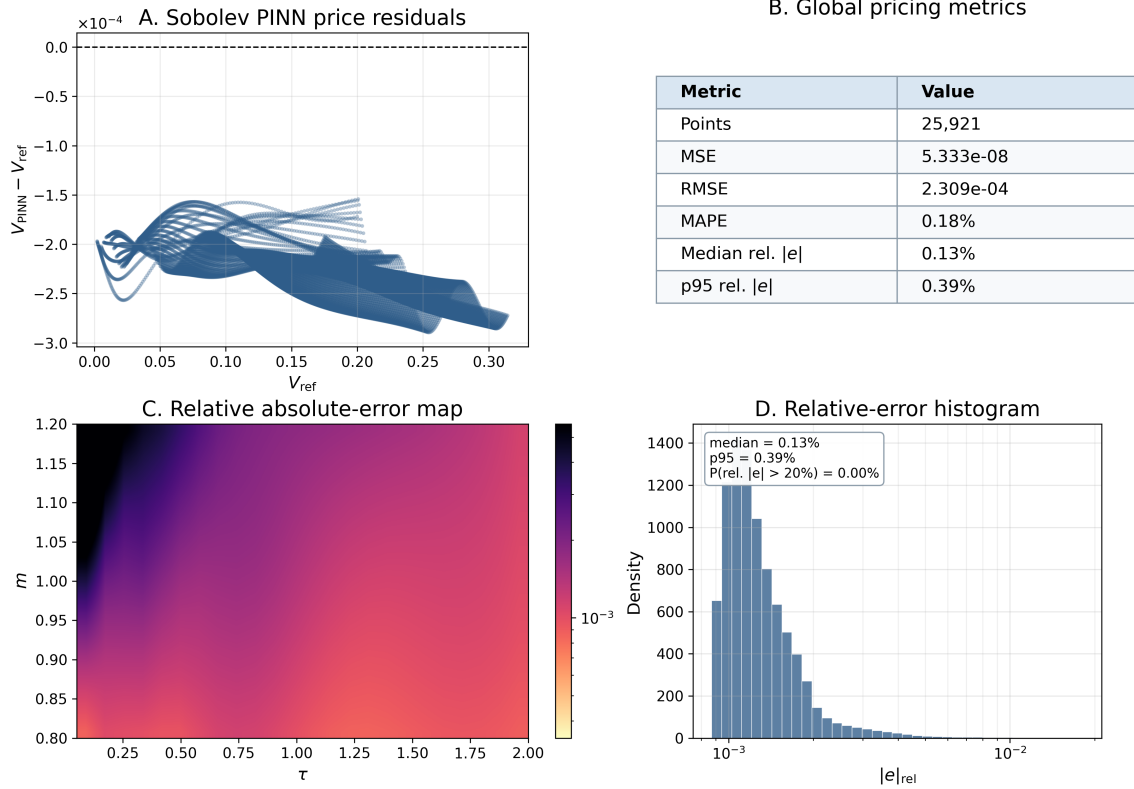


Fig. 5.5. Sobolev PINN pricing benchmark versus reference. Top-left: price residual scatter, with V_{ref} on the horizontal axis and $V_{\text{PINN}} - V_{\text{ref}}$ on the vertical axis. Top-right: global MSE/RMSE/MAPE and relative-error summary. Bottom-left: mean relative absolute-error map over (τ, m) , with the color scale representing the cellwise mean relative absolute error. Bottom-right: histogram of relative absolute pricing errors.

In the Sobolev pricing benchmark, the residual scatter is more compressed than in the baseline case, but it remains mostly below zero, indicating a small systematic underpricing on this grid.

Ablation Metrics By Greek

The Greek comparison is restricted to the baseline PINN and the Sobolev PINN, evaluated on the same fixed (m, τ) grid and under the same Heston parameter configuration used in the pricing benchmark. Reference Greeks are computed with the Heston characteristic-function formulas introduced in Section 2.7.2. This keeps the ablation focused on the effect of the Sobolev terms themselves: the ACV correction is not included here because it is a separate local hard-region diagnostic, discussed later in the section.

The compact RMSE summary in Table 5.9 reports the five Greeks on $n = 2.5921 \cdot 10^4$ grid points per Greek.

TABLE 5.9. PINN GREEK ERRORS.

Greek	PINN baseline	PINN Sobolev	RMSE ratio
Delta	$3.33 \cdot 10^{-2}$	$3.51 \cdot 10^{-4}$	94.9×
Gamma	$3.85 \cdot 10^{-1}$	$8.94 \cdot 10^{-3}$	43.1×
Vega	$6.55 \cdot 10^{-2}$	$3.17 \cdot 10^{-3}$	20.7×
Theta	$4.82 \cdot 10^{-3}$	$5.30 \cdot 10^{-4}$	9.1×
Rho	$8.37 \cdot 10^{-2}$	$1.43 \cdot 10^{-3}$	58.5×

Source: Own elaboration

Compared with the baseline PINN, Sobolev training reduces RMSE for all five Greeks. The largest relative gains appear in Delta and Rho, followed by Gamma, while Theta shows the smallest but still clear reduction. The absolute scale is also important: Gamma remains the most demanding sensitivity because it starts from the largest baseline error and, even after the Sobolev correction, has the largest RMSE among the five Greeks. This improvement should be read together with the change in training information: the Sobolev run uses external derivative targets, and Greek-inference throughput decreases from $9.48 \cdot 10^3$ to $8.60 \cdot 10^3$ points/s.

The distributional view in Figure 5.6 complements the scalar metrics by showing how the error mass changes across the full grid. The overlaid log-density histograms show that Sobolev training does not only improve the aggregate RMSE values, but also reduces the frequency of large deviations. This distributional reading is especially relevant for Gamma, Rho, and Vega, which are the weakest baseline sensitivities in RMSE terms in Table 5.9.

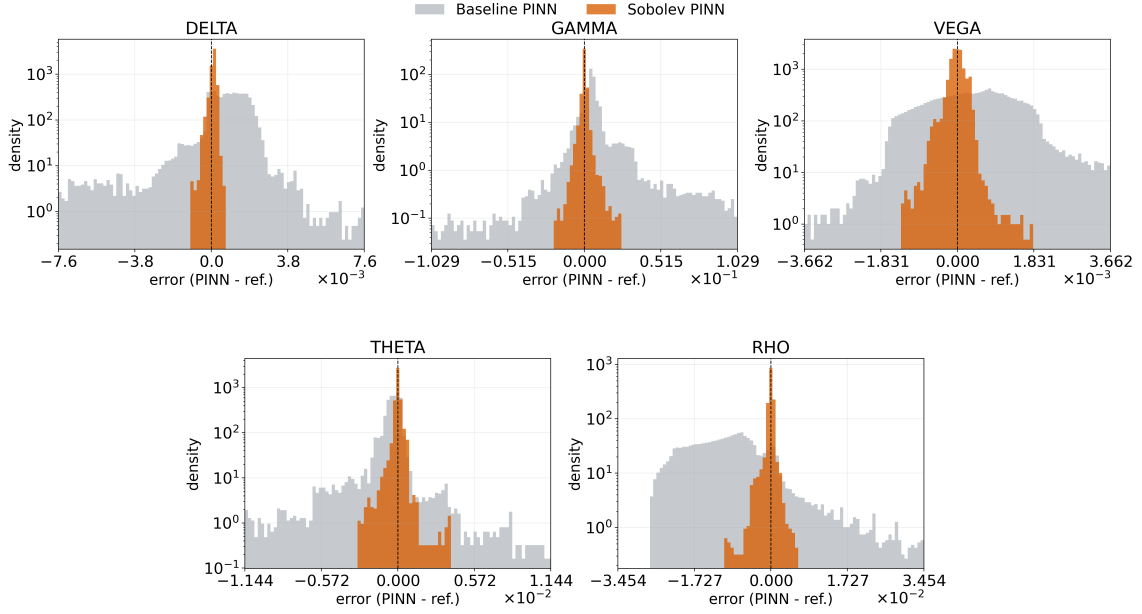


Fig. 5.6. Baseline–Sobolev PINN Greek error histograms. Overlaid density histograms of the pointwise Greek errors summarized in Table 5.9, comparing the baseline PINN with the Sobolev PINN on the fixed (m, τ) benchmark grid. The five panels correspond to Delta, Gamma, Vega, Theta, and Rho; the horizontal axis reports the error with sign convention PINN minus reference, and the vertical axis reports density on a logarithmic scale.

Visual Diagnostics

The visual diagnostics in this subsection refer only to the baseline–Sobolev ablation. The ACV correction is treated separately below because it targets a localized hard region rather than the full (m, τ) grid.

Figure 5.7 gives the spatial counterpart to the global Greek metrics in Table 5.9. As in the ANN case, Sobolev training improves the aggregate errors but does not uniformly reduce the error over the whole domain. The short-maturity region remains the most delicate part of the grid: near small τ , where the payoff kink dominates the derivative behavior, the Sobolev PINN still exhibits visible high-error regions.

This persistence is important for the later ACV analysis. The Sobolev term reduces part of the global Greek error, but the residual maps still contain systematic spatial structure rather than unstructured noise. This motivates treating the remaining short-maturity and near-kink errors as a localized diagnostic problem, instead of interpreting Sobolev augmentation as a complete solution for the Greek surface.

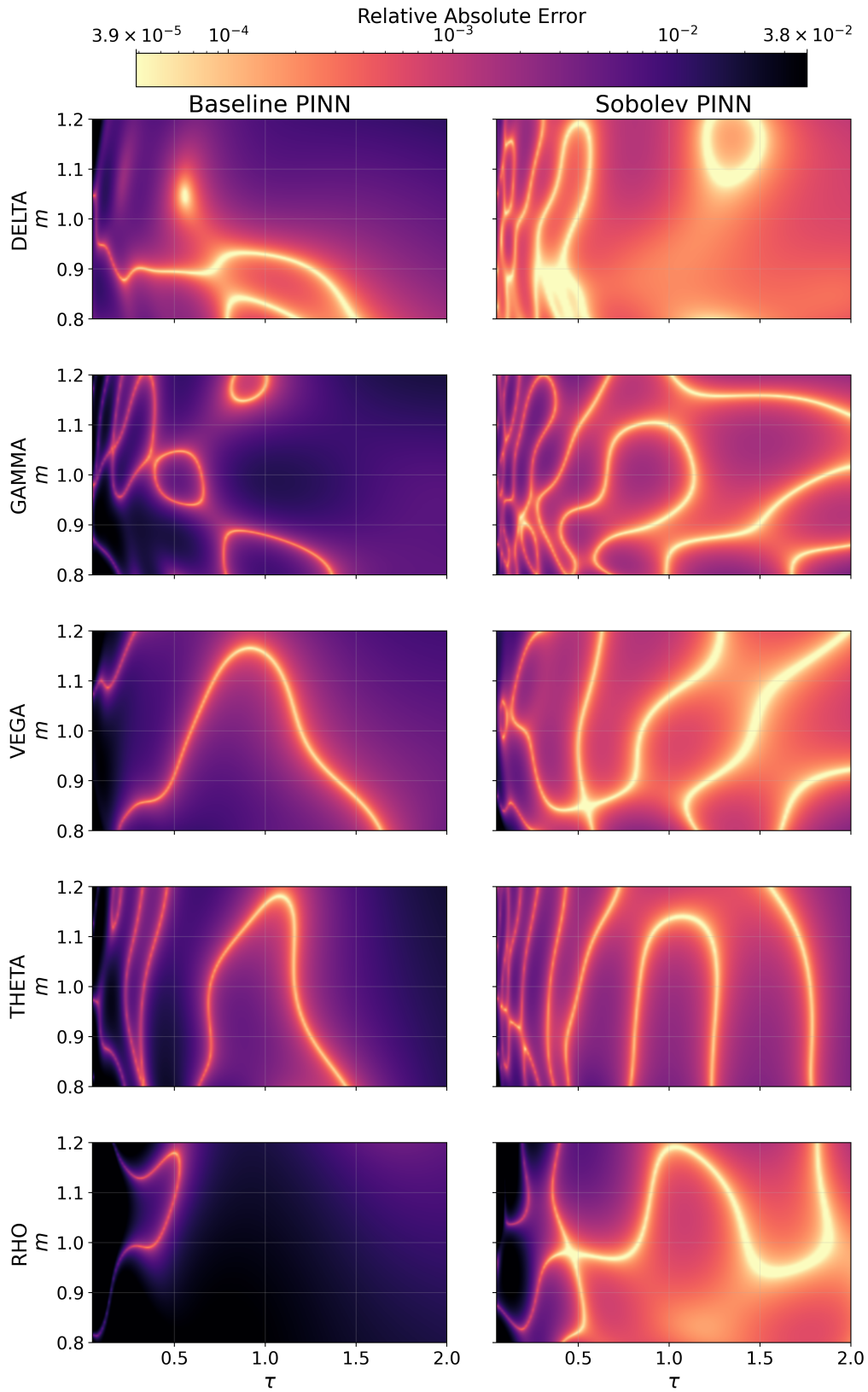


Fig. 5.7. Baseline–Sobolev PINN Greek relative-error maps. The two columns compare the baseline PINN and the Sobolev PINN on the fixed (m, τ) benchmark grid used for the Greek errors summarized in Table 5.9. Rows correspond to Delta, Gamma, Vega, Theta, and Rho; the horizontal axis is maturity τ , the vertical axis is moneyness m , and the shared colorbar reports relative absolute error on a logarithmic scale.

The two-phase optimization schedule used in this ablation trains epochs 0 to 4000 without Sobolev terms, then activates Sobolev augmentation from epoch 4000 onward. Together, the diagnostics indicate that Sobolev augmentation improves local derivative fidelity, not just aggregate scalar scores, while keeping the runtime overhead limited.

5.3.3. PINN Sobolev+ACV: Local Hard-Region Diagnostic

The previous subsection showed that Sobolev training improves the PINN Greek errors on the full benchmark grid, but it also left visible residual structure near short maturities and around the payoff kink. The Sobolev+ACV variant is evaluated here as a local correction to that remaining difficulty. As defined in Section 4.6, the global Sobolev PINN is kept frozen and a Black–Scholes analytical control variate is blended only in the short-maturity ATM region defined in Equation (4.21). The residual network inside the ACV decomposition is initialized at zero and is not trained, so the diagnostic isolates the effect of the analytical control variate and of the smooth local gate.

Figure 5.8 reports the corresponding local error maps. Unlike the full-grid Sobolev ablation above, this experiment is deliberately restricted to the hard region where the payoff kink produces the largest derivative outliers. The three columns are evaluated under the same Heston parameter configuration, reference engine, and local grid; the only difference is the model used to generate the price surface and its autodiff Greeks: baseline PINN, Sobolev PINN, or Sobolev+ACV.

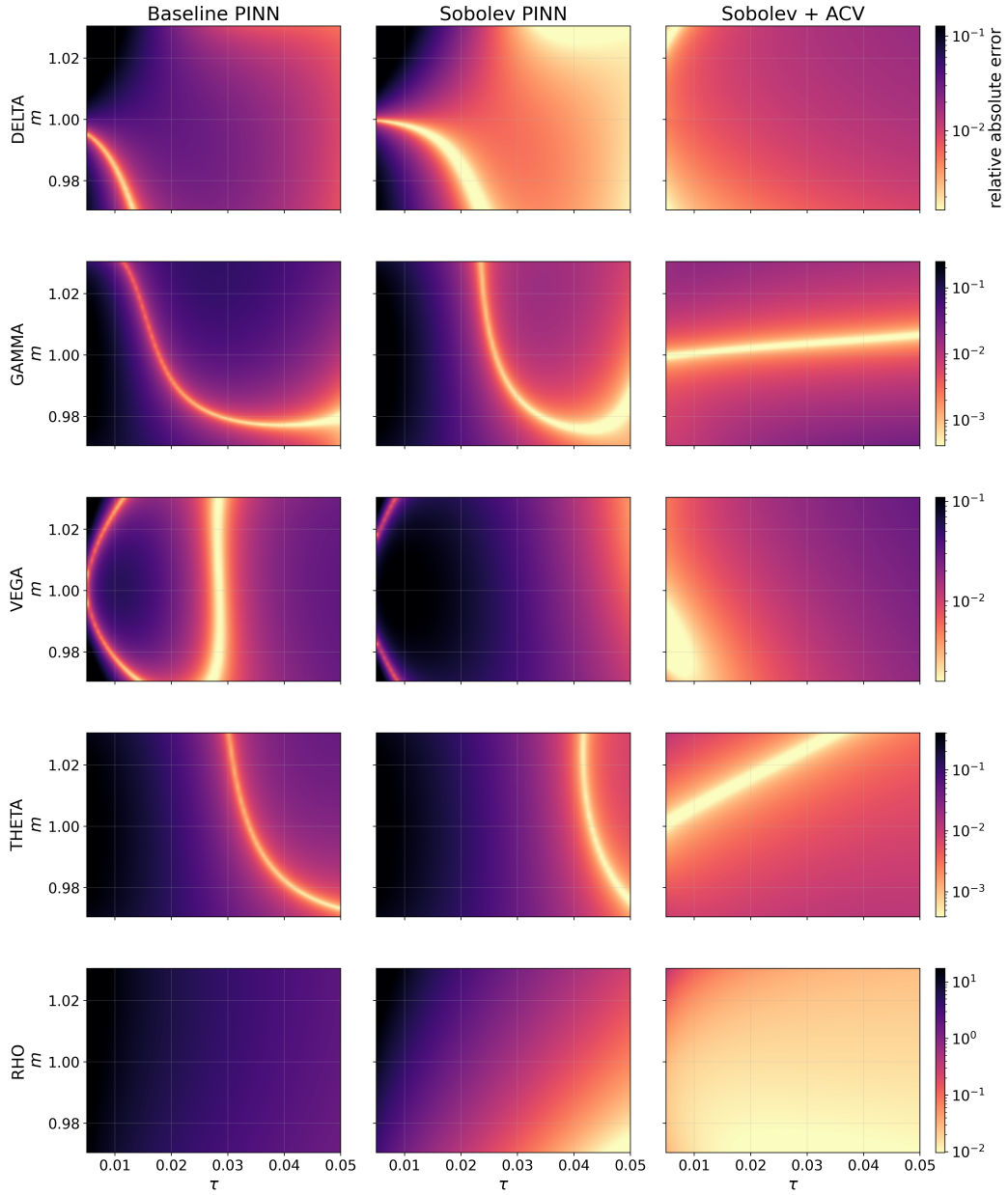


Fig. 5.8. Kink-region PINN Greek relative-error maps. The three columns compare the baseline PINN, the Sobolev PINN, and the final Sobolev+ACV correction on the local short-maturity ATM grid $m \in [0.9704, 1.0305]$, $\tau \in [0.005, 0.05]$. Rows correspond to Delta, Gamma, Vega, Theta, and Rho; the horizontal axis is maturity τ , the vertical axis is moneyness m , and each row uses an independent logarithmic colorbar for the relative absolute error of that Greek.

The row-wise color scales in Figure 5.8 make the local effect of the ACV correction easier to read than a single global colorbar. The baseline and Sobolev columns still contain concentrated high-error regions around the short-maturity ATM layer, especially for Gamma and Theta. After adding the ACV correction, those regions contract sharply, which indicates that the analytical component is acting precisely on the part of the grid for which it was introduced.

Model	Price	Delta	Gamma	Vega	Theta	Rho
Baseline	$1.15 \cdot 10^{-3}$	$1.84 \cdot 10^{-2}$	$7.32 \cdot 10^{-1}$	$1.65 \cdot 10^{-3}$	$1.53 \cdot 10^{-1}$	$5.67 \cdot 10^{-2}$
Sobolev	$1.09 \cdot 10^{-3}$	$1.60 \cdot 10^{-2}$	$8.63 \cdot 10^{-1}$	$3.11 \cdot 10^{-3}$	$1.56 \cdot 10^{-1}$	$1.36 \cdot 10^{-2}$
Sobolev+ACV	$1.10 \cdot 10^{-4}$	$4.99 \cdot 10^{-3}$	$5.29 \cdot 10^{-2}$	$1.20 \cdot 10^{-3}$	$3.03 \cdot 10^{-3}$	$2.60 \cdot 10^{-4}$

Source: Own elaboration.

TABLE 5.10. HARD-REGION RMSE FOR THE PINN VARIANTS.

Table 5.10 gives the scalar counterpart to the local maps. Under this common hard-region protocol, Sobolev+ACV reduces RMSE by roughly one order of magnitude relative to the Sobolev PINN in Price, Gamma, Theta, and Rho, while also improving Delta and Vega. This does not make ACV the preferred global PINN model: on the full grid, the Sobolev PINN remains the cleaner global derivative surface. The role of Sobolev+ACV is narrower and more specific: it corrects the localized short-maturity ATM layer where the payoff kink dominates the Greek error.

5.4. Computational Cost And Model Roles

This final benchmark section summarizes the empirical roles of the models compared in the chapter through the information available at Table 5.11. The comparison is not between two models trained under the same information regime. The ANN branch is supervised and learns implied-volatility labels generated by the Heston-COS/Brent reference pipeline, while the baseline PINN is physics-informed and is trained mainly from the Heston PDE together with terminal and boundary conditions, without price labels. The Sobolev and ACV variants then add derivative-aware or local analytical structure on top of those two branches.

Model	Price	Delta	Gamma	Vega	Theta	Rho
ANN baseline	$2.05 \cdot 10^{-5}$	$1.60 \cdot 10^{-4}$	$1.91 \cdot 10^{-3}$	$3.29 \cdot 10^{-4}$	$1.02 \cdot 10^{-4}$	$7.92 \cdot 10^{-4}$
ANN Sobolev	$1.95 \cdot 10^{-5}$	$2.29 \cdot 10^{-5}$	$1.50 \cdot 10^{-4}$	$1.47 \cdot 10^{-4}$	$2.05 \cdot 10^{-5}$	$3.81 \cdot 10^{-5}$
PINN baseline	$3.07 \cdot 10^{-4}$	$1.58 \cdot 10^{-3}$	$1.65 \cdot 10^{-2}$	$1.15 \cdot 10^{-3}$	$1.65 \cdot 10^{-3}$	$1.34 \cdot 10^{-2}$
PINN Sobolev	$2.31 \cdot 10^{-4}$	$1.72 \cdot 10^{-4}$	$1.90 \cdot 10^{-3}$	$1.91 \cdot 10^{-4}$	$2.73 \cdot 10^{-4}$	$8.58 \cdot 10^{-4}$
PINN Sobolev+ACV	$2.31 \cdot 10^{-4}$	$9.93 \cdot 10^{-4}$	$4.79 \cdot 10^{-2}$	$3.96 \cdot 10^{-4}$	$7.26 \cdot 10^{-4}$	$7.86 \cdot 10^{-4}$

Source: Own elaboration.

TABLE 5.11. RMSE SUMMARY FOR THE FINAL MODELS.

In the supervised branch, the ANN Sobolev model gives the most stable derivative-aware surrogate among the ANN variants, while the CaNN workflow uses the frozen ANN evaluator inside the inverse calibration loop described in Section 5.2.3. The relevant cost distinction is between a fast forward pass and the repeated optimizer evaluations required by calibration. The runtime indicator retained for the ANN evaluator is approximately

$8.43 \cdot 10^5$ implied-volatility evaluations per second, but the calibration cost also depends on the Differential Evolution and local-polish settings used in the inverse problem.

In the physics-informed branch, the baseline PINN provides the reference price-surface benchmark, and the Sobolev PINN improves the full-grid Greek diagnostics analyzed in Section 5.3.2. The throughput reduction reported there, from approximately $9.48 \cdot 10^3$ to $8.60 \cdot 10^3$ Greek points/s, is moderate relative to the improvement in derivative accuracy. The Sobolev+ACV variant has a narrower role: it is not retained as a better global PINN, but as a local correction for the short-maturity ATM region analyzed in Table 5.10. For price evaluation, the pricing benchmark of Section 5.3 gives approximately $4.91 \cdot 10^5$ PINN points/s versus 87.36 COS points/s in repeated evaluation mode.

6. FINAL DISCUSSION

6.1. Main Conclusions

Neural networks are successful in capturing, with a relatively small number of neurons, the behaviour of complex models with a large number of degrees of freedom, such as the Heston model. In this work, we developed and evaluated neural-network-based surrogates for calibration, pricing, and Greek computation of put derivatives. Achieving high accuracy and computational efficiency required several technical and benchmarking choices.

The key idea behind the success of our approach is the use of a dedicated neural network architecture for each task. By reusing a common framework while adapting each network to its target problem, we obtain robust and efficient approximations. This is also the main interpretation of the results in Chapter 5: the ANN, CaNN, PINN, Sobolev, and ACV components should not be read as a single sequence of interchangeable models, but as task-specific tools evaluated under the same numerical benchmark.

Computing an implied volatility surface typically requires the iterative evaluation of an option pricer within an optimization algorithm. The supervised ANN surrogate introduced in Chapter 3 condenses this process into a single evaluation of the network, producing implied volatilities computed with errors in the 10^{-3} IV range on held-out data while maintaining very fast inference times, as summarized in Table 5.2. Figure 5.1 shows that this surrogate is most reliable inside the interpolation region. This makes the ANN branch the natural starting point for the calibration part of the thesis, because the inverse problem needs many repeated evaluations of the forward implied-volatility map.

The calibration of the Heston model is similarly challenging, as state-of-the-art approaches generally rely on carefully designed optimization loops and derivative information [29]. By leveraging the CaNN methodology introduced by Liu et al. [6], we use the previously trained ANN surrogate to recover the Heston model parameters from a given implied volatility surface. On the replicated 35-quote Liu grid, the CaNN calibration reduces the IV MSE to the 10^{-10} range with the baseline ANN and to the 10^{-11} range with the Sobolev ANN, compared with the 10^{-8} reference scale reported by Liu et al. More importantly, all reported Heston parameter errors improve relative to the Liu et al. reference values under the same synthetic-truth comparison reported in Chapter 5. In the final Sobolev CaNN benchmark, the MSE improves by more than two orders of magnitude, while the parameter errors improve from roughly half an order of magnitude to more than two orders of magnitude depending on the parameter; the detailed comparison is reported in Table 5.6.

Taking the proposed model from Liu one step forward, we improved the ANN performance by adding information about the Greeks via Sobolev augmentation. This improves

both the RMSE of pricing and Greek computation. In the ANN branch, the largest effect is observed in the sensitivity diagnostics: Sobolev training reduces the Greek RMSE for all reported quantities, with especially clear gains in Delta, Gamma, Theta, and Rho, as reported in Table 5.3. The distributional and spatial diagnostics in Figures 5.2 and 5.3 show that this improvement is broad, although not uniform over the whole (m, τ) domain. Therefore, the Sobolev ANN should not be interpreted only as a more accurate pointwise regressor, but as a derivative-aware surrogate whose learned surface is more suitable for calibration diagnostics and risk sensitivities.

We also evaluated the performance of PINNs regarding pricing and Greek computation. While ANNs require previous knowledge about a large set of prices, implied volatilities, or Greeks, the baseline PINN relies on the Heston PDE, together with terminal and boundary conditions, to infer the price surface without supervised price labels. For this reason, the PINN is not better than the ANN branch in the global comparison of Table 5.11, but it answers a different and important question: whether a useful pricing surface can be learned from the structure of the differential equation itself. The results show that this is possible, with fast repeated price evaluation and small pricing errors on the benchmark grid, as shown in Figure 5.4. When dealing with Greek computation, however, the baseline PINN is more fragile, especially near short maturities. The combined action of Sobolev augmentation and the analytical control variate improves this behaviour: Sobolev terms reduce the full-grid Greek errors reported in Table 5.9, while the ACV correction targets the short-maturity at-the-money region where the remaining derivative errors are most concentrated, as shown in Figure 5.8 and Table 5.10.

The main contribution of the thesis is therefore the construction of a reproducible benchmark across the main neural techniques considered for Heston-based pricing, calibration, and Greek computation. The comparison covers supervised ANN surrogates, CaNN calibration, Sobolev-augmented ANN models, baseline PINNs, Sobolev PINNs, and the Sobolev+ACV correction. More importantly, the benchmark does not rely only on aggregate errors: it reports IV and pricing metrics, Greek RMSEs, spatial diagnostics, hard-region behaviour, and evaluation throughput. The latter is relevant because the advantage of these models appears after training, when repeated evaluations can replace much slower numerical calls; this is visible in the ANN evaluator and in the PINN pricing benchmark, where the learned models reach orders-of-magnitude faster repeated evaluation than the corresponding reference procedures.

Taken together, these results support a practical division of roles. The ANN/CaNN block is the most mature part of the workflow and gives the strongest calibration and Greek-estimation results in the controlled benchmark. The PINN block is valuable as a differentiable pricing engine trained from the Heston PDE without supervised price labels, but its derivative quality requires additional structure. In our experiments, this structure appears either through Sobolev augmentation, which improves the global Greek diagnostics, or through analytical control variates tailored to the short-maturity at-the-money region.

6.2. Future Work

The most immediate extension would be to exploit put-call parity. Since the thesis has focused on European put options, the corresponding call prices can be obtained consistently from the same learned put surface through Equation (2.2). This would enlarge the set of vanilla instruments covered by the framework without changing the core Heston benchmark, and would also provide a first consistency check before moving to more complex payoffs.

From there, it would be natural to study more explicitly the interaction between networks that learn volatility surfaces and networks that produce prices and Greeks. In the present thesis these components have been benchmarked separately to keep the comparison clean. A more coupled approach could use calibrated implied-volatility surfaces as inputs for pricing and Greek engines, while derivative-aware models could provide diagnostics on the smoothness and stability of the learned volatility representation.

The controlled benchmark should also be extended to market data with arbitrage-aware implied-volatility surfaces. The synthetic Heston benchmark is useful because the true parameters and reference values are known, but real surfaces introduce noise, liquidity effects, bid-ask spreads, and static-arbitrage constraints. Extending the workflow to this setting would show whether the calibration and Greek improvements remain stable under realistic data imperfections.

A particularly important open question is how to improve PINN Greeks without introducing precomputed Greek targets in the loss. Sobolev augmentation improves the sensitivity diagnostics, but it also changes the information regime because derivative labels are added during training. A more self-contained PINN method should preserve the PDE-based character of the baseline model while improving derivative stability through other mechanisms, such as better loss balancing, adaptive sampling near difficult regions, architectures with smoother second derivatives, or analytical corrections that do not require supervised Greek data.

Beyond the vanilla setting, the same methodology could be tested on American, Bermudan, or Asian options. These products introduce early-exercise features or path dependence, so the validation setup would have to change. The neural architecture may still be reusable, but the reference engine and the training data would become more complex. This is precisely where neural surrogates may become more useful, because the cost of repeated numerical pricing grows quickly for these products.

These extensions would require a more complex setup, but they are realistic given recent progress in scientific machine-learning libraries, automatic differentiation, and neural PDE solvers. The benchmark built in this thesis provides a controlled baseline from which those more complex experiments can be added without losing diagnostic discipline.

BIBLIOGRAPHY

- [1] J. C. Hull, *Options, Futures, and Other Derivatives*, 11th ed. Pearson, 2022.
- [2] F. Black and M. Scholes, “The pricing of options and corporate liabilities,” *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973. doi: [10.1086/260062](https://doi.org/10.1086/260062).
- [3] S. L. Heston, “A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options,” en, *The Review of Financial Studies*, vol. 6, no. 2, pp. 327–343, 1993. doi: [10.1093/rfs/6.2.327](https://doi.org/10.1093/rfs/6.2.327).
- [4] P. P. Boyle, “Options: A Monte Carlo approach,” *Journal of Financial Economics*, vol. 4, no. 3, pp. 323–338, 1977. doi: [10.1016/0304-405X\(77\)90005-8](https://doi.org/10.1016/0304-405X(77)90005-8).
- [5] P. Glasserman, *Monte Carlo Methods in Financial Engineering* (Stochastic Modelling and Applied Probability). Springer, 2004, vol. 53. doi: [10.1007/978-0-387-21617-1](https://doi.org/10.1007/978-0-387-21617-1).
- [6] S. Liu, A. Borovykh, L. A. Grzelak, and C. W. Oosterlee, “A neural network-based framework for financial model calibration,” en, *Journal of Mathematics in Industry*, vol. 9, no. 1, p. 9, Dec. 2019. doi: [10.1186/s13362-019-0066-7](https://doi.org/10.1186/s13362-019-0066-7). Accessed: Oct. 21, 2025. [Online]. Available: <https://mathematicsinindustry.springeropen.com/articles/10.1186/s13362-019-0066-7>.
- [7] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic Differentiation in Machine Learning: A Survey,” *Journal of Machine Learning Research*, vol. 18, no. 153, pp. 1–43, 2018. [Online]. Available: <http://jmlr.org/papers/v18/17-468.html>.
- [8] P. Carr and D. B. Madan, “Option valuation using the fast fourier transform,” *Journal of Computational Finance*, vol. 2, no. 4, pp. 61–73, 1999. doi: [10.21314/JCF.1999.043](https://doi.org/10.21314/JCF.1999.043).
- [9] F. Fang and C. W. Oosterlee, “A novel pricing method for european options based on fourier-cosine series expansions,” *SIAM Journal on Scientific Computing*, vol. 31, no. 2, pp. 826–848, 2009. doi: [10.1137/080718061](https://doi.org/10.1137/080718061).
- [10] D. Hainaut and A. Casas, “Option pricing in the heston model with physics inspired neural networks,” en, *Annals of Finance*, vol. 20, no. 3, pp. 353–376, 2024. doi: [10.1007/s10436-024-00452-7](https://doi.org/10.1007/s10436-024-00452-7). [Online]. Available: <https://doi.org/10.1007/s10436-024-00452-7>.
- [11] J. Yu, L. Lu, X. Meng, and G. E. Karniadakis, “Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems,” *Computer Methods in Applied Mechanics and Engineering*, vol. 393, p. 114 823, 2022. doi: [10.1016/j.cma.2022.114823](https://doi.org/10.1016/j.cma.2022.114823). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0045782522001438>.

- [12] K. A. Malandain, S. Kalici, and H. Chakhoyan, *DeepSVM: Learning stochastic volatility models with physics-informed deep operator networks*, 2025. doi: [10.48550/arXiv.2512.07162](https://doi.org/10.48550/arXiv.2512.07162). arXiv: [2512.07162](https://arxiv.org/abs/2512.07162) [q-fin.CP]. [Online]. Available: <https://arxiv.org/abs/2512.07162>.
- [13] S. Wang, Y. Teng, and P. Perdikaris, “Understanding and mitigating gradient flow pathologies in physics-informed neural networks,” *SIAM Journal on Scientific Computing*, vol. 43, no. 5, A3055–A3081, 2021.
- [14] S. Wang, X. Yu, and P. Perdikaris, “When and why PINNs fail to train: A neural tangent kernel perspective,” *Journal of Computational Physics*, vol. 449, p. 110 768, 2022. doi: [10.1016/j.jcp.2021.110768](https://doi.org/10.1016/j.jcp.2021.110768). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S002199912100663X>.
- [15] J. F. Urbán, P. Stefanou, and J. A. Pons, “Unveiling the optimization process of physics informed neural networks: How accurate and competitive can PINNs be?” *Journal of Computational Physics*, vol. 523, p. 113 656, Feb. 2025. doi: [10.1016/j.jcp.2024.113656](https://doi.org/10.1016/j.jcp.2024.113656). Accessed: Mar. 19, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0021999124009045>.
- [16] W. M. Czarnecki, S. Osindero, M. Jaderberg, G. Swirszcz, and R. Pascanu, “Sobolev Training for Neural Networks,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017. [Online]. Available: <https://papers.nips.cc/paper/7015-sobolev-training-for-neural-networks>.
- [17] G. Amici, M. Morandotti, and C. Zhang, *Calibrating the heston model with deep differential networks*, 2024. doi: [10.48550/arXiv.2407.15536](https://doi.org/10.48550/arXiv.2407.15536). arXiv: [2407.15536](https://arxiv.org/abs/2407.15536) [q-fin.CP]. [Online]. Available: <https://arxiv.org/abs/2407.15536>.
- [18] M. Noguer i Alonso, *Derivative-informed operator learning for finance: On-the-fly greeks, surfaces, hedging, and control*, 2026. doi: [10.48550/arXiv.2606.05900](https://doi.org/10.48550/arXiv.2606.05900). arXiv: [2606.05900](https://arxiv.org/abs/2606.05900) [q-fin.MF]. [Online]. Available: <https://arxiv.org/abs/2606.05900>.
- [19] B. Oksendal, *Stochastic Differential Equations: An Introduction with Applications*, 6th ed. Springer, 2003.
- [20] F. D. Rouah and S. L. Heston, *The Heston Model and its Extensions in Matlab and C#*. John Wiley & Sons, 2013.
- [21] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge University Press, 2007.
- [22] K. Levenberg, “A method for the solution of certain non-linear problems in least squares,” *Quarterly of Applied Mathematics*, vol. 2, no. 2, pp. 164–168, 1944. doi: [10.1090/qam/10666](https://doi.org/10.1090/qam/10666).

- [23] D. W. Marquardt, "An algorithm for least-squares estimation of nonlinear parameters," *Journal of the Society for Industrial and Applied Mathematics*, vol. 11, no. 2, pp. 431–441, 1963. doi: [10.1137/0111030](https://doi.org/10.1137/0111030).
- [24] M. D. McKay, R. J. Beckman, and W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output From a Computer Code," en, *Technometrics*, vol. 42, no. 1, pp. 55–61, Feb. 2000. doi: [10.1080/00401706.2000.10485979](https://doi.org/10.1080/00401706.2000.10485979). Accessed: Mar. 21, 2026. [Online]. Available: <http://www.tandfonline.com/doi/abs/10.1080/00401706.2000.10485979>.
- [25] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989. doi: [10.1007/BF02551274](https://doi.org/10.1007/BF02551274).
- [26] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991. doi: [10.1016/0893-6080\(91\)90009-T](https://doi.org/10.1016/0893-6080(91)90009-T).
- [27] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. Springer, 2006.
- [28] D. Guterding and W. Boenkost, "The Heston stochastic volatility model with piecewise constant parameters: Efficient calibration and pricing of window barrier options," *Journal of Computational and Applied Mathematics*, vol. 343, 2018. doi: [10.1016/j.cam.2018.04.054](https://doi.org/10.1016/j.cam.2018.04.054). [Online]. Available: <https://arxiv.org/abs/1805.04704>.
- [29] Y. Cui, S. del Baño Rollin, and G. Germano, "Full and fast calibration of the Heston stochastic volatility model," *European Journal of Operational Research*, vol. 263, no. 2, pp. 625–638, 2017. doi: [10.1016/j.ejor.2017.05.018](https://doi.org/10.1016/j.ejor.2017.05.018). [Online]. Available: <https://doi.org/10.1016/j.ejor.2017.05.018>.

A. ANALYSIS OF THE COS METHOD ERROR

This appendix complements the Fourier-COS method introduced in Section 2.7. Its purpose is only to make explicit how the COS reference error is controlled in the implementation, since the detailed pricing formula is already given in Chapter 2.

A.1. COS Error Control

Starting from the pricing integral in Equation (2.29), the COS method replaces the transition density by the truncated cosine expansion in Equations (2.27)–(2.28). Following Fang and Oosterlee [9], the numerical error can be read as the sum of three effects: truncating the integration domain to $[a, b]$, using only N cosine terms, and approximating the exact density coefficients by characteristic-function-based coefficients. In the shorter notation used in Equation (2.33),

$$|V - V_N^{\text{COS}}| \leq \varepsilon_{\text{trunc}}([a, b]) + \varepsilon_N.$$

For plain-vanilla European options, the payoff coefficients H_k in Equation (2.31) are available in closed form, so no separate payoff quadrature error is introduced.

The truncation interval is selected from cumulants of the log-price state. Using the cumulants obtained from Equation (2.11), the implementation follows the standard cumulant rule

$$[a, b] = \left[c_1 - L \sqrt{c_2 + \sqrt{c_4}}, c_1 + L \sqrt{c_2 + \sqrt{c_4}} \right],$$

which is the interval stated in Equation (2.32). Increasing L reduces probability-mass truncation error, but it also widens the interval and may require a larger N to preserve resolution. Therefore, L and N are treated jointly rather than as independent numerical knobs.

A.2. Reference Configuration

In the experiments, the reference configuration uses $N = 1500$ and $L = 50$, with fallback $L = 80$ in low-IV or otherwise unstable inversion cases. These settings are deliberately conservative because COS prices are used downstream as ANN labels, implied-volatility inversion inputs, and PINN benchmark references. The benchmark protocol in Section 5.1 therefore checks COS convergence with respect to (N, L) , put-call parity as stated in Equation (2.2), and Black–Scholes consistency in near-constant-variance regimes before neural-model errors are interpreted.

B. PINN PRICING BENCHMARK SETUP

Revisar si este apendice queda como setup de PINN baseline o si debe ampliarse para PINN Sobolev y PINN Sobolev+ACV. The pricing benchmark compares a baseline parametric PINN against the COS/Heston reference under a common evaluation protocol and shared numerical conventions.

The evaluated model is the baseline parametric PINN without Sobolev terms. Inputs follow the 8-dimensional convention

$$[\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r],$$

with affine standardization fitted on training statistics. Concretely, each feature x_j is transformed as

$$\tilde{x}_j = \frac{x_j - \mu_j^{\text{train}}}{\sigma_j^{\text{train}}},$$

where μ_j^{train} and σ_j^{train} are computed only on the training split and then reused unchanged in validation, test, and benchmark evaluation. This keeps features on comparable scales and stabilizes training and gradient computation, which is especially important in PINN settings; it also avoids data leakage from evaluation sets.

The evaluation sample contains $1.0 \cdot 10^4$ candidates, with $9.996 \cdot 10^3$ valid comparisons after numerical filtering. COS pricing is computed with $N = 1500$, $L = 50$, and fallback $L = 80$, under European put convention ($K = 1$) and no-arbitrage tolerance 10^{-8} . All reported metrics are computed in double precision on CPU.

C. SOFTWARE IMPLEMENTATION

This appendix documents the implementation layer used to generate the numerical results reported in the thesis. Its purpose is not to reproduce the full source code, but to make explicit the computational choices that affect reproducibility: the programming environment, the role of automatic differentiation, the separation between experiment scripts and reusable modules, and the repository in which the implementation was developed.

C.1. Computational Environment

All experiments were implemented in Python. The final neural workflows use PyTorch as the main deep-learning framework, together with NumPy, SciPy, pandas, scikit-learn, and matplotlib for numerical computation, data handling, evaluation, and figure generation. PyTorch is especially useful in this project because the same automatic differentiation engine supports both standard network training and derivative-based diagnostics. In the ANN branch, it is used to train the supervised implied-volatility surrogate and to compute derivatives of the learned surface when Greeks are evaluated. In the PINN branch, it is also used to differentiate the learned price surface with respect to the PDE variables and to build the residual terms that define the physics-informed loss.

The experiments were developed on Apple Silicon hardware with an Apple M4 Max chip, 36 GB of unified memory, and macOS 26.6. PyTorch device selection was handled automatically in the training scripts, using the Apple Metal backend (mps) when available and falling back to CPU otherwise. Numerical reference calculations and benchmark metrics that require double precision were evaluated on CPU when the selected backend did not support the required dtype.

The implementation follows the methodological split used in the thesis. The ANN/-CaNN branch is supervised and works mainly in implied-volatility space: synthetic labels are generated through the Heston-COS and Brent pipeline, the ANN surrogate is trained on those labels, and the frozen surrogate is then reused as a fast evaluator in the calibration problem. The PINN branch is implemented separately and works in price space: the baseline model is trained from the Heston PDE residual, terminal condition, and boundary conditions, while the Sobolev and Sobolev+ACV variants add derivative-aware corrections or analytic structure without turning the method into the ANN calibration pipeline.

C.2. Code Excerpts

Only short source-code fragments are included here, because the relevant methodological definitions are already given in Chapters 3 and 4. The objective of these excerpts is to

show where the mathematical operations described in the thesis enter the implementation.

CÓDIGO C.1. Sobolev derivative loss used in the ANN implied-volatility surrogate. The network is differentiated with respect to its normalized inputs and the resulting gradients are converted back to raw derivative units before comparison with the reference derivative targets.

```
1  x_req = x_model.detach().clone().requires_grad_(True)
2  y_model = model(x_req)
3
4  grad_model = torch.autograd.grad(
5      outputs=y_model,
6      inputs=x_req,
7      grad_outputs=torch.ones_like(y_model),
8      create_graph=True,
9      retain_graph=True,
10 ) [0]
11
12 idx = torch.as_tensor(
13     derivative_indices,
14     dtype=torch.long,
15     device=x_model.device,
16 )
17 x_std_t = torch.as_tensor(x_std, dtype=x_model.dtype, device=x_model
18     .device)
19 y_std_t = torch.as_tensor(y_std, dtype=x_model.dtype, device=x_model
20     .device)
21
22 raw_scale = y_std_t / torch.clamp(x_std_t[idx], min=1.0e-12)
23 pred_derivatives_raw = (
24     grad_model.index_select(dim=1, index=idx)
25     * raw_scale.reshape(1, -1)
26 )
27
28 target = target_derivatives_raw.to(dtype=x_model.dtype, device=
29     x_model.device)
30 scales = torch.clamp(derivative_scales.reshape(1, -1), min=1.0e-12)
31 loss = torch.mean(
32     ((pred_derivatives_raw - target) / scales) ** 2
33 )
```

Listing C.1 corresponds to the Sobolev term used by the ANN implied-volatility surrogate. The important implementation point is the change of units: if the network receives normalized inputs and, possibly, a normalized output, the derivative returned by automatic differentiation is not yet the physical derivative used in the loss. The factor $y_{\text{std}}/x_{\text{std},j}$ converts the derivative with respect to the model input back to the raw derivative direction j , after which each derivative component is normalized by its robust scale.

CÓDIGO C.2. Construction of the Heston PDE residual used by the baseline PINN. The code shows the moneyiness-coordinate version used in the price-space PINN loss, omitting input checks and optional residual normalization for clarity.

```

1  x_interior = x_interior.requires_grad_(True)
2  x_net = apply_input_affine(x_interior, input_affine)
3  u = model(x_net)
4
5  grads = gradient(y=u, x=x_interior)
6  u_tau = grads[:, 0:1]
7  u_m = grads[:, 1:2]
8  u_v = grads[:, 2:3]
9
10 grad_u_m = gradient(y=u_m, x=x_interior)
11 grad_u_v = gradient(y=u_v, x=x_interior)
12 u_mm = grad_u_m[:, 1:2]
13 u_mv = grad_u_m[:, 2:3]
14 u_vv = grad_u_v[:, 2:3]
15
16 m = x_interior[:, 1:2]
17 v = x_interior[:, 2:3]
18 rho = x_interior[:, 3:4]
19 kappa = x_interior[:, 4:5]
20 gamma = x_interior[:, 5:6]
21 bar_v = x_interior[:, 6:7]
22 r = x_interior[:, 7:8]
23
24 diffusion_m = 0.5 * v * (m**2) * u_mm
25 mixed = rho * gamma * v * m * u_mv
26 diffusion_v = 0.5 * (gamma**2) * v * u_vv
27 drift_m = r * m * u_m
28 drift_v = kappa * (bar_v - v) * u_v
29 discount = r * u
30
31 residual = (
32     u_tau
33     - diffusion_m
34     - mixed
35     - diffusion_v
36     - drift_m
37     - drift_v
38     + discount
39 )
40 loss_pde = torch.mean(residual**2)

```

Listing C.2 shows the corresponding construction for the baseline PINN. The tensor x_{interior} contains interior collocation points ordered as $(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$. The derivatives of the neural price surface are obtained with nested calls to automatic differentiation, and the residual is then assembled term by term according to the Heston PDE written in the moneyiness coordinate. In the actual training code, this residual can also be scaled locally

before computing the mean-square PDE loss, but the baseline mathematical structure is the one shown above.

C.3. Repository

The code used to generate datasets, train the neural models, run CaNN calibration experiments, evaluate Greeks, and reproduce the figures was maintained in a version-controlled repository during the development of the thesis. The repository contains the reusable modules, experiment scripts, configuration files, trained-run metadata, and plotting utilities associated with the reported benchmarks. The public repository is currently available at <https://github.com/jenriquezafra/TFM>.

D. GLOBAL NOTATION GLOSSARY

This appendix contains a global glossary of symbols used throughout the thesis. It is intended as a living reference and can be expanded as additional notation appears in later revisions.

TABLE D.1. GLOBAL NOTATION GLOSSARY: CORE PRICING
AND MODEL NOTATION

Symbol	Meaning
$(\Omega, \mathcal{F}, (\mathcal{F}_t), \mathbb{P})$	Filtered probability space: outcomes, measurable events, information available over time, and original probability measure.
t, T, τ	Valuation time, maturity, and time to maturity ($\tau = T - t$).
S_t, K, m	Spot price, strike, and moneyness ($m = S_0/K$ for quote grids, $m = S/K$ as a PDE coordinate).
r	Constant risk-free interest rate.
$\mathbb{Q}$	Risk-neutral probability measure.
V_t	Arbitrage-free option value at time t .
$V(t_0, x)$	Option value at valuation time t_0 when the state variable is x .
$V_{BS}, V_{\text{target}}$	Black–Scholes option value and target option value used in IV inversion.
Ψ, Ψ_{put}	Option payoff function at maturity; for the thesis pricing target, $\Psi_{\text{put}} = (K - S_T)^+$.
X_t, x, y	State variable and its initial/terminal values, typically $x = X(t_0)$ and $y = X(T)$.
F_X, f_X	Cumulative distribution function and probability density function of X .
θ	Generic model-parameter vector.
μ	Generic drift under the original probability measure.
$W_t^{(S)}, W_t^{(\nu)}$	Brownian motions driving the spot and variance processes.
$\kappa, \bar{\nu}, \gamma, \rho, \nu_0$	Heston parameters: mean reversion, long-run variance, vol-of-vol, correlation, and initial variance. Later chapters also use $\bar{\nu}$ and ν_0 for the variance parameters.

Source: Author's elaboration

TABLE D.2. GLOBAL NOTATION GLOSSARY: NEURAL
BENCHMARK NOTATION

Symbol	Meaning
ANN baseline, ANN Sobolev	Supervised implied-volatility surrogates used in the ANN branch. Both use the same input-output map; the Sobolev variant augments the baseline value loss with IV-derivative information.
CaNN baseline, CaNN Sobolev	Calibration workflows that use a frozen ANN surrogate as fast forward evaluator in the inverse Heston calibration problem. The two variants keep the same calibration objective and differ only in whether the evaluator is the ANN baseline or the ANN Sobolev surrogate.
f_{NN} , $\mathbf{x}$, $\mathbf{w}$, $\hat{\sigma}_{\text{imp}}$	ANN implied-volatility map, input vector, trainable weights and biases, and predicted implied volatility. In Chapter 3, $\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$.
$\hat{V}_{\text{ANN}}$, $\text{Greek}_{\text{ANN}}(q)$	Price and sensitivity obtained from the ANN predicted implied volatility by composing it with the Black–Scholes pricing convention and differentiating with respect to input direction q .
$\mathcal{L}_{\text{MSE}}$	Baseline ANN mean-squared-error loss in implied-volatility space.
$\mathcal{L}_{\text{Sob}}$, $\mathcal{L}_{\text{ANN-Sob}}$	Sobolev derivative loss for IV-derivative targets and total Sobolev ANN objective.
$\mathcal{J}$, $d_{i,q}$, s_q	Sobolev derivative directions, reference IV derivative target at anchor point i in direction q , and robust scale used to normalize that derivative direction.
B , B_{Sob} , λ_{Sob}	Baseline mini-batch size, Sobolev mini-batch size, and weight of the Sobolev derivative loss in the ANN Sobolev objective.
PINN baseline, PINN Sobolev, PINN Sobolev+ACV	Physics-informed price-surface variants. The baseline PINN is the PDE-trained pricing model; the Sobolev variant adds derivative targets; the ACV variant adds an analytic control-variate correction for the hard short-maturity ATM region.
u_ϕ	Neural price surface produced by the PINN, with trainable parameters ϕ .
$\mathcal{R}_\phi$	Heston PDE residual evaluated on the PINN output.
$\mathcal{L}_{\text{PDE}}$, $\mathcal{L}_{\text{term}}$, $\mathcal{L}_{\text{low}}$	Baseline PINN loss terms for the interior PDE residual, terminal payoff, and lower-boundary

TABLE D.3. GLOBAL NOTATION GLOSSARY: TRANSFORM,
COS, AND IV NOTATION

Symbol	Meaning
$\phi_X(u)$	Characteristic function of random variable X .
$\mathcal{M}_X(u), \zeta_X(u), \zeta_k$	Moment generating function, cumulant characteristic function, and k -th cumulant.
u, u_k, i	Fourier variable, COS frequency $u_k = k\pi/(b - a)$, and imaginary unit.
$[a, b], L, N, k$	COS truncation interval, truncation scaling parameter, number of terms, and series index.
$\bar{F}_k, H_k, U_k$	COS density, payoff, and Heston-COS payoff coefficients.
$\hat{V}_T(u), V_N^{\text{COS}}$	Fourier transform of the maturity payoff and N -term COS price approximation.
φ_H, C, D, d, g	Heston characteristic function and its affine auxiliary functions.
χ_k, ψ_k	Closed-form payoff integrals used in COS coefficients.
σ_{imp}	Implied volatility obtained from numerical inversion.
$F(\sigma), \varepsilon_{\text{price}}, \varepsilon_{\sigma}, n_{\text{max}}$	IV root function, price tolerance, volatility tolerance, and maximum number of solver iterations.
$\hat{z}_i, z_i^{\text{ref}}$	Model and reference values for a generic benchmark target at evaluation point i .
MSE, RMSE, MAPE	Mean squared error, root mean squared error, and mean absolute percentage error used for benchmark reporting.
Π_j, Ψ_j	Heston call-pricing probability terms and their Fourier integrands.
Delta, Gamma, Vega Rho, Theta	Standard option Greeks: spot first derivative, spot second derivative, variance/volatility sensitivity, interest-rate sensitivity, and time sensitivity.

Source: Author's elaboration